

UNIVERSIDADE FEDERAL DE SÃO JOÃO DEL-REI

Departamento de Engenharia de Telecomunicações

Thierry Gonçalves Braga

**PIPELINE MODULAR PARA DETECÇÃO DE  
ÁUDIO SINTÉTICO  
USANDO ARQUITETURAS NEURAIIS HÍBRIDAS  
E  
ANÁLISE EMPÍRICA DE CARACTERÍSTICAS**

São João del-Rei — MG

2026

Thierry Gonçalves Braga

**PIPELINE MODULAR PARA DETECÇÃO DE  
ÁUDIO SINTÉTICO  
USANDO ARQUITETURAS NEURAIS HÍBRIDAS  
E  
ANÁLISE EMPÍRICA DE CARACTERÍSTICAS**

Trabalho de Conclusão de Curso apresentado ao Departamento de Engenharia de Telecomunicações da Universidade Federal de São João del-Rei, como requisito parcial para a obtenção do título de Bacharel em Engenharia de Telecomunicações.

Orientadora: Prof.<sup>a</sup> Ana Cláudia

Braga, Thierry Gonçalves

Pipeline modular para detecção de áudio sintético usando arquiteturas neurais híbridas e análise empírica de características / Thierry Gonçalves Braga. — São João del-Rei, 2026.

<n.> f.: il.

Orientadora: <nome completo da orientadora>.

Trabalho de Conclusão de Curso (Graduação em Engenharia de Telecomunicações) — Universidade Federal de São João del-Rei, 2026.

1. Detecção de áudio sintético. 2. *Anti-spoofing*. 3. Aprendizado profundo. 4. Processamento de fala. 5. Biometria de voz. I. <sobrenome>, <nome>, orient. II. Título.

CDU: <a completar pela Biblioteca>

## FOLHA DE APROVAÇÃO

Trabalho de Conclusão de Curso defendido e aprovado em \_\_\_\_\_, como requisito parcial para obtenção do título de Bacharel em Engenharia de Telecomunicações.

### Banca Examinadora:

---

Prof.<sup>a</sup> Ana Cláudia (Orientadora)  
Universidade Federal de São João del-Rei

---

Prof. \_\_\_\_\_  
Instituição: \_\_\_\_\_

---

Prof. \_\_\_\_\_  
Instituição: \_\_\_\_\_

## RESUMO

Este trabalho apresenta o desenvolvimento e a avaliação de um *pipeline* modular e de código aberto para detecção de áudio sintético, em resposta à proliferação de *deepfakes* de voz. A metodologia integra pré-processamento acústico (detecção de atividade de voz e controle automático de ganho), extração de características heterogêneas — áudio bruto, representações tempo-frequência (espectrogramas Mel) e um vetor tabular de descritores temporais, cepstrais (MFCC) e espectro-perceptuais (RASTA-PLP) —, treinamento supervisionado, inferência padronizada e geração automática de relatórios de *benchmark*. A versão experimental consolidada avalia 11 modelos reconhecidos na literatura, abrangendo modelos auto-supervisionados (SSL), redes de áudio bruto, grafos de atenção, *Transformers* espectrais, redes convolucionais multi-escala e classificadores estatísticos clássicos. Os experimentos utilizaram um *corpus* pareado e balanceado de 15.000 amostras de fala em português (CETUC com clones XTTS-v2), particionado por disjunção dupla de locutor e de frase (12.162/1456/1382) e com janelas monofônicas padronizadas em 16 kHz e 3 s. Os modelos neurais foram treinados em perfil GPU (WSL2/CUDA), com métricas de inferência extraídas no ambiente de execução de cada modelo (CPU ou GPU, conforme a família); SVM e *Random Forest* foram otimizados por validação cruzada. No conjunto de teste limpo, AST e CCT obtiveram 99,71% e 99,57% de acurácia (EER de 0,14% e 0,43%), seguidos por Conformer (98,91%) e Res2Net (97,76%); os testes pareados com correção de Holm não separam AST, CCT e Conformer, que devem ser lidos como empate técnico. Sob ruído aditivo a 10 dB de SNR o ordenamento se reorganiza: Res2Net lidera com 92,40%, seguido de AASIST (91,68%) e Conformer (91,46%), enquanto AST cai para 88,21% e RawGAT-ST para 77,21% — evidenciando que acurácia em distribuição fechada é indicador insuficiente de qualidade e que *nem* a expressividade da representação de entrada *nem* a capacidade do modelo explicam a dissociação: AASIST e RawGAT-ST compartilham as duas e separam-se por 14,5 pontos percentuais, sendo a estabilidade de treino o fator que discrimina. No nível *não visto* de 5 dB, ausente do *augmentation* de treino, o SVM colapsa para 58,68% de acurácia mantendo AUC-ROC de 0,884: o que falha é o ponto de operação no limiar fixo, não a capacidade de ordenar. Os resultados confirmam a funcionalidade do *pipeline* para comparação padronizada de arquiteturas sob um protocolo de partição auditado (sobreposição nula de locutor, frase e conteúdo entre as partições), mas também evidenciam a necessidade de validação externa, robustez a degradações reais de canal (codecs, reverberação, ruído não estacionário) e generalização entre bases antes do uso em cenários reais.

**Palavras-chave:** *deepfake* de áudio; *anti-spoofing*; arquiteturas neurais; extração de características; *min* t-DCF; robustez a ruído.

## ABSTRACT

This work presents the development and evaluation of a modular open-source pipeline for synthetic audio detection, addressing the proliferation of voice deepfakes. The methodology integrates acoustic preprocessing (voice activity detection and automatic gain

control), extraction of heterogeneous features — raw audio, time–frequency representations (Mel spectrograms), and a tabular vector of temporal, cepstral (MFCC), and spectro-perceptual (RASTA-PLP) descriptors —, supervised training, standardized inference, and automatic benchmark report generation. The consolidated experimental version evaluates 11 models recognized in the literature, spanning self-supervised (SSL) models, raw-audio networks, graph-attention networks, spectral Transformers, multi-scale convolutional networks, and classical statistical classifiers. Experiments used a paired, balanced 15,000-sample Portuguese speech corpus (CETUC with XTTS-v2 clones), partitioned by double speaker **and** sentence disjunction (12,162/1,456/1,382), with 16 kHz mono 3 s windows. Neural models were trained on a GPU profile (WSL2/CUDA), with inference metrics extracted in each model’s execution environment (CPU or GPU, depending on the family); SVM and Random Forest were optimized through cross-validation. On the clean test set, AST and CCT achieved 99.71% and 99.57% accuracy (EER of 0.14% and 0.43%), followed by Conformer (98.91%) and Res2Net (97.76%); paired tests with Holm correction do not separate AST, CCT and Conformer, which should be read as a statistical tie. Under additive noise at 10 dB SNR the ordering reorganizes: Res2Net leads with 92.40%, followed by AASIST (91.68%) and Conformer (91.46%), while AST drops to 88.21% and RawGAT-ST to 77.21% — showing that closed-distribution accuracy is an insufficient quality indicator, and that neither input representation nor model capacity explains the dissociation: AASIST and RawGAT-ST share both and differ by 14.5 percentage points, the discriminating factor being training stability. At the *unseen* 5 dB level, absent from training augmentation, SVM collapses to 58.68% accuracy while retaining an AUC-ROC of 0.884: what fails is the fixed-threshold operating point, not the ability to rank. The results confirm the pipeline’s ability to compare heterogeneous architectures in a standardized way, while also showing the need for external validation, robustness to real channel degradations (codecs, reverberation, non-stationary noise), and cross-dataset generalization before real-world deployment.

**Keywords:** audio deepfake; anti-spoofing; neural architectures; feature extraction; min t-DCF; noise robustness.

## Lista de Figuras

|    |                                                                                                                                                                                                                                                                                                                                                                                                                 |    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Diagrama do <i>pipeline</i> proposto. O modelo treinado (b) é implantado ( <i>deploy</i> ) na fase de inferência (a). A pontuação $p_{\text{fake}}$ é a probabilidade predita de a amostra ser sintética: valores acima do limiar de decisão $\theta$ resultam em SINTÉTICO. No <i>benchmark</i> , $\theta = 0,5$ para os 11 modelos, sem exceção (política <code>fixed_comparison</code> ; Seção 5.5). . . . . | 33 |
| 2  | Acurácia e AUC-ROC no conjunto de teste limpo. . . . .                                                                                                                                                                                                                                                                                                                                                          | 55 |
| 3  | Robustez a AWGN na forma de onda (acurácia vs. SNR). . . . .                                                                                                                                                                                                                                                                                                                                                    | 56 |
| 4  | Taxa de Erro Igual (EER) por arquitetura. . . . .                                                                                                                                                                                                                                                                                                                                                               | 57 |
| 5  | $t$ -DCF* por arquitetura. . . . .                                                                                                                                                                                                                                                                                                                                                                              | 58 |
| 6  | Latência e tamanho dos modelos. . . . .                                                                                                                                                                                                                                                                                                                                                                         | 83 |
| 7  | Trade-off entre acurácia, latência e tamanho dos artefatos. . . . .                                                                                                                                                                                                                                                                                                                                             | 84 |
| 8  | Resumo descritivo das curvas de validação em execução única; a diferença pico-final não demonstra, isoladamente, estabilidade de otimização. Modelos clássicos não possuem trajetória por época. . . . .                                                                                                                                                                                                        | 84 |
| 9  | Matrizes de confusão dos onze modelos avaliados no artigo. . . . .                                                                                                                                                                                                                                                                                                                                              | 85 |
| 10 | Importância por permutação das 15 características mais relevantes do <i>Random Forest</i> , coloridas por família (temporal, LFCC, MFCC, RASTA-PLP), medida na partição de teste com 30 repetições. Agregadas, as famílias respondem por 44,7% (temporal), 41,6% (LFCC), 8,7% (MFCC) e 5,0% (RASTA-PLP) da importância total. . . . .                                                                           | 85 |
| 11 | Distribuições da pontuação por classe para AASIST e RawGAT-ST . . . .                                                                                                                                                                                                                                                                                                                                           | 86 |
| 12 | Curvas DET dos onze modelos no conjunto de teste limpo . . . . .                                                                                                                                                                                                                                                                                                                                                | 86 |
| 13 | Arquitetura em camadas do sistema . . . . .                                                                                                                                                                                                                                                                                                                                                                     | 91 |

## Lista de Tabelas

|    |                                                                                                                                                                                                                                                                                                                                                    |    |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Principais modalidades de geração e manipulação de fala sintética. . . . .                                                                                                                                                                                                                                                                         | 21 |
| 2  | <i>Survey</i> das gerações de métodos de detecção de áudio sintético. Os desempenhos citados referem-se aos protocolos em que foram originalmente publicados — majoritariamente o ASVspoof — e não são diretamente transponíveis para o protocolo deste trabalho, que usa outro idioma, outro gerador e outro particionamento (Seção 6.4). . . . . | 22 |
| 3  | Posicionamento em relação aos protocolos de avaliação existentes. Os eixos em que este trabalho é mais fraco (gerador e idioma únicos) e mais forte (controle de confundidores, auditoria de partição) são complementares, e não concorrentes. . . . .                                                                                             | 25 |
| 4  | Taxonomia bidimensional do recorte: representação de entrada $\times$ regime de treinamento. As subseções seguintes seguem as linhas desta tabela. . . .                                                                                                                                                                                           | 26 |
| 5  | Representação de entrada consumida por família de modelo. . . . .                                                                                                                                                                                                                                                                                  | 36 |
| 6  | Descritores acústicos disponíveis no <i>pipeline</i> : síntese da literatura. <b>Custo Computacional</b> : estimativa qualitativa (Baixo/Médio/Alto) do consumo de tempo e memória para extração. . . . .                                                                                                                                          | 40 |
| 7  | Composição do <i>corpus</i> pareado (15.000 amostras) e auditorias de disjunção entre partições. Os valores de sobreposição são recontados a cada execução e gravados no artefato do <i>benchmark</i> . . . . .                                                                                                                                    | 41 |
| 8  | Especificações técnicas do protocolo de pré-processamento acústico e partições de amostragem. . . . .                                                                                                                                                                                                                                              | 42 |
| 9  | Correspondência entre a convenção de classificação (classe positiva = <i>spoof</i> ) e a convenção ASV/ <i>t</i> -DCF herdada do ASVspoof. . . . .                                                                                                                                                                                                 | 45 |
| 10 | Especificações de infraestrutura, hardware e parametrização do ambiente de <i>benchmarking</i> . . . . .                                                                                                                                                                                                                                           | 50 |
| 11 | Custo de treinamento por modelo, na RTX 3060 (neurais e SSL) ou em CPU multi- <i>core</i> (clássicos). <i>Ajuste</i> é o tempo do treinamento; <i>Total</i> inclui preparação e avaliação. . . . .                                                                                                                                                 | 51 |
| 12 | Resultados consolidados no conjunto de teste limpo. . . . .                                                                                                                                                                                                                                                                                        | 53 |
| 13 | Eficiência computacional dos modelos consolidados. . . . .                                                                                                                                                                                                                                                                                         | 53 |
| 14 | Robustez a ruído AWGN, aplicada à forma de onda antes dos frontends, por modelo consolidado. Os níveis de 30, 20 e 10 dB são <b>casados</b> com o <i>augmentation</i> de treino; 5 dB é <b>não visto</b> . A última coluna traz a acurácia no PIOR dos 11 locutores do teste (nenhum visto no treino), medida em áudio limpo. . . . .              | 54 |
| 15 | Resumo descritivo das curvas de validação em execução única. . . . .                                                                                                                                                                                                                                                                               | 54 |

|    |                                                                                                                                                                                                                           |    |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 16 | Os dois modelos diagnosticados como instáveis e sua posição na robustez a 10 dB. A instabilidade de validação, isoladamente, não ordena o recorte. . .                                                                    | 61 |
| 17 | Ablação do pré-treinamento no AST. Única variável alterada: a transferência de pesos AudioSet. Execução de 100 épocas (12,15 h), classificada como estável pelo diagnóstico do <i>pipeline</i> . . . . .                  | 63 |
| 18 | Avaliação cega de SVM e <i>Random Forest</i> sobre partição fresca ( $n = 4916$ ), contra os valores publicados no conjunto de teste de 1382 amostras. Nenhum retreinamento: apenas o conjunto de avaliação muda. . . . . | 65 |
| 19 | Latência de inferência dos 11 modelos no <i>mesmo</i> dispositivo (CPU), contra os valores publicados em três <i>runtimes</i> distintos. A coluna <i>Fator</i> é a razão entre as duas medidas. . . . .                   | 68 |
| 20 | Ambiente de software e parâmetros de reprodução das execuções. . . . .                                                                                                                                                    | 87 |
| 21 | Hiperparâmetros efetivos por modelo . . . . .                                                                                                                                                                             | 88 |
| 22 | Correspondência entre o nome usado no texto, a entrada do <i>benchmark</i> e o módulo de implementação. . . . .                                                                                                           | 88 |
| 23 | Seções da interface interativa e telas que cada uma agrega. . . . .                                                                                                                                                       | 92 |
| 24 | Módulos reais do <i>pipeline</i> referenciados pelo <i>benchmark</i> . . . . .                                                                                                                                            | 94 |

## Lista de Abreviaturas e Siglas

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <b>AASIST</b>     | <i>Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks</i> |
| <b>AGC</b>        | <i>Automatic Gain Control</i> (Controle Automático de Ganho)                          |
| <b>AM-Softmax</b> | <i>Additive Margin Softmax</i>                                                        |
| <b>AST</b>        | <i>Audio Spectrogram Transformer</i>                                                  |
| <b>ASV</b>        | <i>Automatic Speaker Verification</i> (Verificação Automática de Locutor)             |
| <b>AUC-ROC</b>    | <i>Area Under the Receiver Operating Characteristic Curve</i>                         |
| <b>AWGN</b>       | <i>Additive White Gaussian Noise</i> (Ruído Gaussiano Branco Aditivo)                 |
| <b>CCT</b>        | <i>Compact Convolutional Transformer</i>                                              |
| <b>CETUC</b>      | Centro de Estudos em Telecomunicações da PUC-Rio ( <i>corpus</i> de fala)             |
| <b>CM</b>         | <i>Countermeasure</i> (contramedida; o detector <i>anti-spoofing</i> )                |
| <b>CNN</b>        | <i>Convolutional Neural Network</i> (Rede Neural Convolucional)                       |
| <b>CPU</b>        | <i>Central Processing Unit</i>                                                        |
| <b>CQCC</b>       | <i>Constant-Q Cepstral Coefficients</i>                                               |
| <b>CQT</b>        | <i>Constant-Q Transform</i> (Transformada de Q Constante)                             |
| <b>CUDA</b>       | <i>Compute Unified Device Architecture</i>                                            |
| <b>CV</b>         | <i>Cross-Validation</i> (Validação Cruzada)                                           |
| <b>dBFS</b>       | <i>decibels relative to Full Scale</i> (decibéis referidos ao fundo de escala)        |
| <b>DCT</b>        | <i>Discrete Cosine Transform</i> (Transformada Discreta do Cosseno)                   |
| <b>DET</b>        | <i>Detection Error Tradeoff</i>                                                       |
| <b>ECE</b>        | <i>Expected Calibration Error</i> (Erro Esperado de Calibração)                       |
| <b>EER</b>        | <i>Equal Error Rate</i> (Taxa de Erro Igual)                                          |
| <b>F0</b>         | Frequência Fundamental                                                                |
| <b>FFT</b>        | <i>Fast Fourier Transform</i> (Transformada Rápida de Fourier)                        |
| <b>FMS</b>        | <i>Feature Map Scaling</i> (Escalamento de Mapas de Características)                  |
| <b>FNR</b>        | <i>False Negative Rate</i> (Taxa de Falsos Negativos)                                 |
| <b>FPR</b>        | <i>False Positive Rate</i> (Taxa de Falsos Positivos)                                 |
| <b>GAT</b>        | <i>Graph Attention Network</i> (Rede de Atenção em Grafos)                            |
| <b>GMM-UBM</b>    | <i>Gaussian Mixture Model – Universal Background Model</i>                            |
| <b>GPU</b>        | <i>Graphics Processing Unit</i>                                                       |
| <b>GRU</b>        | <i>Gated Recurrent Unit</i>                                                           |
| <b>HNR</b>        | <i>Harmonic-to-Noise Ratio</i> (Relação Harmônico-Ruído)                              |
| <b>HS-GAL</b>     | <i>Heterogeneous Stacking Graph Attention Layer</i>                                   |

|                  |                                                                                             |
|------------------|---------------------------------------------------------------------------------------------|
| <b>LA</b>        | <i>Logical Access</i> (partição de acesso lógico do ASVspoof)                               |
| <b>LCNN</b>      | <i>Light CNN</i>                                                                            |
| <b>LFCC</b>      | <i>Linear Frequency Cepstral Coefficients</i> (Coeficientes Cepstrais em Frequência Linear) |
| <b>LSTM</b>      | <i>Long Short-Term Memory</i>                                                               |
| <b>MDI</b>       | <i>Mean Decrease in Impurity</i> (Redução Média de Impureza)                                |
| <b>MFCC</b>      | <i>Mel-Frequency Cepstral Coefficients</i> (Coeficientes Cepstrais na Escala Mel)           |
| <b>MLP</b>       | <i>Multilayer Perceptron</i> (Perceptron de Múltiplas Camadas)                              |
| <b>PCM</b>       | <i>Pulse-Code Modulation</i> (Modulação por Código de Pulso)                                |
| <b>RASTA-PLP</b> | <i>RelAtive SpecTrA – Perceptual Linear Prediction</i>                                      |
| <b>RBF</b>       | <i>Radial Basis Function</i> (Função de Base Radial)                                        |
| <b>RMS</b>       | <i>Root Mean Square</i> (Valor Quadrático Médio)                                            |
| <b>RVC</b>       | <i>Retrieval-based Voice Conversion</i>                                                     |
| <b>SHAP</b>      | <i>SHapley Additive exPlanations</i>                                                        |
| <b>SNR</b>       | <i>Signal-to-Noise Ratio</i> (Relação Sinal-Ruído)                                          |
| <b>SSL</b>       | <i>Self-Supervised Learning</i> (Aprendizado Auto-Supervisionado)                           |
| <b>STFT</b>      | <i>Short-Time Fourier Transform</i> (Transformada de Fourier de Curto Termo)                |
| <b>SUPERB</b>    | <i>Speech processing Universal PERformance Benchmark</i>                                    |
| <b>SVM</b>       | <i>Support Vector Machine</i> (Máquina de Vetores de Suporte)                               |
| <b>t-DCF</b>     | <i>Tandem Detection Cost Function</i> (Função de Custo de Detecção em Tandem)               |
| <b>TPR</b>       | <i>True Positive Rate</i> (Taxa de Verdadeiros Positivos)                                   |
| <b>TTS</b>       | <i>Text-to-Speech</i> (Síntese de Fala)                                                     |
| <b>VAD</b>       | <i>Voice Activity Detection</i> (Detecção de Atividade de Voz)                              |
| <b>VC</b>        | <i>Voice Conversion</i> (Conversão de Voz)                                                  |
| <b>ViT</b>       | <i>Vision Transformer</i>                                                                   |
| <b>VRAM</b>      | <i>Video Random Access Memory</i> (memória da GPU)                                          |
| <b>XTTS</b>      | <i>Cross-lingual Text-to-Speech</i>                                                         |
| <b>ZCR</b>       | <i>Zero-Crossing Rate</i> (Taxa de Cruzamento por Zero)                                     |

# Sumário

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                           | <b>15</b> |
| 1.1      | Biometria de Voz: Contexto de Aplicação . . . . .           | 16        |
| 1.2      | Objetivos . . . . .                                         | 17        |
| 1.2.1    | Objetivo Geral . . . . .                                    | 17        |
| 1.2.2    | Objetivos Específicos . . . . .                             | 17        |
| 1.3      | Questões de Pesquisa . . . . .                              | 17        |
| 1.4      | Organização do Trabalho . . . . .                           | 18        |
| <b>2</b> | <b>Tecnologias de Síntese de Voz</b>                        | <b>19</b> |
| 2.1      | Da Síntese Clássica à Síntese Neural . . . . .              | 19        |
| 2.2      | Text-to-Speech, Clonagem e Conversão de Voz . . . . .       | 19        |
| 2.3      | Implicações para a Detecção . . . . .                       | 21        |
| <b>3</b> | <b>Trabalhos Relacionados</b>                               | <b>24</b> |
| 3.1      | Campanhas de Avaliação . . . . .                            | 24        |
| 3.2      | Recursos para o Português . . . . .                         | 24        |
| 3.3      | Generalização e Correlações Espúrias . . . . .              | 24        |
| 3.4      | Posicionamento deste Trabalho . . . . .                     | 25        |
| <b>4</b> | <b>Arquiteturas de Classificação</b>                        | <b>26</b> |
| 4.1      | Duas Dimensões de Classificação . . . . .                   | 26        |
| 4.2      | Modelos de Áudio Bruto . . . . .                            | 27        |
| 4.2.1    | WavLM . . . . .                                             | 27        |
| 4.2.2    | HuBERT . . . . .                                            | 28        |
| 4.2.3    | RawNet2 . . . . .                                           | 28        |
| 4.2.4    | AASIST . . . . .                                            | 29        |
| 4.2.5    | RawGAT-ST . . . . .                                         | 29        |
| 4.3      | Arquiteturas Neurais de Domínio Espectro-Temporal . . . . . | 29        |
| 4.3.1    | Conformer . . . . .                                         | 29        |
| 4.3.2    | Compact Convolutional Transformer (CCT) . . . . .           | 30        |
| 4.3.3    | Audio Spectrogram Transformer (AST) . . . . .               | 30        |
| 4.3.4    | Res2Net (Topologia Multi-escala) . . . . .                  | 31        |
| 4.4      | Classificadores Estatísticos Clássicos . . . . .            | 31        |
| 4.4.1    | Support Vector Machine (SVM) . . . . .                      | 31        |
| 4.4.2    | Random Forest . . . . .                                     | 31        |

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Metodologia</b>                                                        | <b>33</b> |
| 5.1      | Visão Geral do Pipeline . . . . .                                         | 33        |
| 5.2      | Pré-processamento: Normalização e Detecção de Atividade de Voz . . . . .  | 34        |
| 5.3      | Representações de Entrada e Características Acústicas . . . . .           | 36        |
| 5.4      | Conjunto de Dados e Protocolo Experimental . . . . .                      | 40        |
| 5.4.1    | Composição e Proveniência . . . . .                                       | 40        |
| 5.4.2    | Pré-processamento e Particionamento . . . . .                             | 42        |
| 5.4.3    | Controles Contra Vazamento e Escopo de Validade . . . . .                 | 43        |
| 5.5      | Métricas de Avaliação . . . . .                                           | 45        |
| 5.6      | Ferramentas de Software Livre . . . . .                                   | 48        |
| 5.7      | Protocolo de Treinamento . . . . .                                        | 49        |
| <b>6</b> | <b>Experimentos e Resultados</b>                                          | <b>50</b> |
| 6.1      | Ambiente Experimental e Hardware . . . . .                                | 50        |
| 6.2      | Custo de Treinamento . . . . .                                            | 51        |
| 6.3      | Resultados Consolidados . . . . .                                         | 53        |
| 6.4      | Análise dos Resultados . . . . .                                          | 54        |
| 6.4.1    | Desempenho no Conjunto Limpo . . . . .                                    | 54        |
| 6.4.2    | A Previsão sobre Artefatos de Fase não se Confirma . . . . .              | 56        |
| 6.4.3    | Importância de Características no Vetor Tabular . . . . .                 | 58        |
| 6.4.4    | Robustez a Ruído: o Achado Central . . . . .                              | 59        |
| 6.4.5    | O SNR Não Visto de 5 dB . . . . .                                         | 62        |
| 6.4.6    | Ablação do AST: Quanto Vale o Pré-Treinamento . . . . .                   | 63        |
| 6.4.7    | Quanto Custa a Não Cegueira dos Classificadores Clássicos . . . . .       | 64        |
| 6.4.8    | Dispersão entre Locutores Não Vistos . . . . .                            | 65        |
| 6.4.9    | Ponto de Operação: AUC, EER e t-DCF . . . . .                             | 66        |
| 6.4.10   | Vazamento de Domínio e Separabilidade . . . . .                           | 67        |
| 6.4.11   | Eficiência Computacional e Trade-offs . . . . .                           | 68        |
| 6.4.12   | Estabilidade de Treinamento . . . . .                                     | 70        |
| 6.4.13   | Ressalva sobre Execução Única . . . . .                                   | 70        |
| <b>7</b> | <b>Conclusões</b>                                                         | <b>72</b> |
| 7.1      | Principais Contribuições . . . . .                                        | 74        |
| 7.2      | Limitações . . . . .                                                      | 75        |
| 7.3      | Trabalhos Futuros . . . . .                                               | 76        |
| <b>A</b> | <b>Configurações Detalhadas do Pipeline</b>                               | <b>87</b> |
| A.1      | Hiperparâmetros Consolidados . . . . .                                    | 87        |
| A.2      | Correspondência entre Nomes Acadêmicos e Módulos do Repositório . . . . . | 88        |
| A.3      | Descritores Acústicos Adicionais Disponíveis no Pipeline . . . . .        | 89        |

---

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| A.4      | Organização dos Artefatos . . . . .             | 91        |
| <b>B</b> | <b>Sistema Desenvolvido</b>                     | <b>91</b> |
| B.1      | Arquitetura em Camadas . . . . .                | 91        |
| B.2      | Interfaces de Acesso . . . . .                  | 92        |
| B.3      | Organização da Interface Interativa . . . . .   | 92        |
| B.4      | Implantação . . . . .                           | 93        |
| B.5      | Delimitação . . . . .                           | 93        |
| <b>C</b> | <b>Implementação de Referência</b>              | <b>94</b> |
| <b>D</b> | <b>Comandos de Execução e Reprodutibilidade</b> | <b>94</b> |

# 1 Introdução

A síntese de fala, ou *Text-to-Speech* (TTS), evoluiu de sistemas mecânicos rudimentares para modelos computacionais capazes de reproduzir, com alta fidelidade, as nuances, a emoção e a identidade de um locutor específico — processo hoje conhecido como clonagem de voz [1]. Os primeiros marcos da síntese eletrônica surgiram com os *Vocoders* (*Voice Coders*) de Homer Dudley, na década de 1930 [2]. Baseados no modelo fonte-filtro de produção de fala, esses dispositivos separavam a excitação (a fonte, como as cordas vocais) das características do trato vocal (o filtro). Embora revolucionários e fundamentais para a compressão em telecomunicações, os sistemas puramente paramétricos produziam vozes de aspecto robótico, carentes de naturalidade [3].

Para superar essa limitação, a década de 1990 foi marcada pela síntese concatenativa, que se baseia na gravação de extensos bancos de voz humana segmentados em pequenas unidades fonéticas (difones ou sílabas) [4]. A verdadeira quebra de paradigma, contudo, ocorreu na última década com o advento do aprendizado profundo (*Deep Learning*). Modelos neurais generativos, como o WaveNet [5] e o Tacotron [6], substituíram os *pipelines* tradicionais por arquiteturas de ponta a ponta (*end-to-end*), aprendendo o mapeamento direto entre texto (ou fonemas) e a forma de onda acústica, capturando variações prosódicas complexas. É nesse cenário que a clonagem de voz se torna viável e acessível: por meio de adaptação de locutor e aprendizado *few-shot* ou *zero-shot*, tornou-se possível sintetizar a voz de um indivíduo com poucos segundos de áudio de referência [1, 7].

A crescente democratização dessas tecnologias tem facilitado a criação de *deepfakes* de áudio com qualidade cada vez mais próxima à voz humana natural [8, 9]. Sistemas neurais como Tacotron 2 [10] e WaveNet [5] estabeleceram o patamar de naturalidade da fala sintética, enquanto abordagens de clonagem *few-shot* [1, 7] e sistemas de conversão de voz em tempo real, como o RVC (*Retrieval-based Voice Conversion*), cuja produção já motiva detectores dedicados [11], replicam uma voz-alvo com poucos segundos de áudio de referência. Embora benéfica para aplicações legítimas (assistentes virtuais, dublagem automatizada), essa evolução tem sido explorada maliciosamente em fraudes financeiras, manipulação política e chantagem [12, 13]. No contexto brasileiro, o DFRLab identificou 78 casos confirmados ou alegados de *deepfakes* políticos entre maio e outubro de 2024, período das eleições municipais [13], além de documentar os desafios de identificação desse conteúdo no período pré-eleitoral [14] — evidenciando a necessidade de sistemas de detecção robustos voltados ao português brasileiro.

Estudos perceptuais indicam que humanos identificam corretamente *deepfakes* de áudio em cerca de 73% dos casos (e fala genuína em apenas 68%), com ganhos apenas marginais mesmo após familiarização com exemplos [8]. Esse teto perceptual humano justifica o desenvolvimento de sistemas automatizados e explica o surgimento de *benchmarks* padronizados como o ASVspoof [15–17] e o desafio ADD [18], hoje eixos centrais

de avaliação.

## 1.1 Biometria de Voz: Contexto de Aplicação

A biometria de voz — autenticação por características vocais individuais — é um dos principais contextos de aplicação para detectores de áudio sintético, e sua adoção crescente amplia a superfície de ataque dos *deepfakes* de voz. Relatórios do setor indicam expansão sustentada da adoção — e, simultaneamente, dos ataques com áudio sintético [12] —, puxada por instituições que lidam com alto volume de transações críticas remotas: o setor financeiro (autenticação passiva durante o atendimento), o de telecomunicações (mitigação de fraudes como o *SIM swapping*), o de saúde (acesso a prontuários eletrônicos) e o setor público (concessão de benefícios e controle de acesso). Nesses cenários, um sistema de verificação de locutor opera tipicamente em *tandem* com uma contramedida anti-*spoofing* — exatamente a configuração modelada pela métrica *t*-DCF adotada neste trabalho (Seção 5.5) — o que torna a detecção robusta de áudio sintético um requisito operacional, e não apenas acadêmico.

Esse contexto impõe uma **assimetria de custo** que orienta a escolha do ponto de operação dos detectores. Aceitar um ataque como legítimo (falso negativo do detector) abre acesso indevido a uma conta ou a um prontuário; rejeitar um usuário genuíno (falso positivo) causa atrito e o encaminha a um canal alternativo de verificação. Os dois erros não custam o mesmo, e a parametrização do *t*-DCF adotada exprime isso numericamente:  $C_{fa} = 10 \gg C_{miss} = 1$ .

A consequência prática é que o EER, sozinho, descreve um ponto de operação que a aplicação-alvo não utiliza: ele pondera os dois erros igualmente. Reportar também o *t*-DCF\* permite verificar se o ordenamento dos detectores sobrevive à ponderação por custo. Adianta-se o resultado, que é negativo e nem por isso desinteressante: neste *corpus* a ponderação **não reordena** o recorte (Seção 6.4). A assimetria de custo, portanto, não muda *qual* detector escolher — muda apenas *em que ponto* operá-lo.

Nem por isso a assimetria deixa de importar. Ela se manifesta *dentro* de cada detector, na escolha do limiar: um modelo pode exibir EER baixo e, ainda assim, degradar abruptamente ao ser levado ao regime de baixo falso-alarme que a aplicação exige. É o que ocorre com o AASIST, cuja taxa de perda salta de 1,9% para 42,3% ao apertar o falso-alarme de 5% para 1% (Seção 6.4). Por isso o protocolo reporta EER, AUC-ROC e *t*-DCF\* em conjunto, e a curva DET como complemento: o ordenamento responde qual detector escolher, e a curva responde em que ponto operá-lo.

## 1.2 Objetivos

### 1.2.1 Objetivo Geral

Desenvolver e avaliar um *pipeline* modular, reproduzível e de código aberto para detecção de áudio sintético em português, contemplando 11 modelos consolidados na literatura e permitindo a comparação sistemática entre representações de áudio bruto, descritores acústicos tabulares e mapas tempo-frequência.

### 1.2.2 Objetivos Específicos

- Revisar as principais gerações de síntese, clonagem e conversão de voz, relacionando seus artefatos acústicos aos desafios de detecção.
- Consolidar uma base experimental balanceada em português, com particionamento reproduzível e especificação padronizada de taxa de amostragem, duração, canal e rótulo.
- Implementar pré-processamento acústico — normalização de amplitude, janelamento temporal padronizado e extração de características compatíveis com cada família de modelo —, além de detecção de atividade de voz (VAD) como componente do *pipeline* de inferência, desativada no *corpus* experimental para preservar a simetria do pareamento (Seção 5.2).
- Implementar e comparar 11 modelos referenciados na literatura: WavLM Original, HuBERT Original, RawNet2, AASIST, RawGAT-ST, Conformer, CCT, AST, Res2Net, SVM e *Random Forest*.
- Avaliar os modelos sob um protocolo comum, reportando acurácia, F1-score, AUC-ROC, EER, *t*-DCF, latência, tamanho dos artefatos e matrizes de confusão.
- Investigar a robustez dos modelos sob ruído AWGN em múltiplos níveis de SNR — incluindo um nível não visto no treino — e delimitar o escopo de validade dos resultados quanto à generalização para geradores de síntese e condições de canal não observados.
- Disponibilizar um *pipeline* de treinamento, inferência e *benchmark* baseado em tecnologias de código aberto, com artefatos e figuras gerados de forma rastreável.

## 1.3 Questões de Pesquisa

Três questões orientam a avaliação experimental:

1. **QP1:** Como famílias heterogêneas de modelos — classificadores clássicos sobre descritores tabulares, redes espectro-temporais, redes de áudio bruto/grafos e *backbones* SSL congelados — se comparam sob um protocolo único de treinamento e avaliação em um *corpus* de fala em português?

2. **QP2:** Em que medida a acurácia em conjunto de teste limpo prediz a robustez sob ruído aditivo, e que fatores explicam eventuais dissociações? Partiu-se das duas hipóteses mais plausíveis — a representação de entrada e a capacidade do modelo. Adianta-se que **ambas foram rejeitadas** pelos dados, e que a Seção 6.4 discute o que resta e o que fica em aberto.
3. **QP3:** Quais compromissos de custo computacional (latência, memória, número de parâmetros) acompanham cada família, e existe um modelo dominante em todas as dimensões?

As três questões são retomadas na análise dos resultados (Seção 6.4) e nas conclusões (Seção 7).

## 1.4 Organização do Trabalho

O restante do documento está organizado da seguinte forma: a Seção 2 revisa as tecnologias de síntese e manipulação de voz e a evolução dos métodos de detecção; a Seção 3 percorre as campanhas de avaliação, os recursos disponíveis para o português e os estudos de generalização, situando este trabalho entre eles; a Seção 4 descreve as 11 arquiteturas comparadas — cujos nomes acadêmicos correspondem às entradas do repositório conforme a Tabela 22 (Apêndice A.2) —; a Seção 5 detalha o *pipeline*, o *corpus*, as métricas e o protocolo de treinamento; a Seção 6 apresenta e analisa os resultados; e a Seção 7 sintetiza conclusões, contribuições, limitações e trabalhos futuros. A implementação de referência que serve os modelos avaliados está no Apêndice B.

## 2 Tecnologias de Síntese de Voz

A síntese de voz reúne técnicas capazes de gerar ou transformar fala humana por meio de modelos acústicos, linguísticos e neurais. Nesta pesquisa, essas tecnologias são relevantes porque definem o tipo de artefato que o detector deve identificar: descontinuidades espectrais de *vocoders*, inconsistências prosódicas, vazamento de identidade vocal, ruído de canal ou padrões de codificação aprendidos por sistemas generativos. Assim, este capítulo separa as tecnologias de geração de fala dos métodos de detecção, preservando a ligação entre os ataques considerados e as arquiteturas avaliadas no restante do trabalho.

### 2.1 Da Síntese Clássica à Síntese Neural

Os primeiros sistemas eletrônicos de fala baseavam-se no modelo fonte-filtro, em que a excitação glotal e o trato vocal eram parametrizados separadamente [2, 3]. Posteriormente, a síntese concatenativa passou a combinar unidades gravadas de fala humana, obtendo maior naturalidade ao custo de grandes bases de áudio e menor flexibilidade prosódica [4]. A transição para modelos neurais substituiu regras explícitas por funções aprendidas: WaveNet modelou a forma de onda de modo autoregressivo [5], Tacotron e Tacotron 2 mapearam texto para espectrogramas Mel [6, 10], e arquiteturas posteriores como FastSpeech e VITS exploraram inferência mais rápida e geração *end-to-end* [19, 20].

Modelos recentes ampliaram esse cenário para síntese multilíngue, clonagem de voz e adaptação com poucas amostras. YourTTS e VALL-E exemplificam essa tendência ao combinar representação de locutor, conteúdo linguístico e codificadores neurais de áudio para generalizar a falantes não vistos [7, 21]. No conjunto utilizado neste trabalho, a fração Fake Voices é composta por amostras geradas com XTTS, sistema *cross-lingual* de TTS associado à classe sintética do *corpus* [22]. Essa origem justifica a avaliação de detectores capazes de lidar com fala neural em português brasileiro e com variações de locutor.

### 2.2 Text-to-Speech, Clonagem e Conversão de Voz

A síntese de fala por texto (*Text-to-Speech*, TTS) recebe uma sequência textual ou fonética e produz áudio inteligível. Em sistemas neurais clássicos, um modelo acústico transforma o texto em espectrograma Mel, que é convertido em forma de onda por um *vocoder* neural:

$$\mathbf{s}_{\text{mel}} = h_{\psi}(\mathbf{t}_{\text{text}}), \quad \mathbf{y}_{\text{audio}} = v_{\omega}(\mathbf{s}_{\text{mel}}) \quad (1)$$

em que  $\mathbf{t}_{\text{text}}$  é o texto de entrada,  $\mathbf{s}_{\text{mel}}$  é o espectrograma intermediário,  $\mathbf{y}_{\text{audio}}$  é a forma de onda sintetizada,  $h_{\psi}$  representa o modelo acústico e  $v_{\omega}$  representa o *vocoder*. A Equação 1 é uma abstração funcional do processo: implementações modernas podem empregar codificadores de fala, modelos auto-regressivos, difusão, *flow* ou variacionais, mas a separação entre conteúdo linguístico, representação acústica e síntese de onda permanece útil para

discutir artefatos detectáveis.

A clonagem de voz (*voice cloning*) consiste em replicar as características acústicas e prosódicas de um falante a partir de amostras limitadas [1]. Modelos como VALL-E [7] empregam abordagem *few-shot* baseada em codificadores neurais de áudio, com alta similaridade perceptual mesmo quando a referência de voz é curta:

$$\mathbf{y}_{\text{clone}} = f_{\theta}(\mathbf{x}_{\text{text}}, \mathbf{z}_{\text{speaker}}) \quad (2)$$

em que  $\mathbf{y}_{\text{clone}}$  é o áudio sintético,  $\mathbf{x}_{\text{text}}$  é o texto,  $\mathbf{z}_{\text{speaker}}$  é o vetor de características do falante e  $f_{\theta}$  é o modelo neural treinado. A Equação 2 evidencia que a ameaça não depende apenas da naturalidade da fala, mas também da preservação da identidade vocal, fator central em ataques contra biometria de voz.

A conversão de voz (*voice conversion*, VC) transforma a voz de um falante para soar como a de outro, preservando o conteúdo linguístico [11]. Diferentemente do TTS, a entrada já contém fala; o modelo busca separar conteúdo, prosódia e identidade para reemitir o sinal com características do falante-alvo:

$$\mathbf{y}_{\text{conv}} = g_{\phi}(\mathbf{x}_{\text{source}}, \mathbf{z}_{\text{target}}) \quad (3)$$

A Equação 3 formaliza esse mapeamento. Modelos como StarGAN-VC [23] usam redes adversariais para aprender mapeamentos muitos-para-muitos. A VC é particularmente desafiadora para detecção porque preserva boa parte do conteúdo linguístico e pode deslocar artefatos para regiões sutis do espectro, aumentando a dependência de modelos robustos a geradores não vistos [17].

A manipulação em tempo real representa a aplicação operacional da conversão de voz em chamadas, transmissões ou atendimento remoto. Em vez de assumir que o ataque seja imperceptível, a questão técnica relevante é que latências baixas reduzem a janela de verificação humana e automatizada [12]. Trata-se do mesmo mapeamento da Equação 3, executado sob restrição de latência de processamento (tipicamente dezenas de milissegundos por bloco de áudio): sistemas RVC e abordagens correlatas atendem a essa restrição com extratores de conteúdo leves e redes otimizadas para inferência contínua [11].

A Tabela 1 sintetiza as quatro modalidades descritas nesta seção, relacionando entrada, saída, artefatos acústicos esperados e o risco associado a cada uma.

**Tabela 1:** Principais modalidades de geração e manipulação de fala sintética.

| Modalidade                | Entrada típica                     | Saída                            | Artefatos esperados                                                  | Risco principal                    |
|---------------------------|------------------------------------|----------------------------------|----------------------------------------------------------------------|------------------------------------|
| TTS neural                | Texto ou fonemas                   | Fala sintética completa          | Prosódia artificial, marcas de <i>vocoder</i> , suavização espectral | Conteúdo falso atribuído a uma voz |
| Clonagem de voz           | Texto e poucos segundos do locutor | Fala no timbre do alvo           | Inconsistência de identidade, entonação e respiração                 | Fraudes de identidade              |
| Conversão de voz          | Fala de origem e identidade-alvo   | Fala convertida                  | Artefatos de separação conteúdo/falante, fase e formantes            | Impersonação em fala real          |
| Manipulação em tempo real | Fluxo de áudio e perfil-alvo       | Saída alterada em baixa latência | Compressão, instabilidade temporal e ruído de canal                  | Ataques em chamadas ao vivo        |

## 2.3 Implicações para a Detecção

As tecnologias descritas acima motivam detectores capazes de operar sobre representações complementares. Modelos baseados em descritores manuais capturam estatísticas acústicas agregadas; redes espectrais exploram padrões tempo-frequência; arquiteturas de áudio bruto modelam microestrutura temporal; e modelos auto-supervisionados herdam representações treinadas em grandes coleções de fala.

Essa correspondência não é arbitrária: cada família de artefato de síntese deixa vestígio em um domínio específico do sinal, e é isso que determina qual representação consegue detectá-lo. Quatro casos organizam o mapeamento.

- **Artefatos de reconstrução de fase.** *Vocoders* que sintetizam a partir de espectrogramas de magnitude precisam estimar a fase, e a estimativa deixa inconsistências entre quadros vizinhos. Como a representação Mel de magnitude *descarta* a fase, a hipótese natural é que esses artefatos sejam inacessíveis às redes espectrais e detectáveis apenas por arquiteturas que operam sobre a forma de onda — RawNet2, AASIST e RawGAT-ST, cujos *front-ends* SincConv preservam a estrutura temporal fina. Trata-se de uma **previsão falseável**, e ela é confrontada com os resultados na Seção 6.4.
- **Suavização espectral.** Modelos autoregressivos e de *difusão* tendem a produzir envoltórias mais suaves que a fala natural, com harmônicos de alta frequência atenuados. É um artefato de *envoltória*, capturado por descritores cepstrais e por mapas tempo-frequência — o que explica a eficácia histórica de MFCC, LFCC e CQCC, e depois das CNNs sobre espectrograma.

- **Compressão de faixa dinâmica.** Sintetizadores normalizam a própria saída, comprimindo a variação de amplitude que a fala gravada preserva. O vestígio está em estatísticas *temporais* de amplitude — e é justamente o que a análise de importância deste trabalho encontra como fator dominante no vetor tabular (Seção 6.4).
- **Prosódia e coarticulação.** Sistemas *zero-shot* de clonagem reproduzem timbre com fidelidade, mas erram no ritmo e nas transições entre fonemas. É informação de *longo alcance*, que exige contexto além do quadro — o argumento a favor de mecanismos de atenção e de *backbones* SSL pré-treinados em larga escala.

Nenhuma representação isolada cobre os quatro domínios, o que sustenta a decisão metodológica central deste trabalho: comparar famílias heterogêneas sob protocolo idêntico, em vez de eleger a priori uma representação. A Tabela 2 sintetiza essa progressão histórica como evolução dos métodos de detecção, não como taxonomia de síntese.

**Tabela 2:** *Survey* das gerações de métodos de detecção de áudio sintético. Os desempenhos citados referem-se aos protocolos em que foram originalmente publicados — majoritariamente o ASVspoof — e não são diretamente transponíveis para o protocolo deste trabalho, que usa outro idioma, outro gerador e outro particionamento (Seção 6.4).

| Geração                    | Representação                                          | Classificador                                                          | Pontos Fortes                            | Limitações                                                                       |
|----------------------------|--------------------------------------------------------|------------------------------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------|
| 1 <sup>a</sup> (até 2015)  | MFCC, LFCC, CQCC                                       | GMM-UBM, SVM [24]                                                      | Baixo custo computacional                | Baixa generalização                                                              |
| 2 <sup>a</sup> (2016–2019) | Espectrograma<br>Mel                                   | CNN, LSTM [25],<br>LCNN                                                | Robustez a artefatos de <i>vocoder</i>   | Sensibilidade a geradores não vistos                                             |
| 3 <sup>a</sup> (2020–2022) | Forma de onda, grafos                                  | RawNet2, AASIST                                                        | EER < 1% <i>no ASVspoof 2019 LA</i>      | Alta demanda computacional (GPU); resultado não transfere para outros protocolos |
| 4 <sup>a</sup> (2021+)     | Representações pré-treinadas em larga escala e atenção | WavLM, HuBERT (auto-supervisionados); AST (supervisionado em AudioSet) | Melhor aproveitamento de contexto global | Dependência de pré-treinamento e validação externa                               |

Até o desafio ASVspoof 2015 [26], o estado da arte envolvia a extração manual de características paramétricas — como MFCC, LFCC e CQCC — seguida por classificadores tradicionais. Sahidullah et al. [27] demonstraram que o LFCC supera o MFCC como *front-end* para detecção de *spoofing*, enquanto Todisco et al. [28] introduziram o CQCC como representação de referência. A partir do ASVspoof 2019 [15], CNNs, LCNNs [29], Res2Net [30] e modelos recorrentes passaram a explorar espectrogramas e mapas tempo-frequência. Em seguida, RawNet2 [31], AASIST [32] e RawGAT-ST aproximaram a detec-

ção do sinal bruto e de grafos de atenção. Mais recentemente, WavLM [33], HuBERT [34], wav2vec 2.0 [35] e AST [36] incorporaram representações SSL e mecanismos de atenção inspirados por Transformers [37] e ViT [38]. Essa trajetória fundamenta a seleção das arquiteturas comparadas na Seção 4 e justifica a avaliação de modelos heterogêneos sob um mesmo protocolo.

## 3 Trabalhos Relacionados

A avaliação de contramedidas contra fala sintética organiza-se hoje em torno de campanhas competitivas, de recursos de dados por idioma e de estudos metodológicos sobre generalização. Esta seção percorre os três eixos e delimita, ao final, o que este trabalho acrescenta e o que deliberadamente não faz.

### 3.1 Campanhas de Avaliação

As edições do ASVspoof [15–17, 26] estabeleceram o formato dominante: partições oficiais, métrica primária comum e conjunto de ataques crescente. A edição de 2015 tratou de síntese e conversão de voz com o EER como métrica primária; a de 2019 introduziu o *t*-DCF [53], acoplando o custo do detector ao de um sistema de verificação de locutor, e consolidou RawNet2 [31] e AASIST [32] como linhas de base de áudio bruto; a de 2021 acrescentou a trilha *DeepFake*, com degradação por codec e compressão; e o ASVspoof 5 [17] ampliou a escala e incorporou ataques adversariais. O desafio ADD [18] cumpre papel análogo para o mandarim.

Esses protocolos são o padrão de referência para *generalização a ataques não vistos* — o eixo em que o presente trabalho é explicitamente mais fraco, por operar com gerador único (Seção 7.2). Em contrapartida, eles não controlam texto, locutor e canal entre as classes, que é o eixo em que o protocolo aqui adotado é mais forte. Os dois desenhos respondem a perguntas diferentes.

### 3.2 Recursos para o Português

Para o português brasileiro os recursos públicos são recentes. BRSpeech-DF [39] e Fake Voices [22] disponibilizam áudio sintético e genuíno, mas como *conjuntos de dados*, não como protocolo: não fixam partição oficial, métrica primária nem recorte de arquiteturas, de modo que resultados publicados sobre eles não são diretamente comparáveis entre si. É essa lacuna — ausência de um *benchmark* padronizado entre famílias arquiteturais em português — que motiva o protocolo descrito na Seção 5.

### 3.3 Generalização e Correlações Espúrias

Há ainda uma lacuna metodológica que não é específica do idioma. Corpora de *anti-spoofing* costumam agregar fala genuína e sintética de *origens distintas*, o que confunde o processo de síntese com texto, locutor e canal de gravação — e permite que um detector aparente alto desempenho explorando correlações que nada têm a ver com síntese. Müller et al. [40] mostram que detectores treinados nessas condições degradam acentuadamente fora da base de origem. É a essa ameaça à validade interna que o **pareamento** adotado aqui responde.

### 3.4 Posicionamento deste Trabalho

A Tabela 3 sintetiza o contraste. Este trabalho não propõe arquitetura nova: submete 11 modelos heterogêneos a um mesmo protocolo de dados, treinamento e métricas — incluindo custo de treinamento, custo de inferência e robustez a ruído — sobre um *corpus* pareado em que cada amostra sintética é o clone da mesma locução do mesmo falante. O pareamento converte a comparação entre classes em contraste controlado: mantém texto, identidade e canal constantes e deixa o processo de síntese como única variável sistemática.

**Tabela 3:** Posicionamento em relação aos protocolos de avaliação existentes. Os eixos em que este trabalho é mais fraco (gerador e idioma únicos) e mais forte (controle de confundidores, auditoria de partição) são complementares, e não concorrentes.

| Protocolo                          | Idioma    | Geradores                | Classes pareadas | Foco da comparação                                         |
|------------------------------------|-----------|--------------------------|------------------|------------------------------------------------------------|
| ASVspoof 2019/2021 [15, 16]        | Inglês    | Dezenas                  | Não              | Generalização a ataques não vistos                         |
| ASVspoof 5 [17]                    | Inglês    | Dezenas (+ adversariais) | Não              | Escala e robustez adversarial                              |
| ADD 2022 [18]                      | Mandarim  | Múltiplos                | Não              | Detecção em condições reais                                |
| BRSpeech-DF [39], Fake Voices [22] | Português | Múltiplos                | Não              | Recurso de dados (sem protocolo)                           |
| <b>Este trabalho</b>               | Português | <b>Um</b> (XTTS-v2)      | <b>Sim</b>       | Comparação controlada entre famílias, com custo e robustez |

Diferentemente das tabelas de liderança do ASVspoof, o objetivo não é o estado da arte absoluto, e sim a comparabilidade controlada entre famílias sob condições idênticas e auditáveis. A contrapartida está declarada: com gerador único, os resultados medem a detecção *daquele* processo de síntese, e a validação *cross-generator* permanece o passo seguinte (Seção 7.3).

## 4 Arquiteturas de Classificação

Esta seção descreve as 11 arquiteturas comparadas. Para cada modelo, apresentam-se a síntese arquitetural, o formato de entrada do artigo de referência e o escopo exato adotado neste trabalho.

### 4.1 Duas Dimensões de Classificação

Classificar detectores apenas pela representação de entrada — áudio bruto, espectrograma, vetor tabular — é insuficiente para este recorte, porque esconde a variável de que as conclusões dependem: o **regime de treinamento**. WavLM e RawNet2 consomem a mesma forma de onda, mas o primeiro é um *backbone* congelado com 94 mil horas de pré-treinamento e apenas uma cabeça rasa ajustada, enquanto o segundo aprende todos os seus parâmetros a partir das 12.162 amostras de treino. Chamá-los de “mesma família” apagaria justamente a diferença que a Seção 7 precisa discutir.

A Tabela 4 organiza o recorte pelas *duas* dimensões. Ela torna visível uma assimetria do desenho experimental que uma taxonomia unidimensional esconde: o AST é o **único** representante da célula “espectral com pré-treinamento”, de modo que qualquer comparação entre ele e as demais redes espectrais confunde arquitetura com regime de treinamento (Seção 5.3).

**Tabela 4:** Taxonomia bidimensional do recorte: representação de entrada × regime de treinamento. As subseções seguintes seguem as linhas desta tabela.

| Representação        | Treinado do zero           | Pré-treinado (congelado)        | Pré-treinado (pesos transferidos) |
|----------------------|----------------------------|---------------------------------|-----------------------------------|
| Áudio bruto          | RawNet2, AASIST, RawGAT-ST | WavLM Original, HuBERT Original | —                                 |
| Espectrograma<br>Mel | Conformer, CCT, Res2Net    | —                               | <b>AST</b>                        |
| Vetor tabular        | SVM, <i>Random Forest</i>  | —                               | —                                 |

As subseções a seguir percorrem essas células: modelos de áudio bruto (Seção 4.2), redes de domínio espectro-temporal (Seção 4.3) e classificadores estatísticos clássicos (Seção 4.4), assinalando em cada caso o regime de treinamento adotado.

Três dos nomes usados aqui — AST, CCT e Res2Net — são os consagrados na literatura, enquanto o repositório registra as entradas correspondentes pelo nome descritivo da família (*SpectrogramTransformer*, *Hybrid CNN-Transformer* e *MultiscaleCNN*). A correspondência completa, necessária para reproduzir os experimentos a partir do código, está na Tabela 22 (Apêndice A.2).

## 4.2 Modelos de Áudio Bruto

Esta linha da Tabela 4 reúne os cinco modelos que consomem a forma de onda diretamente, sem extrator externo — mas ela se divide em **dois regimes de treinamento** que não devem ser confundidos.

**Pré-treinados e congelados.** WavLM e HuBERT herdam representações auto-supervisionadas aprendidas em grandes coleções de fala; aqui o *backbone* não é ajustado, e treinam-se apenas a soma ponderada de camadas e uma cabeça rasa. O custo de aprendizagem foi pago fora deste trabalho.

**Treinados do zero.** RawNet2, AASIST e RawGAT-ST aprendem filtros e relações temporais a partir das 12.162 amostras de treino, sem transferência de pesos.

A entrada em áudio bruto é o eixo comum aos cinco; o que muda é de onde vem a representação. Essa distinção é retomada na Seção 7, ao discutir por que os *backbones* congelados ficam na metade inferior do recorte.

### 4.2.1 WavLM [33]

**Síntese Arquitetural:** WavLM é um modelo SSL baseado em Transformer, treinado por predição mascarada de fala combinada a uma tarefa de denoising. Essa formulação busca preservar conteúdo linguístico, identidade do locutor e informações paralinguísticas, tornando o *backbone* útil para tarefas de fala além de reconhecimento automático.

**Formato de entrada no artigo de referência:** O modelo opera sobre forma de onda de fala, usualmente representada como sinal monofônico amostrado a 16 kHz. A extração de atributos é feita internamente pelo codificador convolucional e pelas camadas Transformer, sem necessidade de espectrogramas ou MFCCs pré-computados.

**Escopo neste trabalho:** O manifesto experimental avalia o WavLM Original por meio do *checkpoint microsoft/wavlm-base-plus* (94,4 milhões de parâmetros, pré-treinado em 94 mil horas de fala), carregado via PyTorch/*transformers*. O *backbone* permanece **congelado**; treinam-se apenas a soma ponderada das 13 camadas ocultas — a receita SUPERB, em que os pesos por camada são aprendidos junto com a tarefa — e a cabeça MLP de classificação (394 mil parâmetros, ou 0,4% do total), sobre *pooling* de média $\oplus$ desvio-padrão ao longo do tempo.

A soma ponderada importa metodologicamente: em modelos SSL, o conteúdo linguístico concentra-se nas camadas intermediárias, enquanto artefatos de baixo nível — os relevantes para anti-*spoofing* — persistem nas camadas iniciais. Ler apenas a última camada, como faria um uso ingênuo do *backbone*, descartaria justamente o sinal de interesse.

O qualificador “Original” distingue este uso do *checkpoint* pré-treinado real de portes simplificados da arquitetura; ele não implica ajuste fino integral, de modo que os resultados refletem a qualidade dos *embeddings* congelados, não o teto de desempenho do WavLM.

#### 4.2.2 HuBERT (*Hidden-Unit BERT*) [34]

**Síntese Arquitetural:** HuBERT também pertence à família SSL, mas sua ideia central é prever unidades acústicas ocultas obtidas por agrupamento offline. O treinamento aplica perda apenas sobre regiões mascaradas, forçando o modelo a aprender regularidades acústicas e linguísticas a partir de rótulos discretos pseudo-supervisionados.

**Formato de entrada no artigo de referência:** O modelo recebe fala em forma de onda contínua, sem extratores cepstrais externos. As unidades ocultas usadas como alvo de pré-treinamento são derivadas de agrupamentos sobre representações acústicas, mas a entrada operacional do modelo permanece o áudio bruto, tipicamente em 16 kHz.

**Escopo neste trabalho:** O HuBERT Original é avaliado com o *checkpoint facebook/hubert-base-ls960* (94,4 milhões de parâmetros, pré-treinado em 960 horas do LibriSpeech), também via PyTorch/*transformers*, sob *exatamente* o mesmo protocolo do WavLM: *backbone* congelado, soma ponderada das 13 camadas ocultas, *pooling* média $\oplus$ desvio e a mesma cabeça MLP de 394 mil parâmetros.

A equivalência de protocolo é o que torna o par informativo. As duas arquiteturas têm o mesmo tamanho e diferem essencialmente em duas coisas: o objetivo de pré-treino (predição de unidades ocultas no HuBERT; predição mascarada com *denoising* e viés posicional com *gating* no WavLM) e a escala do *corpus* de pré-treino — 960 horas contra 94 mil. A diferença de desempenho observada entre eles (Seção 6.4) é, portanto, atribuível ao pré-treinamento, e não a diferenças de capacidade ou de *back-end*.

#### 4.2.3 RawNet2 [31]

**Síntese Arquitetural:** RawNet2 é uma rede *end-to-end* para anti-*spoofing* que substitui descritores manuais por filtros passa-banda aprendíveis. A arquitetura combina uma frente SincNet, blocos residuais com escalamento de mapas de características e uma camada GRU para agregação temporal antes da decisão binária.

**Formato de entrada no artigo de referência:** O artigo de referência processa diretamente a forma de onda bruta, sem converter previamente o sinal para espectrograma. Essa entrada preserva informações de fase, microvariações temporais e artefatos de síntese que podem ser suavizados por representações tempo-frequência.

**Escopo neste trabalho:** O *benchmark* utiliza a variante RawNet2 alinhada ao artigo (*paper-like*), operando sobre áudio bruto padronizado: frente SincNet, blocos residuais com escalamento de mapas de características (FMS, *Feature Map Scaling*), GRU e camada densa de 1024 unidades, com treinamento supervisionado em GPU sob o protocolo comum (Seção 5.7). O modelo representa a referência principal de aprendizagem direta sobre PCM no conjunto experimental.

#### 4.2.4 AASIST [32]

**Síntese Arquitetural:** AASIST estende a família RawNet ao introduzir atenção em grafos espectrais e temporais. Após uma frente convolucional sobre a forma de onda, o modelo constrói subgrafos para capturar relações entre bandas frequenciais e segmentos temporais, integrando-os por uma camada heterogênea de atenção com nó de empilhamento.

**Formato de entrada no artigo de referência:** A entrada é áudio bruto. Os grafos espectro-temporais não são fornecidos como dados externos: eles são derivados internamente a partir das representações aprendidas pelo codificador inicial.

**Escopo neste trabalho:** A variante `aasist` é usada como implementação alinhada ao artigo, com SincConv, blocos residuais, ramos de GAT espectral e temporal, camadas HS-GAL (*Heterogeneous Stacking Graph Attention Layer*), leitura por grafo e cabeça AM-Softmax (*Additive Margin Softmax*). O modelo é treinado sob o mesmo protocolo padronizado (Seção 5.7) e integra o grupo de modelos de áudio bruto/grafos do manifesto oficial.

#### 4.2.5 RawGAT-ST [41]

**Síntese Arquitetural:** RawGAT-ST modela artefatos de falsificação por meio de redes de atenção em grafos aplicadas a dois domínios complementares: subbandas espectrais e intervalos temporais. A fusão em nível de modelo permite combinar evidências locais e globais sem depender de um *ensemble* externo de detectores.

**Formato de entrada no artigo de referência:** Assim como RawNet2 e AASIST, o RawGAT-ST recebe a forma de onda bruta. A estrutura espectro-temporal é aprendida a partir de um codificador inicial e depois organizada em subgrafos espectral e temporal.

**Escopo neste trabalho:** O *benchmark* utiliza a variante `rawgat_st_tak2021_paper_attention_multiply_topk12`, fiel à topologia do artigo: frente SincConv, **dois** ramos codificadores residuais 2D (espectral e temporal), um GAT sobre cada ramo, fusão por multiplicação elemento a elemento e um **terceiro** GAT sobre o grafo fundido, antes da leitura e da decisão binária. Com 650.204 parâmetros, é a segunda menor rede do recorte — e, apesar disso, a de maior custo de treinamento (Seção 6.2), pois a construção dinâmica dos grafos a cada lote tem intensidade aritmética baixa e amortiza mal em GPU.

### 4.3 Arquiteturas Neurais de Domínio Espectro-Temporal

#### 4.3.1 Conformer [42]

**Síntese Arquitetural:** Topologia híbrida (*Convolution-augmented Transformer*) que integra a excelência de modelagem de contexto global da autoatenção com a eficiência na extração de padrões locais dos módulos convolucionais.

**Desempenho e Observações:** Demonstra alta precisão operando sobre represen-

tações bidimensionais densas (espectrogramas Mel), capturando artefatos de envoltória espectral e de textura tempo-frequência típicos da síntese por *vocoders* — a informação de fase, descartada na representação Mel de magnitude, não é modelada.

**Escopo neste trabalho:** O manifesto experimental usa a configuração **Conformer-M** do artigo original — 16 blocos, dimensão de codificação 256, 4 cabeças de atenção e núcleo convolucional de tamanho 31 — sobre espectrogramas Mel, treinada do zero sob o protocolo comum (Seção 5.7), sem pré-treinamento em tarefas de reconhecimento de fala. Com 28,6 milhões de parâmetros, é a maior das redes espectrais treinadas do zero neste recorte.

#### 4.3.2 Compact Convolutional Transformer (CCT) [43]

**Síntese Arquitetural:** Variação otimizada do *Transformer* que substitui o fatiamento linear tradicional por uma etapa de tokenização convolucional precedente à inserção nos blocos de atenção.

**Desempenho e Observações:** Proporciona expressiva redução no custo paramétrico e na exigência de volume de dados, mitigando o custo quadrático da autoatenção sobre mapas espectrais.

**Escopo neste trabalho:** A variante adotada segue a topologia CCT-2/3×2 adaptada ao áudio: tokenizador convolucional de duas camadas (64 e 128 filtros, núcleo  $3 \times 3$ ) seguido de 4 blocos de autoatenção com 4 cabeças e dimensão de projeção 256, sobre espectrogramas Mel, com *sequence pooling* no lugar do *token CLS*. Com 1,82 milhão de parâmetros, é a menor rede espectral do recorte — duas ordens de grandeza abaixo do AST — e serve de contraponto de custo aos *Transformers* maiores.

#### 4.3.3 Audio Spectrogram Transformer (AST) [36]

**Síntese Arquitetural:** Adaptação direta do *Vision Transformer* (ViT) para o domínio acústico. Trata mapas tempo-frequência como sequências de *patches* processadas via autoatenção global pura.

**Desempenho e Observações:** Demanda calibragem minuciosa de hiperparâmetros e esquemas severos de regularização para evitar instabilidade quando treinado do zero. Neste trabalho, exibiu alta resiliência sob injeção de ruído aditivo (Seção 6.4).

**Escopo neste trabalho:** A variante `ast_gong2021_vit_base_audioset_pretrained_in300x128` usa a topologia ViT-Base com **pesos AudioSet transferidos** do *checkpoint* `MIT/ast-finetuned-audioset-10-10-0.4593`, sobre entrada de  $300 \times 128$  (128 bandas Mel, salto de 10 ms, janela de 25 ms) — em linha com o artigo original. O treinamento usa taxa de pico  $10^{-5}$  com *warmup* de 3000 passos e decaimento por cosseno. Esse pré-treinamento em larga escala é fator relevante na leitura do seu desempenho: os 99,71% de acurácia não decorrem de aprendizado a partir das 12.162 amostras de treino apenas, mas de uma representação acústica já formada em AudioSet e adaptada à tarefa.

#### 4.3.4 Res2Net (Topologia Multi-escala) [30]

**Síntese Arquitetural:** Evolução convolucional caracterizada pela introdução de conexões residuais hierárquicas de múltiplas escalas (resoluções) aninhadas dentro de um único bloco.

**Desempenho e Observações:** A extração multi-resolução permite ao modelo identificar simultaneamente artefatos de *spoofing* que se manifestam tanto em bandas largas quanto em nichos de alta frequência.

**Escopo neste trabalho:** O *benchmark* usa a variante `res2net50_gao2021_scale4_basewidth26_no_se` (profundidade 50, fator de escala  $s = 4$ , largura base 26, sem blocos *squeeze-and-excitation*) sobre espectrogramas Mel, com os blocos *Bottle2neck* fiéis ao artigo original, sob o mesmo protocolo padronizado das demais redes espectrais. Treinada do zero, sem transferência de pesos.

### 4.4 Classificadores Estatísticos Clássicos

#### 4.4.1 Support Vector Machine (SVM) [24]

**Síntese Arquitetural:** Com *kernel* de base radial (RBF), o SVM é um classificador não linear que projeta vetores de características acústicas (por exemplo, LFCCs e CQCCs) em um espaço de maior dimensão, no qual otimiza o hiperplano de separação por maximização de margem entre classes.

**Desempenho e Observações:** Foi referência de desempenho nas primeiras edições de detecção de *spoofing* de voz (por exemplo, ASVspoof 2015), mas sua capacidade preditiva degrada de forma acentuada diante de ataques gerados por sistemas neurais recentes e, sobretudo, em protocolos de generalização cruzada entre bases [40].

**Escopo neste trabalho:** O SVM é treinado sobre o vetor tabular de 183 descritores (estatísticas temporais, MFCC, RASTA-PLP e LFCC com  $\Delta/\Delta\Delta$ ; Seção 5.3) padronizado por *z-score*, com busca em grade por validação cruzada de 5 dobras **agrupadas por frase** sobre o *kernel* (linear, RBF),  $C \in \{0,1; 1; 10\}$  e  $\gamma \in \{\text{scale}; 10^{-3}; 10^{-2}; 10^{-1}\}$  — este último cruzado apenas com o *kernel* RBF, que é o único a usá-lo —, seguida de reajuste final e calibração isotônica das probabilidades. O agrupamento da validação cruzada é necessário porque o *corpus* é pareado: sem ele, o original e o clone do mesmo enunciado cairiam em dobras distintas.

#### 4.4.2 Random Forest [44]

**Síntese Arquitetural:** Método de aprendizagem em comitê (*ensemble learning*) baseado na agregação de múltiplas árvores de decisão estocásticas. O modelo reduz a propensão ao sobreajuste (*overfitting*) combinando amostragem *bootstrap* (*bagging*) com seleção aleatória de subespaços de características a cada nó de ramificação.

**Desempenho e Observações:** Destaca-se pela escalabilidade em arquiteturas multi-

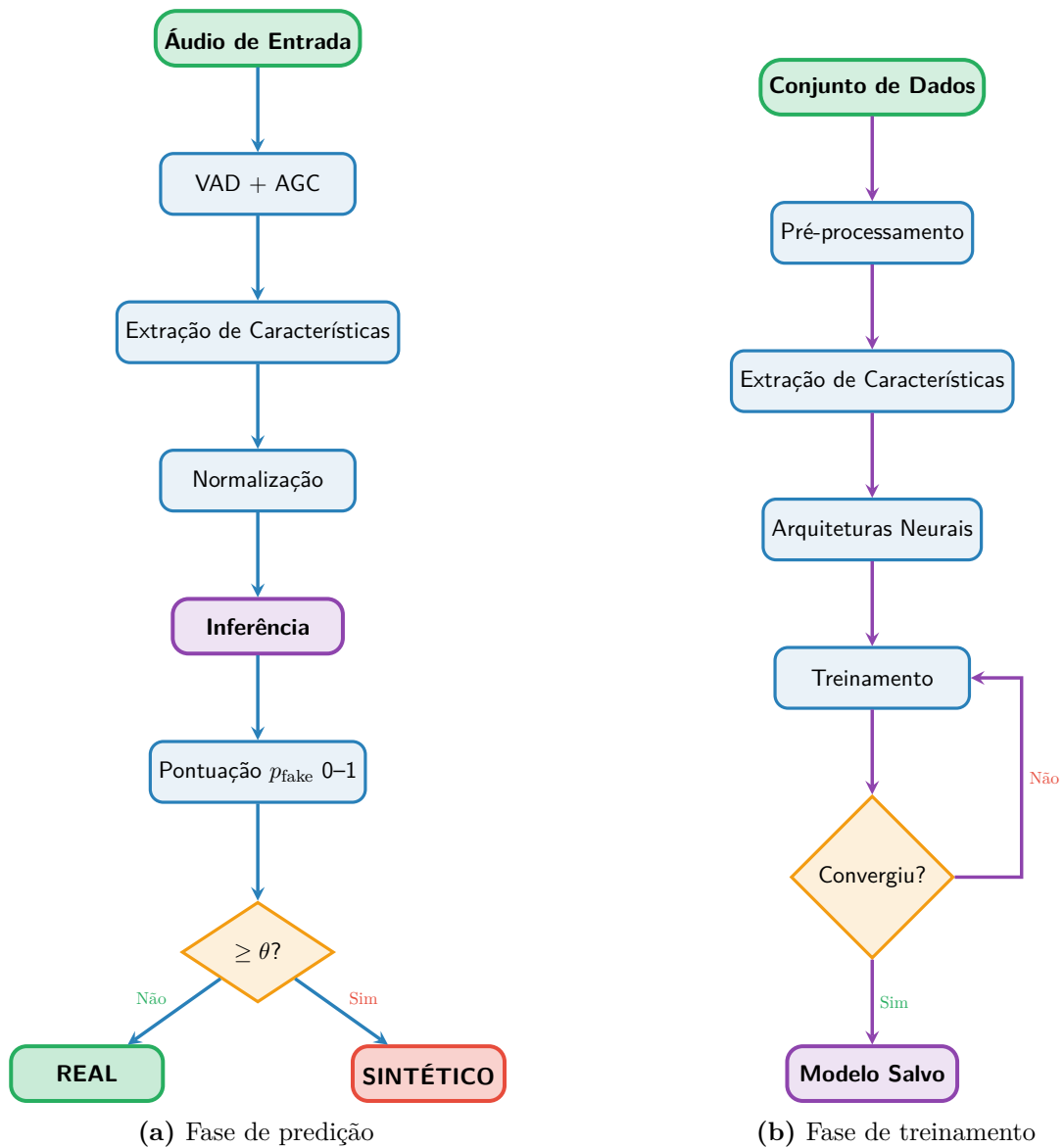
núcleo e pela interpretabilidade direta da relevância dos atributos (*feature importance*). Por outro lado, tem dificuldade em capturar dependências espectro-temporais finas e assinaturas não estacionárias relevantes para a detecção de *deepfakes* de voz mais recentes (padrão ASVspoof 2019 e sucessores) — limitação também observada empiricamente neste trabalho na robustez sob ruído (Seção 6.4).

**Escopo neste trabalho:** O *Random Forest* consome o mesmo vetor tabular de 183 descritores do SVM, com busca em grade por validação cruzada de 5 dobras sobre o número de árvores ( $\{200, 300\}$ ), a profundidade máxima ( $\{10, 15, 20\}$ ), os mínimos de divisão e de folha ( $\{5, 10, 20\}$  e  $\{2, 4, 8\}$ ) e o número de características por nó (`sqrt`, `log2`); a grade restringe deliberadamente a profundidade máxima e exige folhas maiores, como regularização estrutural contra o sobreajuste.

## 5 Metodologia

### 5.1 Visão Geral do Pipeline

O sistema proposto opera em duas fases: **treinamento** e **predição**. Durante o treinamento, o modelo aprende a distinguir características de voz real e sintética. Na predição, processa novos áudios e fornece classificação binária (real/sintético) com pontuação de confiança.



**Figura 1:** Diagrama do *pipeline* proposto. O modelo treinado (b) é implantado (*deploy*) na fase de inferência (a). A pontuação  $p_{fake}$  é a probabilidade predita de a amostra ser sintética: valores acima do limiar de decisão  $\theta$  resultam em SINTÉTICO. No *benchmark*,  $\theta = 0,5$  para os 11 modelos, sem exceção (política *fixed\_comparison*; Seção 5.5).

## 5.2 Pré-processamento: Normalização e Detecção de Atividade de Voz

O bloco “VAD + AGC” da Figura 1 padroniza o áudio de entrada antes da extração de características, cobrindo múltiplos formatos (WAV, MP3, FLAC, OGG) com conversão automática para 16 kHz mono, PCM linear de 16 bit.

### Normalização de Amplitude: um Controle contra Vazamento por Nível

O nível de amplitude é normalizado por RMS para um alvo fixo, com teto de pico:

$$x_{\text{norm}}[n] = \gamma \cdot x[n], \quad \gamma = \min \left( \frac{A_{\text{alvo}}}{\sqrt{\frac{1}{N} \sum_{i=0}^{N-1} x[i]^2}}, \frac{A_{\text{teto}}}{\max_n |x[n]|} \right) \quad (4)$$

em que  $N$  é o número de amostras,  $A_{\text{alvo}} = 10^{-26/20}$  corresponde ao alvo de  $-26$  dBFS de RMS e  $A_{\text{teto}} = 10^{-1/20}$  ao teto de  $-1$  dBFS de pico. O termo mínimo na Equação 4 garante que a normalização de RMS nunca leve uma amostra a ceifar (*clipping*): quando o ganho necessário para atingir o alvo levaria o pico acima do teto, prevalece o ganho limitado pelo pico.

Essa etapa não é cosmética — ela elimina um canal de vazamento concreto. A política anterior preservava o *loudness* original de cada gravação, e sob ela **o nível sozinho separava as classes com AUC de aproximadamente 0,99**: o sintetizador normaliza a própria saída, enquanto a gravação original preserva a variação natural de nível da locução. Um detector treinado sob aquela política aprenderia a medir volume, não artefatos de síntese. Normalizar *as duas classes* ao mesmo alvo remove a variável, e é por isso que a normalização é aplicada simetricamente e não apenas como condicionamento de entrada.

### Detecção de Atividade de Voz (VAD)

O VAD por energia remove segmentos silenciosos ou não vocalizados, calculando a energia em decibéis por janela deslizando:

$$E[m] = 10 \log_{10} \left( \frac{1}{N_w} \sum_{n=0}^{N_w-1} |x[mH + n]|^2 \right) \quad (5)$$

$$\text{Voice}[m] = \begin{cases} 1, & \text{se } E[m] > \theta_{\text{VAD}} \\ 0, & \text{caso contrário} \end{cases} \quad (6)$$

em que  $N_w$  é o comprimento da janela de análise (25 ms),  $H$  é o *hop length* (passo, em amostras, entre janelas consecutivas; a 16 kHz, corresponde a 10 ms) e  $\theta_{\text{VAD}} = -25$  dBFS é o limiar de energia, referido ao fundo de escala do sinal normalizado. Em uma etapa de pós-

filtragem, segmentos contíguos marcados como fala com duração inferior a  $D_{\min} = 0,3\text{ s}$  são descartados, evitando a retenção de transientes isolados (Equações 5–6).

#### VAD: componente do *pipeline*, desativado no *benchmark*

O *pipeline* de inferência integra o Silero VAD (via Torch Hub), com o VAD por energia acima como *fallback* quando o `torchaudio` não está disponível. **No *corpus* experimental, porém, o VAD está desativado** (`vad: disabled` em `configs/dataset.yaml`), e as Equações 5–6 descrevem um componente do sistema, não uma etapa aplicada aos dados avaliados.

A escolha é deliberada e preserva a validade do pareamento. Recortar silêncio depende do conteúdo de cada sinal, e original e clone têm padrões de pausa distintos — o sintetizador não reproduz exatamente a temporização das pausas da gravação. Aplicar VAD removeria porções *diferentes* de cada lado do par, introduzindo justamente a assimetria que o pareamento existe para evitar. Com o VAD desativado, ambos os lados passam pela mesma janela de 3 s e a única diferença sistemática entre eles continua sendo o processo de síntese.

### 5.3 Representações de Entrada e Características Acústicas

O *pipeline* adapta a representação de entrada a cada família de arquitetura, preservando o mesmo áudio pré-processado ( $\mathbf{x} \in \mathbb{R}^{48000 \times 1}$ , 16 kHz mono, 3 s) como origem comum. A Tabela 5 resume essa correspondência.

**Tabela 5:** Representação de entrada consumida por família de modelo.

| Representação                       | Modelos                                                     | Observação                                                                                                                               |
|-------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Áudio bruto<br>( <i>raw-audio</i> ) | WavLM Original, HuBERT Original, RawNet2, AASIST, RawGAT-ST | Forma de onda íntegra de 3 s (48.000 amostras) com <i>z-score</i> por amostra, sem extrator externo                                      |
| Espectrograma Mel 100 × 80          | Conformer, CCT, Res2Net                                     | Mapa em dB (ref. máx.) com <i>z-score</i> por amostra (Equação 9)                                                                        |
| Espectrograma Mel 300 × 128         | AST                                                         | 128 bandas, janela de 25 ms e salto de 10 ms, conforme o artigo original — resolução distinta da das demais redes espectrais (ver texto) |
| Vetor tabular (183 descritores)     | SVM, <i>Random Forest</i>                                   | Estatísticas temporais, MFCC, RASTA-PLP e LFCC com $\Delta/\Delta\Delta$                                                                 |

#### A representação espectral não é constante dentro da família

As quatro redes espectrais **não** recebem o mesmo mapa. Conformer, CCT e Res2Net consomem 100 × 80; o AST consome 300 × 128 — três vezes a resolução temporal e 1,6 vez as bandas — porque a topologia ViT-Base com pesos AudioSet transferidos exige a grade em que foi pré-treinada.

A consequência precisa ser declarada: a comparação entre o AST e as outras três redes espectrais confunde *três* variáveis — arquitetura, resolução da entrada e pré-treinamento em larga escala. O AST é o único modelo do recorte que não parte do zero, e o único que vê a grade mais fina. Os 99,71% da Seção 6.4 não devem, portanto, ser lidos como superioridade arquitetural isolada; e o fato de o AST *não* se separar estatisticamente de CCT e Conformer, apesar dessas duas vantagens, é a leitura mais informativa do resultado. Cabe registrar, a propósito, que uma configuração anterior do AST neste *pipeline* — grade 100 × 80 e treino do zero — degradou até o acaso (EER  $\approx$  51%). Na grade menor, o *patch* 16 × 16 com passo 10 produz apenas 63 *tokens*, contra os 1212 do artigo original, para alimentar um ViT-Base de 85,5 milhões de parâmetros. As duas condições foram corrigidas **simultaneamente**, o que por muito tempo deixou a causa sem atribuição. A ablação da Seção 6.4.6 a resolve: treinar do zero na grade 300 × 128 produz 94,93% de acurácia, e não o acaso — logo o colapso vinha da *escassez de tokens* da grade menor, não da ausência de pesos transferidos.

Para os classificadores clássicos, a ausência de camadas convolucionais ou de atenção exige um vetor de características de dimensão fixa. O módulo `benchmarks/data.py` constrói, para cada amostra, um vetor de 183 descritores compostos por quatro famílias:

- **Estatísticas temporais (11):** média, desvio-padrão, média absoluta, valor RMS,

mínimo, máximo, percentis 25/50/75, energia da diferença entre amostras consecutivas (proxy de rugosidade do sinal) e taxa de cruzamento por zero (ZCR).

- **MFCC (26)**: média e desvio-padrão de 13 coeficientes cepstrais na escala Mel por amostra.
- **RASTA-PLP (26)**: média e desvio-padrão de 13 coeficientes RASTA-PLP (*RelAtive SpecTrA* – *Perceptual Linear Prediction* [45]) por amostra. O filtro RASTA atenua variações espectrais lentas associadas a efeitos de canal, e foi incorporado com a intenção de mitigar a degradação de SVM e *Random Forest* sob ruído — intenção apenas parcialmente confirmada: a família responde por 5,0% da importância por permutação do *Random Forest* (Seção 6.4), a menor das quatro.
- **LFCC com  $\Delta$  e  $\Delta\Delta$  (120)**: média e desvio-padrão de 20 coeficientes cepstrais em escala *linear* de frequência, mais suas primeiras e segundas derivadas temporais. A escala linear não comprime as altas frequências, região em que os *vocoders* neurais deixam artefato, e é o *front-end* do sistema de referência das campanhas ASVspoof; as derivadas, por serem diferenças entre quadros, são invariantes a deslocamentos constantes de canal ou de nível. Esta família foi acrescentada em resposta ao colapso do ponto de operação dos classificadores clássicos sob o SNR não visto de 5 dB — uma revisão do *front-end* tabular cuja natureza e cujo custo metodológico estão declarados adiante nesta seção.

O *front-end* tabular foi revisado após uma rodada exploratória — e os clássicos não são cegos ao teste

As famílias RASTA-PLP e LFCC+ $\Delta$ / $\Delta\Delta$  não faziam parte do vetor original de 63 descritores. Foram acrescentadas, levando-o a 183, *depois* de uma rodada exploratória em que os dois classificadores clássicos perderam o ponto de operação sob AWGN a 5 dB — o nível não visto. A causa diagnosticada é concreta: 8 dos 11 descritores temporais crescem monotonicamente com a potência do ruído, e mínimo e máximo são estatísticas de ordem sobre 48.000 amostras, ou seja, medidores de ruído disfarçados de descritores de fala.

A decisão é tecnicamente defensável, mas tem um custo que precisa ser dito com todas as letras: **ela usou uma observação feita sobre a partição de teste**. O congelamento criptográfico descrito na Seção 5.4.3 protege os nove modelos neurais, cuja arquitetura e cujos hiperparâmetros não foram tocados após o selo; **não protege SVM e *Random Forest***, cujo vetor de entrada foi escolhido com conhecimento de um resultado de teste. Os números desses dois modelos devem, portanto, ser lidos como estimativa otimista, e não como desempenho cego (Seção 7.2, item (viii)).

Registre-se ainda o resultado da revisão, que não é o otimista. O bloco LFCC elevou a AUC do *Random Forest* a 5 dB de 0,838 para 0,852, com a revocação subindo de 28,51% para 65,12%; para o SVM, a AUC no mesmo nível foi de 0,8494 para 0,8499 — *não mudou*. O ganho de revocação do SVM veio da calibração empurrando o ponto de operação de volta para a faixa útil, não de poder discriminativo novo. A tese de que LFCC+ $\Delta$ / $\Delta\Delta$  resolve a fragilidade dos classificadores clássicos fora da distribuição de treino sustenta-se

para a floresta e **não se sustenta para o SVM**.

Os coeficientes cepstrais na escala Mel (MFCC), propostos por Davis e Mermelstein [46] e que compõem a segunda família, são obtidos por:

$$MFCC[m, c] = \alpha_c \sum_{k=0}^{M-1} \log(S[m, k]) \cdot \cos\left(\frac{\pi c(k + 0,5)}{M}\right), \quad \alpha_c = \begin{cases} \sqrt{1/M}, & c = 0 \\ \sqrt{2/M}, & c \geq 1 \end{cases} \quad (7)$$

em que  $S[m, k]$  são as energias dos  $M = 40$  filtros triangulares na escala Mel para o quadro  $m$ , e  $c \in \{0, \dots, 12\}$  indexa os 13 coeficientes efetivamente extraídos na Equação 7 (média e desvio-padrão dos quais formam os 26 descritores da família MFCC). O fator  $\alpha_c$  é a normalização *ortonormal* da DCT-II, que torna a transformada uma isometria — preservando a energia entre o domínio log-Mel e o cepstral — e é a convenção adotada pela implementação utilizada (`scipy.fft.dct` com `norm='ortho'`, via `librosa`). Omiti-lo alteraria a escala dos coeficientes e, por consequência, a padronização subsequente. Antes do ajuste dos classificadores clássicos, cada dimensão do vetor de 183 descritores é padronizada por normalização estatística (*z-score*):

$$X_{\text{norm}} = \frac{X - \mu}{\sigma} \quad (8)$$

na Equação 8, em que  $\mu$  e  $\sigma$  são a média e o desvio-padrão de cada característica, estimados sobre o conjunto de treino (`StandardScaler`, `scikit-learn`) e reaplicados sem reajuste sobre validação e teste.

Esse vetor de 183 dimensões implementa, de forma concreta, os descritores estatísticos temporais e cepstrais mencionados nos objetivos deste trabalho; a contribuição relativa de cada descritor para a decisão do *Random Forest* treinado é quantificada empiricamente na Seção 6.4 (Figura 10). Assim, os 11 modelos comparados neste trabalho cobrem três regimes de entrada — áudio bruto, espectrograma Mel e vetor tabular temporal/cepstral; os descritores adicionais implementados no repositório, mas não consumidos nesta rodada de *benchmark*, são resumidos adiante nesta seção.

### Espectrograma Mel

A representação tempo-frequência consumida por Conformer, CCT e Res2Net (Tabela 5) é computada por:

$$P[m, k] = \sum_f H_k[f] \cdot |STFT[m, f]|^2, \quad MelSpec[m, k] = 10 \log_{10} \left( \frac{P[m, k] + \varepsilon}{\max_{m', k'} P[m', k']} \right) \quad (9)$$

em que  $f$  indexa as *bins* de frequência,  $H_k[f]$  é o banco de 80 filtros triangulares na escala Mel,  $STFT$  é a transformada de Fourier de curto termo com janela de 1024 amostras

( $n_{fft}$ , 64 ms),  $P[m, k]$  é o mapa de potência em escala Mel,  $\varepsilon$  evita  $\log(0)$  e o denominador referencia a escala em decibéis ao máximo do próprio mapa (Equação 9). Como a normalização é feita *por amostra*, o mapa resultante tem máximo de 0 dB em todos os casos: a representação codifica a estrutura espectral relativa, e não o nível absoluto — coerente com a normalização de amplitude da Seção 5.2, que já removeu o nível como variável discriminativa. O *hop length* é dimensionado dinamicamente para produzir exatamente 100 quadros por janela de 3 s (48.000 amostras  $\Rightarrow$  *hop* de 480, ou 30 ms), resultando em mapas de dimensão fixa  $100 \times 80$ , padronizados em seguida por normalização *z-score* por amostra — mesma normalização aplicada, na entrada em áudio bruto, aos modelos AASIST, RawGAT-ST e RawNet2.

A janela de análise não é um parâmetro livre: ela é *derivada* do salto para garantir sobreposição mínima de 50 % entre quadros consecutivos, arredondando para a potência de 2 seguinte. Com salto de 480, isso fixa  $n_{fft} = 1024$  e sobreposição efetiva de 53,1 %. O vínculo é imposto pelo código, e não pela configuração, justamente porque a alternativa — fixar a janela em 512 e deixar o salto crescer — produziria sobreposição de 6,2 % e um **ponto cego** entre quadros: transientes situados no intervalo não coberto simplesmente não seriam representados no mapa tempo-frequência.

O AST é a exceção já registrada na Tabela 5: a sua entrada é o mapa  $300 \times 128$  da receita original (Seção 4.3), com janela de 25 ms e salto de 10 ms, e não a Equação 9.

#### Janela consumida pelos modelos de áudio bruto

As cinco arquiteturas de áudio bruto consomem a **janela canônica inteira** de 3 s — 48.000 amostras, o `target_sequence_length` declarado no registro de cada uma. Não há recorte de 1 s: o valor de 16.000 amostras que aparece nos contratos é o `min_sequence_length`, o piso abaixo do qual a entrada é recusada, e não o comprimento efetivamente processado.

Durante o treino, RawNet2, AASIST e RawGAT-ST usam recorte aleatório (`train_crop_strategy = random`) e, na avaliação, a média da pontuação sobre 3 recortes (`eval_crop_strategy = multicrop`). Como a janela canônica cobre o sinal inteiro, os três recortes coincidem e a média devolve exatamente a pontuação do recorte único — os campos aparecem nos artefatos publicados, mas **nenhuma arquitetura recebe *test-time augmentation* efetiva**, e a comparação permanece simétrica nesse ponto.

#### Descritores Adicionais Disponíveis no Pipeline

Além dos três regimes de entrada efetivamente comparados neste *benchmark*, o repositório implementa extratores adicionais como adaptadores `IFeatureExtractor` (`app/domain/features/extractors/`) — descritores de forma espectral, *Constant-Q Transform*, medidas prosódicas e de qualidade vocal, e derivadas temporais. Eles **não são consumidos por nenhum dos 11 modelos avaliados**: existem para extensões futuras do protocolo (Seção 7.3). Suas formulações estão no Apêndice A.3, e a Tabela 6 sintetiza a hierarquia discriminativa que a literatura de *anti-spoofing* reporta para cada tipo —

hierarquia que motivou a escolha do vetor tabular efetivamente avaliado.

**Tabela 6:** Descritores acústicos disponíveis no *pipeline*: síntese da literatura. **Custo Computacional:** estimativa qualitativa (Baixo/Médio/Alto) do consumo de tempo e memória para extração.

| Tipo                         | Dimen.    | Custo Comp. | Validação na Literatura                                                                      |
|------------------------------|-----------|-------------|----------------------------------------------------------------------------------------------|
| Espectrais (forma)           | 6         | Médio       | Sensíveis a artefatos de <i>vocoder</i> na envoltória espectral [28]                         |
| MFCC                         | 13        | Baixo       | Referência clássica; LFCC supera MFCC em <i>spoofing</i> desde 2015 [15, 27]                 |
| Espectrograma Mel            | 80 bandas | Baixo       | Base para CNNs/ <i>Transformers</i> ; captura artefatos de síntese tempo-frequência [31, 32] |
| CQT                          | 84        | Alto        | Resolução logarítmica; CQCC como <i>front-end</i> robusto [28]                               |
| Prosódicas / qualidade vocal | 6         | Médio       | Desempenho isolado inferior; fusão com espectrais melhora resultados [18]                    |

## 5.4 Conjunto de Dados e Protocolo Experimental

### 5.4.1 Composição e Proveniência

O protocolo experimental desta pesquisa fundamenta-se em um *corpus* **pareado** de 15.000 amostras de áudio em português, balanceado entre as classes (50% *bonafide* e 50% *spoof*). A classe genuína reúne locuções do CETUC [51]; a classe artificial é obtida clonando **a mesma locução, do mesmo locutor**, com XTTS-v2 [52].

O pareamento é a decisão metodológica central do protocolo, e opera no nível do *enunciado*: as 15.000 amostras correspondem a 7.500 identificadores de enunciado, cada um presente exatamente duas vezes — o original e o seu clone. Cada amostra *spoof* tem, assim, uma contrapartida *bonafide* de conteúdo linguístico e identidade de locutor idênticos, de modo que a única variável que separa sistematicamente as classes é o *processo de síntese* — e não o texto, o locutor, o canal de gravação ou a fonte do *dataset*. Isso elimina, por construção, a família de correlações espúrias que a literatura de *anti-spoofing* identifica como principal ameaça à validade interna deste tipo de avaliação.

**Tabela 7:** Composição do *corpus* pareado (15.000 amostras) e auditorias de disjunção entre partições. Os valores de sobreposição são recontados a cada execução e gravados no artefato do *benchmark*.

| Dimensão                                                                                                     | Especificação                                                                                  |
|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Composição</b>                                                                                            |                                                                                                |
| Protocolo                                                                                                    | <b>cetuc-xtts-paired</b>                                                                       |
| Classe <i>bonafide</i>                                                                                       | CETUC — 7.500 locuções                                                                         |
| Classe <i>spoof</i>                                                                                          | Clones XTTS-v2 das <i>mesmas</i> locuções — 7.500                                              |
| Pareamento                                                                                                   | 6.081 pares (Tr), 728 (Vl), 691 (Te) — 0 amostras sem contraparte                              |
| Estratégia de partição                                                                                       | Bloco-diagonal, disjunta em locutor $\times$ frase                                             |
| Locutores por partição                                                                                       | 34 (Tr), 11 (Vl), 11 (Te) — sem interseção                                                     |
| Textos distintos por partição ( <b>text_id</b> )                                                             | 599 (Tr), 195 (Vl), 183 (Te) — sem interseção                                                  |
| Índices de frase distintos ( <b>sentence_index</b> )                                                         | 601 (Tr), 196 (Vl), 183 (Te) — sem interseção                                                  |
| Janela e taxa                                                                                                | 3 s a 16 kHz ( $48.000 \times 1$ amostras)                                                     |
| <b>Auditoria de sobreposição (Tr<math>\times</math>Vl, Tr<math>\times</math>Te, Vl<math>\times</math>Te)</b> |                                                                                                |
| Identidade de locutor                                                                                        | 0 / 0 / 0                                                                                      |
| Índice de frase                                                                                              | 0 / 0 / 0                                                                                      |
| Identificador de texto                                                                                       | 0 / 0 / 0                                                                                      |
| Hash de conteúdo (SHA-256)                                                                                   | 0 / 0 / 0                                                                                      |
| Quase-duplicatas espectrais (cosseno $\geq 0,99$ )                                                           | 0 pares entre partições em 272,7 milhões de comparações (maior similaridade observada: 0,9838) |
| <b>Verificação do pareamento</b>                                                                             |                                                                                                |
| Oráculo majoritário (fonte, locutor, texto)                                                                  | 0,50 / 0,50 / 0,50 — valor esperado sob pareamento correto (ver texto)                         |

O *corpus* reúne 56 locutores e um único sistema gerador (XTTS-v2). As 15.000 amostras são uma subamostragem do conjunto pareado completo, sorteada com semente fixa por rodízio uniforme entre locutores e frases, de modo a preservar a cobertura de ambos os eixos sem concentrar a seleção em falantes ou enunciados particulares. Cada forma de onda resulta de recorte *central* da locução original — de duração mediana 4,67 s — para a janela canônica de 3 s; verificou-se que as 15.000 amostras seguem exatamente essa regra e que nenhuma é mais curta que a janela, o que dispensa repetição de conteúdo ou preenchimento com zeros. A janela retém, portanto, cerca de 64 % da locução mediana: o modelo decide sobre um trecho central, não sobre o enunciado inteiro — o que uniformiza o custo computacional entre arquiteturas, mas descarta informação prosódica de início e fim de frase. A Tabela 7 consolida a composição e as auditorias de disjunção executadas sobre o *corpus*.

O oráculo majoritário merece destaque: ele mede a acurácia que um classificador obteria *apenas* olhando o metadado, sem ouvir o áudio. Valor de 0,50 nas três dimensões significa que fonte, locutor e texto não carregam informação de classe — não há atalho

**Tabela 8:** Especificações técnicas do protocolo de pré-processamento acústico e partições de amostragem.

| Parâmetro Metodológico                     | Especificação Nominal                                                                                                               |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pré-processamento Acústico</b>          |                                                                                                                                     |
| Taxa de amostragem                         | 16 kHz                                                                                                                              |
| Formato e Resolução quantizada             | Monofônico, PCM linear de 16 bit                                                                                                    |
| Janelamento temporal canônico              | 3 s por recorte central, sem <i>zero-padding</i> e sem repetição: o par é descartado quando qualquer lado é mais curto que a janela |
| Dimensionalidade do vetor                  | $\mathbf{x} \in \mathbb{R}^{48.000 \times 1}$ (domínio do tempo discreto)                                                           |
| Representações de entrada                  | <i>Raw-audio</i> , Espectrogramas Mel ou vetor tabular de 183 descritores (MFCC, LFCC, RASTA-PLP e estatísticas temporais)          |
| <b>Particionamento por Disjunção Dupla</b> |                                                                                                                                     |
| Distribuição dos subconjuntos              | Bloco-diagonal, disjunta por locutor e por frase (não estratificada por proporção fixa)                                             |
| Cardinalidade por partição                 | 12.162 (Tr), 1.456 (Vl), 1.382 (Te)                                                                                                 |
| <i>Augmentation</i> de ruído (treino)      | AWGN na forma de onda, $SNR \in \{30, 20, 10 \text{ dB}\}$                                                                          |
| Avaliação de robustez (teste)              | AWGN na forma de onda, $SNR \in \{30, 20, 10 \text{ dB}\}$ ( <b>casado</b> ) e 5 dB ( <b>não visto</b> )                            |

disponível para o modelo explorar.

#### 5.4.2 Pré-processamento e Particionamento

Para assegurar homogeneidade no espaço de características acústicas, estabeleceu-se um fluxo de padronização de sinais e amostragem estratificada. Esse procedimento preserva a proporção de 1 : 1 entre as classes em todas as partições experimentais, mitigando vieses probabilísticos *a priori*. O balanceamento é consequência direta do pareamento: como cada *spoof* é o clone de uma locução genuína, a razão 1 : 1 vale por construção em qualquer partição, sem necessidade de estratificação por proporção. Os parâmetros operacionais desse fluxo estão formalizados na Tabela 8.

A matriz unidimensional resultante ( $\mathbf{x} \in \mathbb{R}^{48.000 \times 1}$ ) atua como estrutura canônica de entrada tanto para o processamento direto ponta a ponta (*end-to-end*) quanto para os módulos extratores de representações tempo-frequência. Em paralelo, a inserção controlada de Ruído Branco Gaussiano Aditivo (AWGN) no conjunto de teste simula uma degradação acústica simples e reprodutível, permitindo mensurar a resiliência dos classificadores sob diferentes níveis de SNR.

A escolha do AWGN merece justificativa, pois a literatura recente de *anti-spoofing* — em particular a trilha *Deepfake* do ASVspoof 2021 [16] — enfatiza degradações por codec e compressão, mais representativas de canais reais. Três razões sustentam o AWGN como

degradação primária aqui. Primeira, ele é o único parâmetro de degradação com **definição fechada de intensidade**: a SNR fixa a razão de potências de forma exata e reprodutível, o que permite comparar onze modelos no *mesmo* ponto de degradação — garantia que codecs não oferecem, pois seu efeito depende de conteúdo, taxa de bits e implementação. Segunda, por ser independente do sinal, o AWGN não interage com o pareamento: aplicado com sementes distintas a original e clone, degrada ambos sob a mesma distribuição, sem introduzir assimetria entre as classes. Terceira, ele isola a questão de interesse da QP2 — a relação entre desempenho limpo e robustez — de efeitos de codec que confundiriam degradação acústica com artefato de compressão, este sim correlacionado ao processo de síntese.

O custo dessa escolha é reconhecido: o AWGN cobre apenas degradação aditiva estacionária, e a robustez a canais reais permanece não medida (Seção 7.2).

O ruído é injetado, com semente fixa, **na forma de onda e antes de qualquer *frontend***, de modo idêntico para as onze entradas (protocolo *waveform-awgn-v2*). Essa uniformidade é condição para que a comparação de robustez entre famílias signifique alguma coisa: injetar ruído no espaço de representação de cada família — log-Mel para as redes espectrais, vetor tabular para os clássicos — mediria degradações de naturezas diferentes sob o mesmo rótulo de SNR, já que a energia do ruído passaria a ser definida sobre grandezas não comparáveis. Aplicando o AWGN ao sinal e recalculando o *frontend* sobre o sinal degradado, todos os modelos veem exatamente a mesma perturbação acústica, e a cadeia de extração de características de cada um responde a ela como responderia em produção.

As sementes de ruído do treino e da avaliação são disjuntas, o que impede que um modelo reencontre em teste a realização exata de ruído vista em treino.

### 5.4.3 Controles Contra Vazamento e Escopo de Validade

O protocolo foi construído para eliminar, na origem, as fontes de vazamento que comprometem a validade interna de avaliações de *anti-spoofing*. Três controles operam em conjunto.

**Pareamento.** Como cada *spoof* é o clone da *mesma* locução do *mesmo* locutor, texto e identidade ficam constantes entre as classes. Um modelo não pode separar *bonafide* de *spoof* olhando o que é dito ou quem fala, porque as duas classes contêm exatamente as mesmas frases e as mesmas vozes.

**Disjunção dupla.** A partição é bloco-diagonal em locutor  $\times$  frase: nenhum locutor e nenhuma frase do teste aparece em treino ou validação. A auditoria (Tabela 7) reconstruiu sobreposição nula nas quatro dimensões — identidade de locutor, índice de frase, identificador de texto e *hash* SHA-256 do conteúdo — para os três pares de partições.

Esse particionamento é **próprio deste trabalho**, e não a divisão oficial distribuída com o CETUC. A opção merece justificativa explícita, porque a divisão oficial existe e

seria a escolha natural. Ela foi projetada para reconhecimento automático de fala, tarefa em que a exigência é a independência de locutor: de fato, ela é disjunta por locutor (44/5/7 falantes, sem interseção). Não é, porém, disjunta por conteúdo — 582 dos 593 textos da sua partição de teste ( $\approx 98\%$ ) reaparecem na de treino. Para reconhecimento de fala isso é irrelevante, e até desejável; para detecção de fala sintética é inaceitável, pois permitiria ao modelo associar enunciados específicos a rótulos em vez de aprender assinaturas de síntese. A partição adotada aqui é, nesse sentido, *estritamente mais forte* que a oficial: preserva a disjunção de locutor e acrescenta a de conteúdo.

Como consequência, as duas divisões se cruzam — o conjunto de teste deste trabalho extrai 1256 de suas 1382 amostras da partição de treino oficial do CETUC. Isso não introduz vazamento algum sob o protocolo aqui definido, cujas auditorias são recontadas sobre a partição efetivamente usada, mas impede a comparação direta com resultados publicados sobre a divisão oficial do CETUC. O campo `cetuc_official_split` é preservado no arquivo do *corpus* para que essa correspondência permaneça auditável.

**Congelamento do teste.** A partição de teste foi definida e criptograficamente selada (SHA-256 do arquivo e da identidade dos membros do arquivo) *antes* do treinamento confirmatório, com a declaração de que não foi usada para escolher arquitetura, hiperparâmetros ou correções. Cada artefato de resultado carrega esse selo, e o *runner* recusa executar se a partição não corresponder ao *hash* registrado.

O *oráculo majoritário* — a acurácia que um classificador obteria decidindo apenas pelo metadado, sem acesso ao áudio — vale 0,50 para fonte, locutor e texto. Cabe, porém, uma qualificação metodológica que o número sozinho não expressa: sob pareamento perfeito, esse resultado é **consequência algébrica do protocolo, não medida empírica**. Como cada locução genuína tem exatamente um clone, qualquer grupo definido por locutor ou por texto contém tantos reais quanto falsos, e o voto majoritário é necessariamente 0,50. Verificou-se que os 11 grupos de locutor e os 183 de texto do conjunto de teste são todos exatamente balanceados.

O oráculo, portanto, *confirma que o pareamento foi corretamente aplicado* — um valor diferente de 0,50 denunciaria pares quebrados — mas não constitui evidência independente de ausência de atalho. No caso de `source_ids` a limitação é dupla: o conjunto de teste tem uma única fonte, de modo que a métrica não teria poder discriminante nem sem o pareamento.

A evidência independente está nas auditorias de disjunção, que testam proposições que o pareamento *não* implica: a ausência de sobreposição de *hash* de conteúdo garante que nenhuma forma de onda do teste aparece em treino ou validação, e a disjunção de locutor e de frase garante que o modelo não viu nem aquelas vozes nem aqueles enunciados. São essas três contagens nulas — e o pareamento em si — que sustentam a validade interna do protocolo.

Duas ressalvas delimitam o escopo de validade. Primeira, o gerador sintético é **único**

(XTTS-v2): os resultados medem a detecção *daquela* processo de síntese, e não a generalização para sistemas de geração não vistos, que é o regime avaliado por campanhas como o ASVspoof com dezenas de ataques distintos. Segunda, o *corpus* é de fonte única e condições de gravação homogêneas, de modo que a robustez a degradações reais de canal (codecs, reverberação, ruído não estacionário) permanece não medida — o AWGN cobre apenas a degradação aditiva estacionária. A validação *cross-dataset* e *cross-generator* é, portanto, o passo seguinte natural (Seção 7.3), não uma correção de vício deste protocolo.

## 5.5 Métricas de Avaliação

O protocolo experimental reporta um conjunto padronizado de métricas de classificação binária (*bonafide* vs. *spoof*), calculadas a partir da pontuação de *spoof*  $p_{\text{fake}} \in [0, 1]$  predita por cada modelo.

### Acurácia, Precisão, Revocação e F1-score

A classe positiva é *spoof*. Sejam  $TP$ ,  $TN$ ,  $FP$  e  $FN$  os verdadeiros positivos, verdadeiros negativos, falsos positivos e falsos negativos, obtidos ao limiar de decisão  $\theta$  pela regra  $\hat{y} = \mathbb{I}[p_{\text{fake}} \geq \theta]$  — a desigualdade é *não estrita*, em conformidade com a implementação:

$$\text{Acurácia} = \frac{TP + TN}{TP + TN + FP + FN}, \quad F1 = 2 \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}} \quad (10)$$

em que  $\text{Precisão} = TP/(TP + FP)$  e  $\text{Revocação} = TP/(TP + FN)$  na Equação 10.

### Duas convenções de nomeação de erro convivem neste texto

Este trabalho reporta métricas sob *duas* convenções herdadas de literaturas distintas, e elas nomeiam os mesmos eventos de forma **invertida**. A Tabela 9 fixa a correspondência; sem ela, o confronto entre a matriz de confusão e a curva DET da Seção 6.4 parece contraditório.

**Tabela 9:** Correspondência entre a convenção de classificação (classe positiva = *spoof*) e a convenção ASV/*t*-DCF herdada do ASVspoof.

| Evento                                                   | Classificação           | ASV / <i>t</i> -DCF           |
|----------------------------------------------------------|-------------------------|-------------------------------|
| Rejeitar um <i>bonafide</i><br>(usuário genuíno barrado) | Falso positivo ( $FP$ ) | $P_{\text{miss}}^{\text{cm}}$ |
| Aceitar um <i>spoof</i> (ataque passa)                   | Falso negativo ( $FN$ ) | $P_{\text{fa}}^{\text{cm}}$   |

A inversão tem origem no referencial: a convenção de classificação toma o *spoof* como classe positiva, enquanto a de verificação de locutor toma o usuário genuíno como alvo — de modo que “perder” (*miss*) significa perder o alvo legítimo, e “alarme falso” significa

aceitar o impostor. Nas Seções 5.5 e 6.4, os símbolos  $P_{\text{miss}}$  e  $P_{\text{fa}}$  seguem *sempre* a coluna da direita; os termos “falso positivo” e “falso negativo” em texto corrido seguem *sempre* a do meio.

O limiar é o padrão  $\theta = 0,5$  para **os 11 modelos**, sem exceção, sob a política declarada nos artefatos como `fixed_comparison`. A uniformidade é intencional: calibrar o ponto de operação de cada modelo separadamente melhoraria os números individuais, mas tornaria a comparação entre eles dependente da qualidade de cada calibração, e não apenas do poder discriminativo. Com limiar comum, as métricas dependentes de limiar medem os detectores no mesmo ponto de operação; AUC-ROC, EER e *min t*-DCF varrem todos os limiares e são, por construção, independentes dessa escolha.

O *pipeline* calcula também um limiar calibrado por modelo e a acurácia que ele produziria, registrados nos artefatos como `calibrated_threshold` e `accuracy_at_calibrated_threshold`; esses valores não alimentam as tabelas comparativas, mas permitem estimar o ganho disponível por recalibração — relevante para o caso do SVM sob ruído não visto (Seção 6.4.5).

## AUC-ROC

A curva ROC (*Receiver Operating Characteristic*) relaciona a taxa de verdadeiros positivos (TPR) à taxa de falsos positivos (FPR) variando-se o limiar de decisão  $\theta$  sobre  $p_{\text{fake}}$ . A AUC-ROC é a área sob essa curva: valor 1,0 indica separação perfeita entre as classes e 0,5 equivale a uma decisão aleatória.

## Taxa de Erro Igual (EER)

O EER é o ponto de operação  $\theta^*$  em que a taxa de falsos positivos iguala a taxa de falsos negativos ( $\text{FNR} = 1 - \text{TPR}$ ):

$$\theta^* = \arg \min_{\theta} |\text{FPR}(\theta) - \text{FNR}(\theta)|, \quad \text{EER} = \frac{\text{FPR}(\theta^*) + \text{FNR}(\theta^*)}{2} \quad (11)$$

Na definição contínua, o EER é o ponto em que  $\text{FPR}(\theta) = \text{FNR}(\theta)$ . Com um conjunto de teste finito, porém, ambas as taxas são funções em escada de  $\theta$  e o cruzamento exato em geral **não existe**: entre duas pontuações consecutivas, uma taxa salta por cima da outra. A Equação 11 adota, por isso, a convenção operacional de tomar o limiar que minimiza a discrepância entre as duas taxas e reportar a média delas nesse ponto — o que reduz à definição clássica sempre que o cruzamento existe. Valores menores indicam maior separabilidade entre as distribuições de pontuação das classes *bonafide* e *spoof*, independentemente da escolha de limiar de operação.

**Score de ordenação nos classificadores clássicos.** AUC-ROC, EER e *t*-DCF\* medem *ordenação*, não *calibração*. Para SVM e *Random Forest*, cujas probabilidades passam por calibração isotônica, essas três métricas são calculadas sobre a **margem bruta**

**do detector**, anterior à calibração; a acurácia, a precisão, a revocação e o ECE continuam sobre a probabilidade calibrada, que é o que a calibração existe para corrigir. A razão é que a isotônica é uma função escada: no SVM deste protocolo ela colapsa 1382 margens distintas em 52 degraus, e os empates resultantes custam 0,75 ponto percentual de AUC (0,9731 na margem contra 0,9656 na probabilidade calibrada) — uma perda que mede o achatamento da escala, não a qualidade do detector. Cada artefato de resultado declara qual *score* alimentou cada métrica no campo `ranking_score_source`, e a coluna `ranking_score` dos arquivos de predição publicados permite reproduzir os valores reportados.

### *t*-DCF (proxy interno)

O *t*-DCF (*Tandem Detection Cost Function*) normalizado, proposto por Kinnunen et al. [53] para o protocolo ASVspoof 2019, pondera em conjunto os custos de falha do detector (*countermeasure*, CM) e de um sistema de verificação de locutor (ASV) operando em tandem:

$$t\text{-DCF}(\theta) = \frac{C_1 \cdot P_{\text{miss}}^{\text{cm}}(\theta) + C_2 \cdot P_{\text{fa}}^{\text{cm}}(\theta)}{\min(C_1, C_2)} \quad (12)$$

com os coeficientes de custo dados por

$$C_1 = P_{\text{tar}}(C_{\text{miss}} - C_{\text{miss}} P_{\text{miss}}^{\text{asv}}) - P_{\text{non}} C_{\text{fa}} P_{\text{fa}}^{\text{asv}}, \quad C_2 = C_{\text{fa}} P_{\text{spoof}}(1 - P_{\text{miss,spoof}}^{\text{asv}}). \quad (13)$$

Na Equação 13,  $P_{\text{miss}}^{\text{cm}}(\theta)$  é a probabilidade de o detector rejeitar (classificar como *spoof*) uma amostra *bonafide*, e  $P_{\text{fa}}^{\text{cm}}(\theta)$  é a probabilidade de aceitar (classificar como *bonafide*) uma amostra *spoof*; o *min t-DCF* reportado nas tabelas minimiza a Equação 12 sobre  $\theta$ . Este trabalho não dispõe de um sistema ASV pareado ao *corpus* experimental: os parâmetros do ASV ( $P_{\text{fa}}^{\text{asv}} = 0,0524$ ,  $P_{\text{miss}}^{\text{asv}} = 0,0104$ ,  $P_{\text{miss,spoof}}^{\text{asv}} = 0,0058$ ) e as prioris ( $P_{\text{tar}} = 0,9405$ ,  $P_{\text{non}} = 0,0095$ ,  $P_{\text{spoof}} = 0,05$ ,  $C_{\text{miss}} = 1$ ,  $C_{\text{fa}} = 10$ ) são fixados nos valores-padrão do particionamento LA do ASVspoof 2019 [53], e apenas o termo do detector ( $P_{\text{miss}}^{\text{cm}}$ ,  $P_{\text{fa}}^{\text{cm}}$ ) é medido sobre os modelos deste *benchmark*. Por essa razão, o valor reportado é tratado como **proxy interno** — comparável entre os 11 modelos deste trabalho, mas não equivalente ao *min t-DCF* oficial de um protocolo com sistema ASV real — e identificado com asterisco (*t*-DCF\*) nas tabelas da Seção 6.

Duas observações delimitam o alcance dessa escolha. A primeira é que a formulação adotada é a do ASVspoof 2019, e o campo revisou suas métricas primárias desde então: as edições seguintes [16, 17] ajustaram tanto o modelo de custo quanto a arquitetura de avaliação em *tandem*. Manter a formulação de 2019 é coerente com o propósito aqui — comparar onze detectores entre si sob custo assimétrico declarado —, mas significa que os valores não devem ser confrontados com tabelas de campanhas posteriores. A segunda é que, fixados os parâmetros do ASV, o *t*-DCF\* torna-se função apenas de  $P_{\text{miss}}^{\text{cm}}$  e  $P_{\text{fa}}^{\text{cm}}$ : ele não mede o desempenho de um sistema em *tandem* real, e sim *como cada detector se*

*sairia* se acoplado a um ASV de referência — que é exatamente a leitura pretendida em um *benchmark* de contramedidas isoladas.

## 5.6 Ferramentas de Software Livre

Para garantir reprodutibilidade e escalabilidade, a arquitetura foi desenvolvida com ferramentas de código aberto e bibliotecas amplamente adotadas pela comunidade científica, conforme descrito a seguir:

- **Pré-processamento e Detecção de Atividade de Voz (VAD):** O *pipeline* integra o modelo *Silero VAD* (via *Torch Hub*) com mecanismo de contingência (*fallback*) baseado em limiar de energia. **Nenhum dos dois foi aplicado ao *corpus* experimental:** o VAD está desativado no protocolo do *benchmark* (`vad: disabled`), por decisão metodológica que preserva a simetria do pareamento (Seção 5.2). As operações de controle automático de ganho (AGC) e reamostragem foram gerenciadas pela biblioteca `librosa`, assegurando eficiência computacional em CPU.
- **Extração de Características Acústicas:** A biblioteca `librosa` foi incorporada para a extração de estatísticas temporais e coeficientes cepstrais na escala Mel (MFCC); os coeficientes RASTA-PLP são calculados pelo extrator próprio do repositório (Seção 5.3).
- **Modelos de Fundação (*Foundation Models*):** WavLM e HuBERT foram integrados via ecossistema `transformers` (Hugging Face). Neste protocolo, os *backbones* permanecem congelados e apenas a cabeça de classificação é treinada sobre os *embeddings* extraídos.
- **Arquiteturas Especializadas:** As redes RawNet2, AASIST e RawGAT-ST foram reimplementadas em TensorFlow/Keras seguindo as implementações de referência publicadas pelos autores em PyTorch (licença MIT), preservando os blocos arquiteturais descritos na Seção 4.2.
- **Treinamento e Classificação:** O arcabouço TensorFlow/Keras foi utilizado no desenvolvimento e treinamento das arquiteturas neurais espectrais e de áudio bruto; WavLM Original e HuBERT Original usam um *runner* dedicado em PyTorch/`transformers`; os classificadores clássicos foram implementados com Scikit-Learn.
- **Mecanismos de Interpretabilidade:** O repositório implementa o cômputo de valores SHAP [54] sobre o vetor tabular (`TreeExplainer` para o *Random Forest*, `KernelExplainer` para o SVM) e mapas de ativação Grad-CAM para as redes espectrais, estes últimos em **TensorFlow/Keras** — o mesmo arcabouço das arquiteturas explicadas, sem dependência de *frameworks* externos de atribuição. Ambos operam como ferramenta *offline*, fora da interface (Apêndice B).

- **Disponibilidade:** o código-fonte completo do *pipeline*, os *scripts* de treinamento e *benchmark* e as configurações utilizadas estão publicamente disponíveis em <https://github.com/thierrybraga/XFakeSong>, sob licença MIT.

## 5.7 Protocolo de Treinamento

O regime de treinamento adota **orçamento fixo de 100 épocas para todas as nove arquiteturas neurais**, *sem parada antecipada*. A escolha é deliberada e metodologicamente relevante: a parada antecipada usa o conjunto de validação para decidir *quando* parar, o que introduz um grau de liberdade que varia entre modelos e torna o custo de treinamento — e o próprio ponto de convergência — dependente de um critério que não é o mesmo para todos. Com orçamento fixo, todas as arquiteturas recebem exatamente o mesmo esforço de otimização, e a validação cumpre um único papel: **selecionar o *checkpoint*** de menor perda de validação limpa, entre as 100 épocas efetivamente executadas.

A consequência prática é que o modelo avaliado nem sempre é o da última época. A época selecionada por modelo, e a distância entre ela e o fim do treino, constam da análise de estabilidade (Seção 6.4) — é essa distância que revela os dois casos de oscilação do recorte.

O *augmentation* de ruído no treino segue o mesmo protocolo **waveform-awgn-v2** da avaliação: AWGN aplicado **à forma de onda**, antes de qualquer *frontend*, com uma cópia ruidosa por amostra (dobrando o conjunto de ajuste de 12.162 para 24.324 exemplos) e SNR sorteado uniformemente em {30, 20, 10 dB}. O nível de 5 dB é **deliberadamente excluído** do treino: é ele que, na avaliação, mede generalização a uma degradação fora da distribuição vista (Seção 6.4.5). As sementes de ruído de treino e de avaliação são disjuntas. O mesmo procedimento vale para SVM e *Random Forest*: o ruído entra na forma de onda e o vetor tabular é recalculado sobre o sinal degradado, e não perturbado no espaço de características.

Todos os experimentos usam semente aleatória fixa (42) para particionamento, inicialização e geração de ruído, registrada nos artefatos de cada execução. Durante o processo, coletaram-se históricos de perda e acurácia, matrizes de confusão, curvas ROC e de Precisão-Revocação, métricas por classe, tamanho em disco e latência de inferência — esta última medida como a mediana de 30 inferências consecutivas de amostra única, após duas execuções de aquecimento (Seção 6.1).

Para os classificadores estatísticos clássicos, o ajuste é uma busca em grade com validação cruzada de 5 dobras **agrupadas por frase**, necessária porque o *corpus* é pareado: sem o agrupamento, o original e o seu clone cairiam em dobras distintas, e a validação cruzada estaria medindo memorização em vez de generalização. O SVM percorre 15 candidatos (75 ajustes) e o *Random Forest*, 108 candidatos (540 ajustes), seguidos de reajuste final e calibração isotônica.

**Tabela 10:** Especificações de infraestrutura, hardware e parametrização do ambiente de *benchmarking*.

| Componente de Infraestrutura              | Configuração e Parâmetros Operacionais                                                               |
|-------------------------------------------|------------------------------------------------------------------------------------------------------|
| Sistema operacional base                  | Windows 11 + WSL2 (Ambiente nativo Linux Ubuntu)                                                     |
| Fase de treinamento                       | Execução em GPU com precisão mista e controle de VRAM                                                |
| Ambiente de avaliação de inferência       | Por <i>runtime</i> : GPU RTX 3060 (7 Keras + 2 PyTorch), CPU (SVM e RF, scikit-learn)                |
| Modelos conexionistas profundos           | TensorFlow 2.21.0 / Keras 3.15.0, com persistência de <i>checkpoints</i>                             |
| Modelos baseados em SSL                   | WavLM Original e HuBERT Original via <b>transformers</b> 4.45.2 sobre PyTorch 2.5.1+cu124            |
| Modelos estatísticos clássicos            | scikit-learn 1.8.0, multi-core, com busca em grade sob validação cruzada agrupada por <i>cluster</i> |
| Volume do <i>corpus</i> consolidado       | 15.000 amostras pareadas e balanceadas (7.500 reais / 7.500 clones XTTS-v2)                          |
| Particionamento de dados ( <i>Split</i> ) | Disjunção dupla locutor $\times$ frase (12.162 treino, 1.456 validação, 1.382 teste)                 |
| Especificação padrão de sinal             | Monofônico, 16 kHz, 3 s ( $\mathbf{x} \in \mathbb{R}^{48.000 \times 1}$ em <i>raw-audio</i> )        |

## 6 Experimentos e Resultados

### 6.1 Ambiente Experimental e Hardware

O ambiente computacional dedicado à execução do *benchmark* foi estruturado sobre *Windows Subsystem for Linux 2* (WSL2), em Windows 11, com apoio de contêineres Docker para isolar dependências e facilitar a reprodutibilidade. Na fase de treinamento, o *pipeline* foi configurado com suporte a processamento gráfico (GPU), precisão mista (*mixed precision*) e controle de alocação de memória de vídeo (VRAM) para otimizar a vazão dos lotes (*batch size*).

As métricas de inferência foram extraídas no ambiente de execução de cada modelo, sobre Python 3.11.15, em **três *runtimes* distintos**: as sete arquiteturas Keras (CCT, AST, Res2Net, Conformer, RawNet2, AASIST e RawGAT-ST) em TensorFlow 2.21.0 sobre GPU; WavLM e HuBERT Original em PyTorch 2.5.1 sobre GPU (CUDA); e SVM e *Random Forest* em scikit-learn 1.8.0 sobre CPU, com paralelismo multi-core. Cada artefato registra o campo `latency_profile.cross_runtime_comparable`, sempre **false**: uma latência de 15,77 ms medida em PyTorch e uma de 44,46 ms medida em Keras separam *runtime* e dispositivo, não arquitetura. As latências da Tabela 13 devem, portanto, ser lidas como ordens de grandeza dentro de cada *runtime*, e não como um *ranking* único (Seção 6.4). O sumário das configurações de hardware e infraestrutura consta na Tabela 10.

**Tabela 11:** Custo de treinamento por modelo, na RTX 3060 (neurais e SSL) ou em CPU multi-core (clássicos). *Ajuste* é o tempo do treinamento; *Total* inclui preparação e avaliação.

| Modelo                           | Estratégia                             | Épocas | Lote | Ajuste  | Total   |
|----------------------------------|----------------------------------------|--------|------|---------|---------|
| RawGAT-ST                        | orçamento fixo                         | 100    | 16   | 26,85 h | 26,86 h |
| AASIST                           | orçamento fixo                         | 100    | 24   | 15,44 h | 15,45 h |
| AST                              | orçamento fixo                         | 100    | 8    | 11,01 h | 11,02 h |
| RawNet2                          | orçamento fixo                         | 100    | 16   | 4,29 h  | 4,30 h  |
| Conformer                        | orçamento fixo                         | 100    | 32   | 1,67 h  | 1,68 h  |
| Res2Net                          | orçamento fixo                         | 100    | 32   | 1,32 h  | 1,33 h  |
| CCT                              | orçamento fixo                         | 100    | 32   | 0,92 h  | 0,93 h  |
| SVM                              | grade + CV                             | —      | —    | 0,99 h  | 1,00 h  |
| <i>Random Forest</i>             | grade + CV                             | —      | —    | 0,58 h  | 0,59 h  |
| WavLM Original                   | cabeça sobre <i>backbone</i> congelado | 100    | 128  | 62,5 s  | 5,4 min |
| HuBERT Original                  | cabeça sobre <i>backbone</i> congelado | 100    | 128  | 79,9 s  | 4,7 min |
| <b>Total do <i>benchmark</i></b> |                                        |        |      |         | 63,34 h |

## 6.2 Custo de Treinamento

A Tabela 11 reporta o custo de cada modelo. A coluna *Ajuste* mede o treinamento propriamente dito; a coluna *Total* inclui preparação de dados e avaliação completa (limpa e nos quatro níveis de SNR). O *benchmark* completo consumiu 63,3 **horas** de processamento.

Três leituras decorrem da tabela, e a primeira exige cautela na comparação.

**Os dois SSL não são comparáveis aos demais nesta coluna.** Seus 62,5 e 79,9 segundos medem apenas o ajuste da cabeça MLP sobre *embeddings* já extraídos; o *backbone* permanece congelado e sua passagem direta ocorre uma única vez, na preparação. É por isso que o ajuste representa apenas 19% e 29% do tempo total desses modelos, contra mais de 98% em todos os outros. O custo real de usar um *backbone* SSL não está no treino, e sim no pré-treinamento de 94 mil horas de fala que ele carrega — custo que este trabalho herda sem pagar, e que qualquer comparação de eficiência de treinamento precisa declarar.

**O custo não acompanha o número de parâmetros.** O RawGAT-ST, com 650.204 parâmetros, treina em 26,85 h; o AST, com 85,5 milhões — 131 vezes mais —, treina em 11,01 h, menos da metade. O que domina não é o tamanho do modelo, e sim o custo por passo da topologia: os grafos de atenção do RawGAT-ST e do AASIST constroem matrizes de adjacência dinâmicas a cada lote, operação de baixa intensidade aritmética que a GPU não amortiza bem. Os dois modelos mais lentos do recorte são justamente os dois mais compactos.

**O melhor custo-benefício de treinamento é do CCT**, que alcança a 2.<sup>a</sup> melhor acurácia limpa (99,57%) em 0,92 h — 29 vezes menos que o RawGAT-ST, que termina em último em todas as métricas de desempenho. Somados, RawGAT-ST e AASIST consomem

42,3 h, ou 67% de todo o *benchmark*, e entregam a 11.<sup>a</sup> e a 7.<sup>a</sup> posições em acurácia limpa.

No cômputo geral, avaliaram-se 11 modelos consolidados: Random Forest, SVM, CCT, AST, Res2Net, Conformer, RawNet2, AASIST, RawGAT-ST, WavLM Original e HuBERT Original. Extensões de fusão tardia via *ensemble* adaptativo permanecem delineadas na Seção 7.3.

### 6.3 Resultados Consolidados

Os resultados oficiais, tabelas de eficiência, robustez, estabilidade, gráficos comparativos e matrizes de confusão são incorporados a partir do fragmento gerado automaticamente em `tabelas_benchmark.tex`. Esse procedimento evita divergência entre os artefatos experimentais e o texto principal. O fragmento constitui a fonte única para as Tabelas 12, 13, 14 e 15, além das Figuras 2, 3, 4, 5, 6, 7, 8 e 9. O protocolo de treinamento e as ferramentas de software que geraram esses artefatos estão descritos nas Seções 5.6 e 5.7.

**Tabela 12:** Resultados consolidados no conjunto de teste limpo.

| Modelo          | Acur.  | EER    | $t$ -DCF* | AUC   | F1     | Lat. (ms) | Treino | Status |
|-----------------|--------|--------|-----------|-------|--------|-----------|--------|--------|
| Random Forest   | 91,53% | 6,95%  | 0,1966    | 0,984 | 91,22% | 56,43     | CV+fit | OK     |
| SVM             | 93,20% | 6,95%  | 0,1457    | 0,973 | 92,85% | 0,90      | CV+fit | OK     |
| CCT             | 99,57% | 0,43%  | 0,0085    | 1,000 | 99,57% | 31,83     | 100    | OK     |
| AST             | 99,71% | 0,14%  | 0,0041    | 0,999 | 99,71% | 44,46     | 100    | OK     |
| Res2Net         | 97,76% | 2,17%  | 0,0513    | 0,998 | 97,74% | 46,41     | 100    | OK     |
| Conformer       | 98,91% | 0,29%  | 0,0054    | 1,000 | 98,91% | 50,46     | 100    | OK     |
| RawNet2         | 95,88% | 3,18%  | 0,0762    | 0,997 | 95,72% | 90,08     | 100    | OK     |
| AASIST          | 94,72% | 2,60%  | 0,0721    | 0,989 | 94,93% | 53,38     | 100    | OK     |
| RawGAT-ST       | 87,55% | 11,87% | 0,3149    | 0,947 | 88,55% | 54,45     | 100    | OK     |
| WavLM Original  | 96,09% | 3,62%  | 0,0792    | 0,996 | 96,12% | 15,77     | 100    | OK     |
| HuBERT Original | 93,92% | 5,93%  | 0,1308    | 0,987 | 93,93% | 15,98     | 100    | OK     |

$t$ -DCF\*: proxy interno de custo normalizado, sem acoplamento a um sistema ASV oficial.

**Tabela 13:** Eficiência computacional dos modelos consolidados.

| Modelo          | Parâmetros | Tam. (MB) | Lat. (ms) | Acur.  | EER    |
|-----------------|------------|-----------|-----------|--------|--------|
| Random Forest   | —          | 52,04     | 56,43     | 91,53% | 6,95%  |
| SVM             | —          | 11,31     | 0,90      | 93,20% | 6,95%  |
| CCT             | 1.819.715  | 7,13      | 31,83     | 99,57% | 0,43%  |
| AST             | 85.523.714 | 326,73    | 44,46     | 99,71% | 0,14%  |
| Res2Net         | 23.707.438 | 91,01     | 46,41     | 97,76% | 2,17%  |
| Conformer       | 28.632.962 | 110,23    | 50,46     | 98,91% | 0,29%  |
| RawNet2         | 7.004.418  | 26,90     | 90,08     | 95,88% | 3,18%  |
| AASIST          | 396.312    | 1,88      | 53,38     | 94,72% | 2,60%  |
| RawGAT-ST       | 650.204    | 2,80      | 54,45     | 87,55% | 11,87% |
| WavLM Original  | 94.775.935 | 361,63    | 15,77     | 96,09% | 3,62%  |
| HuBERT Original | 94.765.711 | 361,58    | 15,98     | 93,92% | 5,93%  |

Nota sobre latência: mediana de 30 inferências de amostra única (duas execuções de aquecimento), medida no ambiente de execução de cada modelo — GPU (RTX 3060) para os sete modelos Keras e os dois SSL em PyTorch; CPU para SVM e Random Forest em scikit-learn. Os valores não são, portanto, diretamente comparáveis entre *runtimes* (Seção 6.1).

**Tabela 14:** Robustez a ruído AWGN, aplicada à forma de onda antes dos frontends, por modelo consolidado. Os níveis de 30, 20 e 10 dB são **casados** com o *augmentation* de treino; 5 dB é **não visto**. A última coluna traz a acurácia no PIOR dos 11 locutores do teste (nenhum visto no treino), medida em áudio limpo.

| Modelo          | Limpo  | 30 dB  | 20 dB  | 10 dB  | 5 dB <sup>†</sup> | Pior locutor |
|-----------------|--------|--------|--------|--------|-------------------|--------------|
| Random Forest   | 91,53% | 86,69% | 85,31% | 82,71% | 74,96%            | 73,39%       |
| SVM             | 93,20% | 88,42% | 87,63% | 85,53% | 58,68%            | 54,03%       |
| CCT             | 99,57% | 98,99% | 96,31% | 90,30% | 81,84%            | 97,58%       |
| AST             | 99,71% | 98,05% | 95,08% | 88,21% | 84,88%            | 98,41%       |
| Res2Net         | 97,76% | 95,73% | 94,79% | 92,40% | 88,13%            | 87,30%       |
| Conformer       | 98,91% | 97,25% | 95,95% | 91,46% | 86,25%            | 90,32%       |
| RawNet2         | 95,88% | 95,30% | 94,14% | 90,38% | 84,01%            | 74,19%       |
| AASIST          | 94,72% | 95,01% | 95,08% | 91,68% | 84,73%            | 88,10%       |
| RawGAT-ST       | 87,55% | 87,63% | 85,38% | 77,21% | 77,64%            | 65,08%       |
| WavLM Original  | 96,09% | 93,27% | 91,46% | 88,35% | 85,96%            | 81,45%       |
| HuBERT Original | 93,92% | 91,24% | 88,21% | 85,75% | 80,17%            | 70,97%       |

<sup>†</sup> SNR ausente do *augmentation* de treino: é a única coluna que mede generalização a uma degradação fora da distribuição vista.

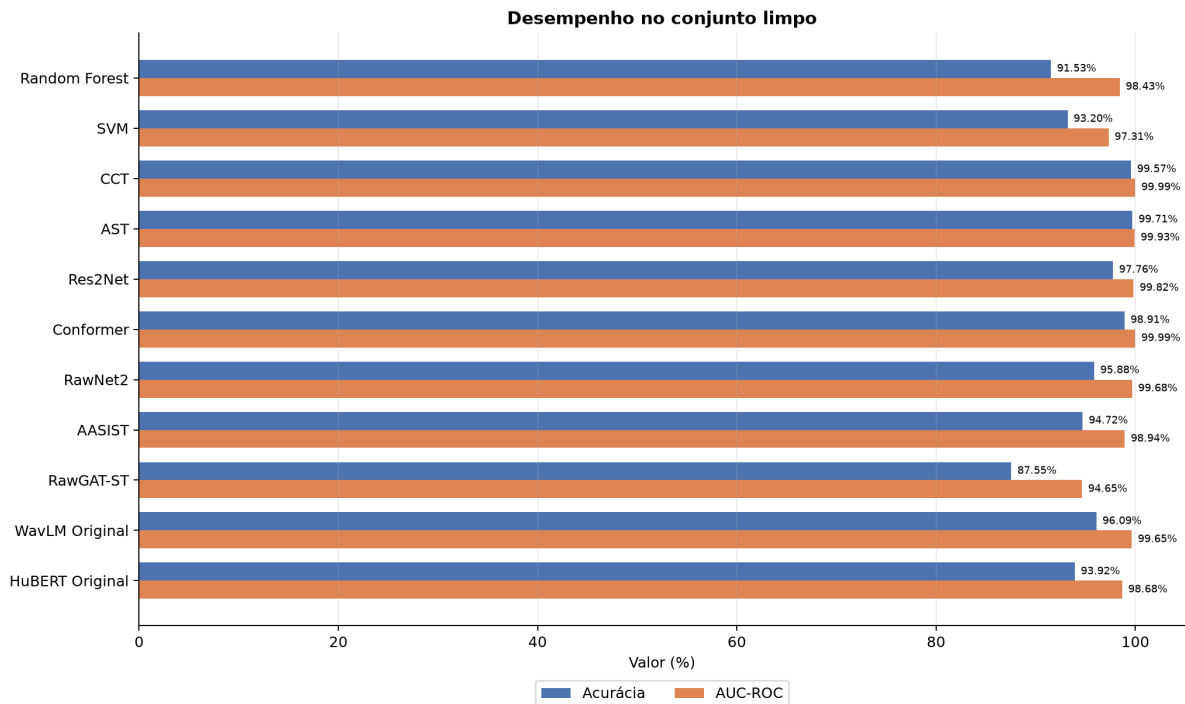
**Tabela 15:** Resumo descritivo das curvas de validação em execução única.

| Modelo          | Val. pico | Época  | Val. final | Queda | Nota                            |
|-----------------|-----------|--------|------------|-------|---------------------------------|
| Random Forest   | –         | CV+fit | –          | –     | não aplicável (modelo clássico) |
| SVM             | –         | CV+fit | –          | –     | não aplicável (modelo clássico) |
| CCT             | 99,45%    | 48     | 98,35%     | 1,10% | descritivo; 1 execução          |
| AST             | 99,38%    | 33     | 98,63%     | 0,76% | descritivo; 1 execução          |
| Res2Net         | 99,45%    | 17     | 98,01%     | 1,44% | descritivo; 1 execução          |
| Conformer       | 99,59%    | 69     | 99,24%     | 0,34% | descritivo; 1 execução          |
| RawNet2         | 98,01%    | 30     | 96,50%     | 1,51% | descritivo; 1 execução          |
| AASIST          | 96,63%    | 52     | 94,64%     | 1,99% | descritivo; 1 execução          |
| RawGAT-ST       | 89,97%    | 17     | 88,12%     | 1,85% | descritivo; 1 execução          |
| WavLM Original  | 97,46%    | 57     | 96,50%     | 0,96% | descritivo; 1 execução          |
| HuBERT Original | 95,88%    | 12     | 95,19%     | 0,69% | descritivo; 1 execução          |

## 6.4 Análise dos Resultados

### 6.4.1 Desempenho no Conjunto Limpo

O recorte consolidado da Tabela 12 e da Figura 2 mostra maior desempenho para AST e CCT, com 99,71% e 99,57% de acurácia no conjunto de teste limpo (EER de 0,14% e 0,43%). Em seguida aparecem Conformer (98,91%, EER de 0,29%) e Res2Net (97,76%, EER de 2,17%), indicando que redes neurais profundas extraem padrões discriminativos relevantes no cenário experimental controlado (QP1). Com 1.382 amostras de teste, uma acurácia de 99,71% corresponde a apenas 4 amostras mal classificadas, e a diferença entre AST e CCT equivale a duas amostras. AST, CCT e Conformer devem, portanto, ser tratados como **empate técnico**, e não como ordenamento estrito entre os três primeiros colocados. Os testes pareados confirmam essa leitura. O McNemar por *cluster* (frase,

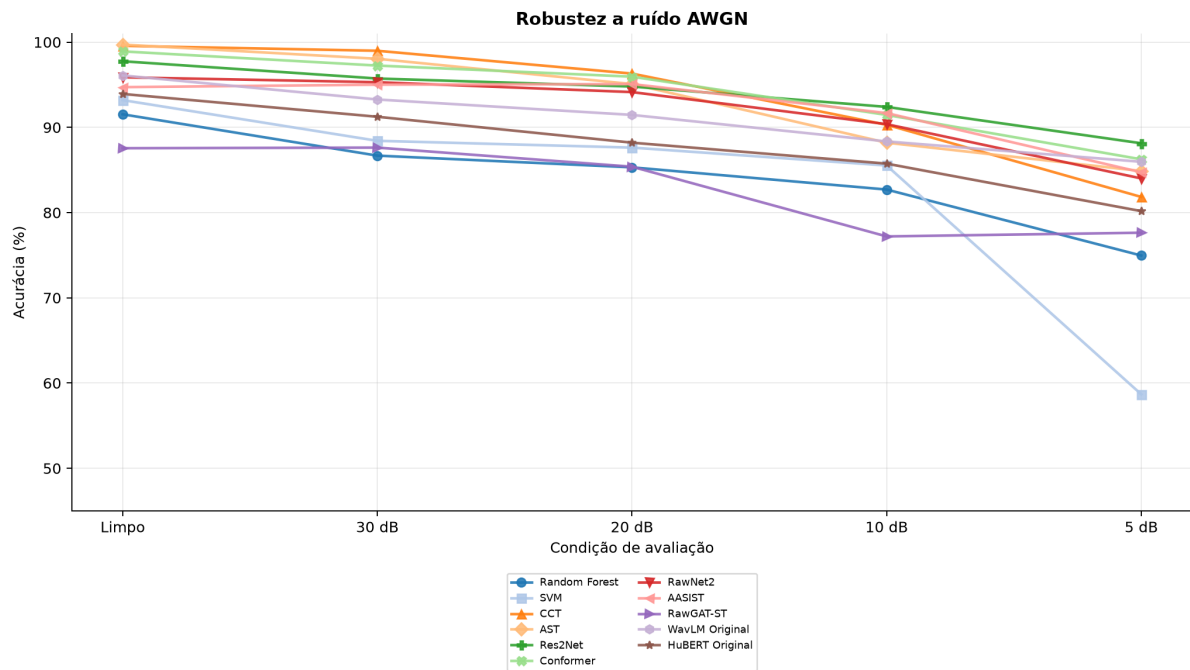


**Figura 2:** Acurácia e AUC-ROC no conjunto de teste limpo.

183 unidades) devolve, após correção de Holm sobre as 55 comparações,  $p = 1,00$  para  $AST \times CCT$ ,  $p = 0,37$  para  $CCT \times Conformer$  e  $p = 0,10$  para  $AST \times Conformer$  — nenhum significativo a 5%. O *bootstrap* pareado da diferença de EER é ainda mais claro: os três intervalos de confiança de 95% contêm o zero ( $\Delta EER = +0,29$  p.p., IC 95%  $[-0,30; +0,64]$  para  $AST \times CCT$ ), com  $p$  ajustado igual a 1,00 nos três pares. O par Res2Net–Conformer também não se separa pelo McNemar ( $p = 0,18$ ), embora o *bootstrap* pareado de EER o distinga.

Cabe registrar o piso de resolução dos testes: com 5000 reamostragens e 55 comparações, o menor  $p$  corrigido por Holm que o procedimento consegue expressar é 0,022. Os pares reportados adiante com  $p = 0,022$  estão *nesse piso* — a evidência é tão forte quanto o número de reamostragens permite afirmar, e não uma medida pontual naquele valor.

A escolha da unidade de agrupamento também importa, e o protocolo reporta as duas. Agrupando por **frase** (183 unidades), 40 dos 55 pares se separam pelo McNemar; agrupando por **locutor** (11 unidades) — a unidade que corresponde à alegação de disjunção de locutores e, por ser muito menor, bastante mais conservadora —, apenas 17 se separam. A conclusão sobre o topo do recorte, porém, é idêntica nas duas leituras: AST, CCT e Conformer permanecem mutuamente indistinguíveis. Sob a unidade locutor os três pares vão a  $p = 1,00$ ; sob a unidade frase, mais sensível por ter dezesseis vezes mais unidades, nenhum dos três alcança significância a 5% após a correção de Holm —  $p = 1,00$ , 0,37 e 0,10, os valores reportados acima. O empate técnico no topo não é, portanto, artefato da unidade de agregação escolhida.



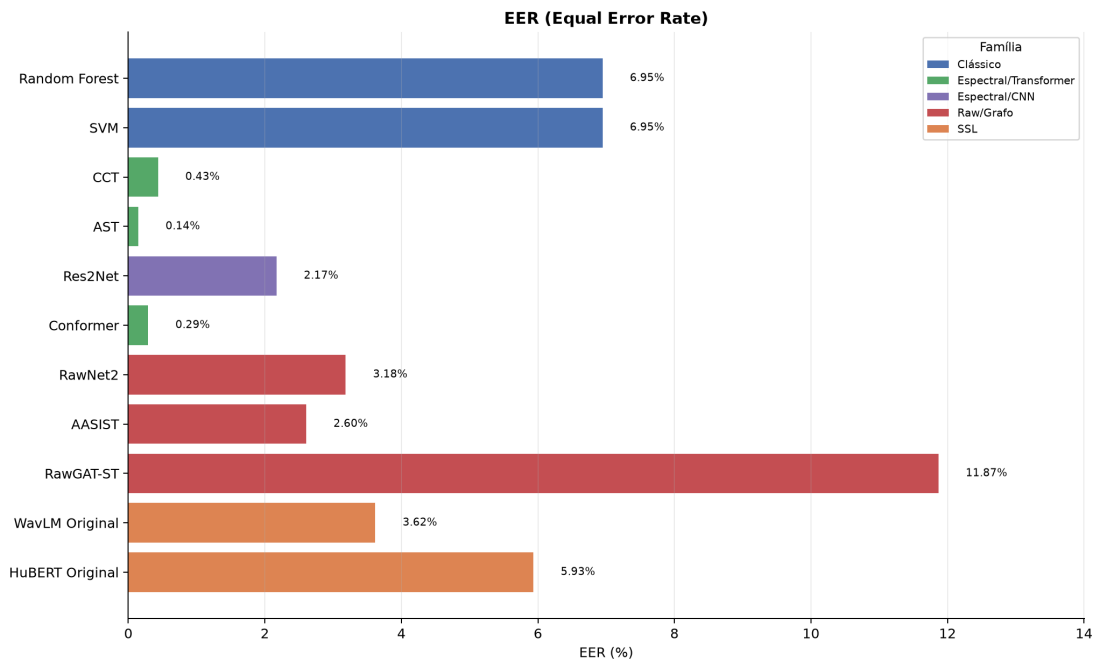
**Figura 3:** Robustez a AWGN na forma de onda (acurácia vs. SNR).

Esse ordenamento contraria em parte a hierarquia sugerida pela Tabela 2. Os *backbones* SSL WavLM e HuBERT (4<sup>a</sup> geração), avaliados com *backbone* congelado, soma ponderada das 13 camadas ocultas e *pooling* média $\oplus$ desvio sobre a janela canônica de 3 s (Seção 4.2), ficam na **metade inferior** do recorte — 7.<sup>o</sup> e 8.<sup>o</sup> lugares em EER (3,62% e 5,93%), abaixo de todas as arquiteturas espectrais e do RawNet2. A discussão desse contraste com a literatura recente está na Seção 7.

#### 6.4.2 A Previsão sobre Artefatos de Fase não se Confirma

O ordenamento também permite fechar a previsão formulada na Seção 2: se os artefatos de reconstrução de fase fossem determinantes, e se de fato fossem inacessíveis a uma representação de magnitude, as três arquiteturas de forma de onda deveriam liderar. Ocorre o contrário. RawNet2, AASIST e RawGAT-ST ocupam a 6.<sup>a</sup>, a 7.<sup>a</sup> e a 11.<sup>a</sup> posições em acurácia limpa, atrás de *todas* as redes espectrais, que operam sobre mapas Mel de magnitude e portanto não têm acesso à fase.

A previsão, portanto, **não se sustenta neste corpus**, e há duas leituras compatíveis com o observado. A primeira é que os clones XTTS-v2 deixam vestígio dominante na *envoltória* espectral e na dinâmica de amplitude — o segundo e o terceiro casos daquela lista —, e não na fase; a análise de importância do vetor tabular reforça essa leitura, ao encontrar estatísticas de amplitude e coeficientes LFCC como fatores dominantes. A segunda é que o acesso à fase não basta: dispor da forma de onda é condição necessária, não suficiente, para explorar inconsistências de fase, e nada garante que o treinamento



**Figura 4:** Taxa de Erro Igual (EER) por arquitetura.

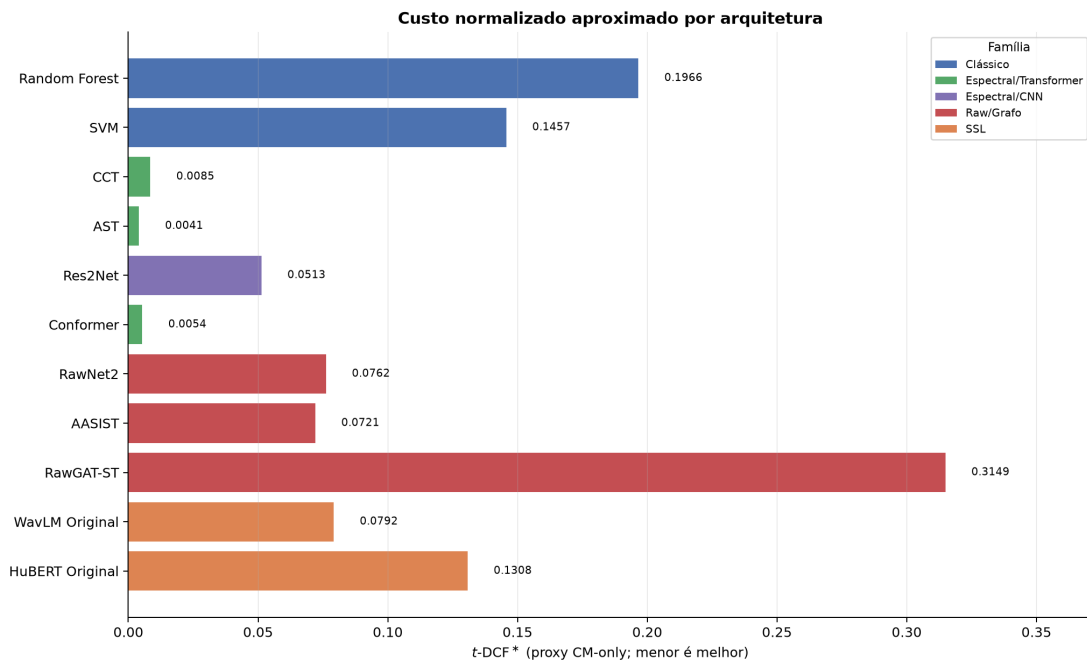
conduza a rede a aprendê-las quando um atalho mais simples está disponível na envoltória.

Distinguir as duas exigiria um ensaio de ablação — por exemplo, avaliar as redes de forma de onda sobre sinais com fase deliberadamente randomizada e verificar quanto do desempenho sobrevive. O recorte atual não o permite, e a questão fica em aberto (Seção 7.3).

O modelo mais fraco do conjunto é o **RawGAT-ST**, com EER de 11,87% e  $\min t\text{-DCF}^*$  de 0,3149 — o único que os testes pareados separam de *todos* os outros dez, com significância após correção de Holm. O AASIST, ao contrário do que a proximidade arquitetural sugeriria, registra EER de 2,60% e ocupa a 5.<sup>a</sup> posição, à frente de RawNet2, dos dois SSL e dos dois classificadores clássicos.

Os valores de AASIST e RawGAT-ST ficam acima dos reportados pelos artigos originais no ASVspoof 2019 LA — EER de 0,83% para o AASIST [32], 1,06% para o RawGAT-ST [41] e  $\approx 1,1\%$  para o RawNet2 [31] —, mas a distância tem magnitudes muito diferentes nos dois casos: o AASIST fica a 1,8 p.p. do valor publicado, enquanto o RawGAT-ST fica a 10,8 p.p. Para o AASIST, a diferença é compatível com a mudança de *corpus* e de escopo; para o RawGAT-ST, ela indica um problema de treinamento específico deste protocolo, discutido na análise de estabilidade adiante. As implementações de ambos são variantes alinhadas aos artigos, não réplicas exatas, e o *corpus* deste trabalho — em português, com um único gerador — é distinto e mais restrito que o ASVspoof, que combina dezenas de sistemas de ataque. A hierarquia observada é, portanto, específica deste protocolo e não constitui refutação dos resultados publicados para essas arquiteturas.

A decomposição dos erros por tipo (Figura 9) revela padrões operacionalmente distintos, que a acurácia agregada esconde. O AST erra 4 amostras no total, **todas falsos**



**Figura 5:**  $t$ -DCF\* por arquitetura.

**positivos:** no limiar fixo de 0,5 ele não aceitou nenhum *spoof* — o comportamento mais favorável possível no cenário de biometria em *tandem*. O CCT distribui seus 6 erros simetricamente (3 contra 3).

AASIST e RawGAT-ST concentram os erros em falsos positivos (AASIST: 66 amostras *bonafide* rejeitadas contra 7 *spoofs* aceitos; RawGAT-ST: 146 contra 26), o que penaliza a usabilidade pela rejeição indevida de usuários legítimos, mas preserva a segurança. O padrão oposto — e mais crítico — aparece em RawNet2 (54 *spoofs* aceitos contra 3 rejeições indevidas), SVM (81 contra 13), *Random Forest* (83 contra 34) e, em menor grau, Conformer (13 contra 2): aceitar um ataque custa mais do que rejeitar um usuário genuíno ( $C_{fa} = 10 \gg C_{miss} = 1$ ; Seção 5.5), e é esse desequilíbrio que o min  $t$ -DCF\* penaliza adiante.

### 6.4.3 Importância de Características no Vetor Tabular

A Figura 10 apresenta a importância **por permutação** das características do vetor tabular de 183 descritores (Seção 5.3), medida sobre a partição de teste com 30 repetições por característica (`scripts/reporting/export_rf_feature_importance.py`). A escolha do método é deliberada: a redução média de impureza (MDI, ou “importância de Gini”) é enviesada a favor de variáveis contínuas e de alta cardinalidade, e por isso não é usada na análise confirmatória — a importância por permutação mede a queda real de acurácia quando a característica é destruída, que é a grandeza de interesse.

Agregando por família, o resultado contraria a expectativa de domínio cepstral: as **estatísticas temporais** concentram 44,7% da importância total, seguidas de perto pelos coeficientes **LFCC** (41,6%), com MFCC (8,7%) e RASTA-PLP (5,0%) bem atrás. Indi-

vidualmente, o descritor mais informativo é o **desvio-padrão** da forma de onda (21,5% sozinho), seguido da média (10,9%) e do LFCC20 médio (5,9%). Entre as 15 características mais importantes, 5 são temporais, 7 são LFCC e 3 são MFCC.

Os percentuais acima normalizam sobre a soma das importâncias **truncadas em zero**. O truncamento não é cosmético e precisa ser declarado: 59 das 183 características têm importância por permutação negativa — o que significa que embaralhá-las *melhorou* a acurácia, um artefato de ruído amostral sem interpretação causal. Sob a normalização bruta, que soma também esses valores, o quadro se desloca: temporais passam a 51,0%, LFCC a 36,7%, MFCC a 6,8% e RASTA-PLP a 5,5%. A leitura qualitativa — domínio das estatísticas de amplitude, LFCC bem à frente do MFCC — é a mesma nas duas; a repartição exata entre os blocos temporal e cepstral, não.

Duas leituras seguem daí. Primeiro, a predominância de estatísticas de amplitude sugere que parte do poder discriminativo do vetor tabular vem de diferenças de *envelope e dinâmica* entre fala natural e sintetizada — não de estrutura espectral fina. Segundo, o peso do LFCC ante o MFCC é consistente com a literatura de *anti-spoofing*, que reporta o espaçamento linear em frequência como mais sensível a artefatos de síntese do que a escala Mel, desenhada para percepção. Esse contraste é coerente com a extensão do vetor de 63 para 183 descritores; note-se, porém, que ele *não* a valida de forma independente, já que a extensão foi decidida com conhecimento do comportamento no teste e a importância aqui é medida na mesma partição (Seção 5.3).

Vale notar a tensão com a leitura clássica da literatura de *anti-spoofing* de primeira geração [27], que atribui à envoltória cepstral o grosso da informação discriminativa. Aqui, somando LFCC e MFCC, o bloco cepstral responde por 50,3% sob a normalização truncada (43,5% sob a bruta) — em torno de metade da importância, mas dividindo espaço com estatísticas de amplitude que aquela literatura trata como secundárias. Uma explicação plausível é o pareamento: com texto e locutor constantes entre as classes, diferenças de conteúdo que normalmente dominariam a variância cepstral desaparecem, e o que resta são as assinaturas do sintetizador — entre elas, a compressão de faixa dinâmica característica de *vocoders* neurais.

Independentemente da família, o *Random Forest* é um dos modelos menos robustos sob ruído (Tabela 14), o que reforça que a fragilidade dos clássicos decorre da natureza *agregada* do vetor — médias e desvios globais por amostra descartam a estrutura tempo-frequência fina que as redes espectrais exploram — e não da ausência de características apropriadas.

#### 6.4.4 Robustez a Ruído: o Achado Central

A Tabela 14 e a Figura 3 mostram degradação suave e majoritariamente monotônica sob o protocolo *waveform-AWGN* (ruído aplicado à forma de onda antes de qualquer *frontend*, com exemplos ruidosos compatíveis também no treino). Duas exceções merecem

registro, e ambas decorrem do casamento entre treino e avaliação: AASIST e RawGAT-ST *melhoram* ligeiramente a 30 dB em relação ao conjunto limpo ( $94,72 \rightarrow 95,01\%$  e  $87,55 \rightarrow 87,63\%$ ), porque metade dos exemplos de ajuste é ruidosa e o limpo passa a ser, para esses dois modelos, a condição menos representada. O RawGAT-ST ainda inverte entre 10 e 5 dB ( $77,21$  contra  $77,64\%$ ), o que é coerente com a sua instabilidade de treino e com o intervalo amostral de  $\pm 2,2$  pontos nesse nível. O ordenamento a 10 dB **difere substancialmente** do observado no conjunto limpo: lidera o Res2Net ( $92,40\%$ ), seguido de AASIST ( $91,68\%$ ), Conformer ( $91,46\%$ ), RawNet2 ( $90,38\%$ ) e CCT ( $90,30\%$ ); depois vêm WavLM Original ( $88,35\%$ ), AST ( $88,21\%$ ), HuBERT Original ( $85,75\%$ ), SVM ( $85,53\%$ ) e *Random Forest* ( $82,71\%$ ); o RawGAT-ST fecha o conjunto com  $77,21\%$  — nenhum deles próximo do acaso ( $50\%$ ).

A reorganização é o achado: os dois melhores modelos no limpo (AST e CCT) caem para 7.<sup>a</sup> e 5.<sup>a</sup> posições sob ruído, enquanto o AASIST sobe da 7.<sup>a</sup> para a 2.<sup>a</sup>. Acurácia em condição limpa, isoladamente, não prediz robustez.

**Quanto dessa ordem é estatisticamente sustentável.** O aparato pareado aplicado ao conjunto limpo foi estendido aos níveis ruidosos — McNemar por *cluster* e *bootstrap* pareado de EER, ambos com 5000 reamostragens e correção de Holm sobre as 55 comparações — e o veredito exige cautela com a leitura do *ranking*. A 10 dB separam-se 31 pares pelo McNemar e 32 pelo *bootstrap* sob a unidade frase; sob a unidade locutor, apenas 7 por ambos.

**Nenhum dos dez pares entre os cinco primeiros atinge significância por nenhum dos dois testes.** Pelo McNemar, o menor  $p$  ajustado do grupo é 0,50; pelo *bootstrap*, 0,055 (Conformer  $\times$  RawNet2) — próximo do limiar, mas ainda acima dele. Res2Net, AASIST, Conformer, RawNet2 e CCT formam, portanto, um **bloco indistinguível** a 10 dB, assim como AST, CCT e Conformer formam um no conjunto limpo.

Isso não desfaz o achado; delimita-o. O que os dados sustentam é o *deslocamento entre blocos*, e não a ordem dentro deles: AST e CCT deixam o bloco de topo do conjunto limpo, passando a integrá-lo (CCT) ou a ficar abaixo dele (AST) sob ruído. No extremo oposto, o RawGAT-ST separa-se de 10 dos 10 demais modelos pelo McNemar e de 9 pelo *bootstrap* de EER a 10 dB. Afirmações do tipo “o Res2Net lidera a robustez” devem ser lidas como ordenação amostral, não como diferença demonstrada; a de que “o RawGAT-ST é o menos robusto” está estabelecida.

A 5 dB o poder de resolução cai, como esperado da degradação mais severa: separam-se 33 pares pelo McNemar e 36 pelo *bootstrap* sob a unidade frase (14 e 10 por locutor), e o próprio RawGAT-ST já não se distingue de todos — apenas de 7 dos 10, por ambos os testes. Sob o SNR não visto, portanto, nem mesmo o pior modelo do recorte é separável do conjunto com a confiança que 10 dB permite.

Dois achados emergem dessa leitura (QP2). Primeiro, o casamento entre o ruído de

treino e o de teste importa mais do que a família arquitetural: com exemplos ruidosos compatíveis no treino, até os classificadores clássicos — limitados a um vetor de 183 descritores agregados — degradam suavemente (SVM: 93,20%  $\rightarrow$  85,53%; *Random Forest*: 91,53%  $\rightarrow$  82,71%), sem o colapso que se observaria caso o ruído aparecesse apenas na avaliação.

Segundo, a fragilidade relativa *não* se distribui por família arquitetural, como se poderia supor. As duas redes de grafos sobre áudio bruto se comportam de maneira oposta: o RawGAT-ST perde 10,3 pontos percentuais entre o conjunto limpo e 10 dB (87,55%  $\rightarrow$  77,21%), a menor robustez absoluta do recorte, enquanto o AASIST — mesma família, mesmo domínio de entrada, mesmo *front-end* SincNet — degrada apenas 3,0 pontos (94,72%  $\rightarrow$  91,68%) e termina como o 2.º mais robusto. O RawGAT-ST é também o único dos dois que apresenta sobreajuste e oscilação de validação (Seção 6.4, “Estabilidade de Treinamento”), o que torna o sobreajuste a explicação mais plausível para o contraste — com a ressalva, decisiva, de que se trata de **uma única comparação**.

Esse resultado **rejeita as duas hipóteses mais plausíveis** formuladas na QP2 (Seção 1.3). A *representação de entrada* não explica a dissociação: AASIST e RawGAT-ST consomem a mesma forma de onda, pelo mesmo *front-end* SincConv, e separam-se por 14,5 pontos percentuais a 10 dB. A *capacidade do modelo* tampouco: os dois são os menores do recorte (396.312 e 650.204 parâmetros), enquanto o maior modelo treinado do zero — o Conformer, com 28,6 milhões — fica entre eles em robustez.

Seria tentador promover o sobreajuste a explicação geral, e a versão preliminar desta análise o fez. **Os próprios dados não sustentam essa generalização.** O diagnóstico automático classifica *dois* modelos como `unstable_oscillation`, e a Tabela 16 mostra que eles ocupam extremos opostos da robustez:

**Tabela 16:** Os dois modelos diagnosticados como instáveis e sua posição na robustez a 10 dB. A instabilidade de validação, isoladamente, não ordena o recorte.

| Modelo    | Diagnóstico                       | Acur. a 10 dB | Posição          |
|-----------|-----------------------------------|---------------|------------------|
| Conformer | <code>unstable_oscillation</code> | 91,46%        | 3. <sup>a</sup>  |
| RawGAT-ST | <code>unstable_oscillation</code> | 77,21%        | 11. <sup>a</sup> |

O Conformer é instável pelo mesmo critério e, ainda assim, é o 3.º mais robusto do conjunto, à frente de seis modelos classificados como estáveis. A variável “estabilidade de validação” tem, neste recorte, duas observações positivas — e uma delas é contraexemplo direto.

A conclusão honesta da QP2 é, portanto, **negativa em três frentes**: nem a representação de entrada, nem a capacidade do modelo, nem o diagnóstico automático de estabilidade ordenam a robustez a ruído. O contraste AASIST  $\times$  RawGAT-ST permanece a observação mais sugestiva do recorte, e o sobreajuste é a leitura mais econômica para

ele, mas com  $n = 2$  ela é *hipótese para trabalho futuro*, não achado estabelecido. Separá-la de alternativas — inicialização infeliz, interação entre a construção dinâmica dos grafos e o regime de ruído, sensibilidade da cabeça AM-Softmax — exigiria as múltiplas sementes que a Seção 7.2 declara ausentes. O que o recorte estabelece com segurança é o resultado negativo: as explicações arquiteturais óbvias falham, e o fator discriminante não foi identificado.

#### 6.4.5 O SNR Não Visto de 5 dB

Os níveis de 30, 20 e 10 dB são **casados**: as mesmas SNRs entram no *augmentation* de treino. O nível de 5 dB é o único **não visto**, e por isso é ele — e não os demais — que mede generalização a uma degradação fora da distribuição de treino.

O resultado mais informativo do recorte está aí. O SVM, que a 10 dB ainda preserva 85,53% de acurácia, **colapsa para** 58,68% a 5 dB, com *recall* de apenas 18,38% — ou seja, deixa de detectar quatro em cada cinco *spoofs*. A AUC-ROC, porém, permanece em 0,884, e o EER em 20,41%.

A dissociação entre as duas leituras é o ponto: uma acurácia de 58,68% com AUC de 0,884 significa que o modelo *continua ordenando* as amostras razoavelmente bem, mas que o limiar fixo em 0,5 deixou de corresponder ao ponto de separação das distribuições sob aquela degradação. O que falha é a *calibração*, não a capacidade discriminativa.

A conclusão prática que essa leitura sugere — bastaria recalibrar o limiar — **não se confirma**, e o artefato do próprio *benchmark* é quem a desmente. O campo `accuracy_at_calibrated_threshold`, que aplica o limiar estimado fora do teste, leva o SVM a 57,31% a 5 dB: *abaixo* dos 58,68% do limiar fixo. Existe, sim, um teto de recuperação — o limiar-oráculo de EER alcançaria 79,59% —, mas ele é obtido ajustando o limiar *no próprio conjunto de teste* e não corresponde a nenhum procedimento disponível em operação. O resultado correto é, portanto, mais forte e menos animador do que a leitura ingênua: sob degradação fora da distribuição de treino, a informação para recalibrar existe no *score*, e nenhuma regra de calibração ajustada em distribuição conhecida consegue extraí-la.

Os demais modelos degradam de forma mais graciosa a 5 dB — Res2Net lidera com 88,13%, seguido de Conformer (86,25%) e WavLM Original (85,96%). A dissociação acurácia/AUC não é exclusiva do SVM, mas nenhum outro modelo a exhibe na mesma magnitude: o CCT cai a 81,84% mantendo AUC de 0,955 (teto de 87,92% sob limiar-oráculo) e o AST a 84,88% com AUC de 0,952 (teto de 91,39%) — perdas de 6,1 e 6,5 pontos por desalinhamento de limiar, contra os 20,9 pontos do SVM. O *Random Forest*, também tabular, cai a 74,96% sem o colapso de *recall*, o que sugere que o efeito depende da geometria da fronteira de decisão e não apenas do vetor de características.

#### 6.4.6 Ablação do AST: Quanto Vale o Pré-Treinamento

O AST é o único modelo do recorte que acumula duas vantagens não controladas sobre as demais redes espectrais: parte de pesos AudioSet e recebe a grade  $300 \times 128$  (Seção 5.3). A primeira delas foi isolada experimentalmente: treinou-se o AST **sob condições idênticas às publicadas** — mesma topologia ViT-Base, mesma grade, mesmas 100 épocas, mesma semente, mesmo protocolo de avaliação — alterando *um único parâmetro*, a transferência de pesos. A Tabela 17 traz o resultado.

**Tabela 17:** Ablação do pré-treinamento no AST. Única variável alterada: a transferência de pesos AudioSet. Execução de 100 épocas (12,15 h), classificada como estável pelo diagnóstico do *pipeline*.

| Métrica              | Com AudioSet | Sem AudioSet | Diferença   |
|----------------------|--------------|--------------|-------------|
| Acurácia (limpo)     | 99,71%       | 94,93%       | −4,78 p.p.  |
| EER (limpo)          | 0,14%        | 4,34%        | 30×         |
| AUC-ROC (limpo)      | 0,9993       | 0,9924       | −0,0069     |
| min <i>t</i> -DCF*   | 0,0041       | 0,1166       | 28×         |
| Acurácia @ 30 dB     | 98,05%       | 92,84%       | −5,21 p.p.  |
| Acurácia @ 20 dB     | 95,08%       | 90,67%       | −4,41 p.p.  |
| Acurácia @ 10 dB     | 88,21%       | 84,66%       | −3,55 p.p.  |
| Acurácia @ 5 dB      | 84,88%       | 80,10%       | −4,78 p.p.  |
| Pior locutor (limpo) | 98,41%       | 82,26%       | −16,15 p.p. |

Três conclusões seguem, e a primeira revisa a leitura da QP1.

**A primeira colocação do AST não sobrevive à ablação.** Sem os pesos AudioSet, o AST cai para 94,93% — *abaixo* de CCT (99,57%), Conformer (98,91%) e Res2Net (97,76%), todos treinados do zero e sobre a grade menor. A topologia ViT-Base, portanto, **não é superior** às alternativas neste *corpus*; o que a colocava em primeiro lugar era o pré-treinamento em larga escala. O empate técnico observado entre AST, CCT e Conformer (Seção 6.4) ganha com isso uma leitura mais forte: os dois modelos treinados do zero alcançam, sem qualquer transferência, o patamar que o AST só atinge com 2 milhões de cliques de AudioSet.

**O ganho é uniforme, não específico de robustez.** A degradação é semelhante no conjunto limpo (−4,78 p.p.) e nos quatro níveis de ruído (−3,55 a −5,21 p.p.), o que afasta a hipótese de que o pré-treinamento atuasse sobretudo na generalização a degradações. Ele desloca o desempenho inteiro. O efeito sobre as métricas de ordenação é bem maior que sobre a acurácia — o EER multiplica por 30 e o min *t*-DCF\* por 28 —, o que indica que a transferência melhora principalmente a *separação das distribuições de pontuação*, e não apenas o acerto no limiar fixo.

**O colapso histórico fica atribuído à resolução.** Uma configuração anterior deste *pipeline* combinava grade  $100 \times 80$  e treino do zero, e degradava até o acaso (Seção 5.3);

as duas condições foram corrigidas no mesmo momento, deixando a causa sem atribuição. Este experimento a resolve: treinar do zero na grade  $300 \times 128$  produz 94,93%, não o acaso. A causa do colapso era, portanto, a **escassez de *tokens*** da grade menor — 63 contra os 1212 do artigo original — e não a ausência de pesos transferidos.

**O ganho concentra-se nos locutores difíceis.** A leitura por locutor (Seção 6.4.8) revela o efeito mais expressivo de todos: a acurácia média cai 4,78 pontos com a remoção dos pesos, mas a do *pior* dos 11 falantes do teste cai 16,15 **pontos** ( $98,41\% \rightarrow 82,26\%$ ) — mais de três vezes a queda agregada. O pré-treinamento em larga escala, portanto, não eleva o desempenho de modo uniforme: ele age sobretudo onde o modelo é frágil, comprimindo a dispersão entre falantes. É uma justificativa funcional para a transferência que a acurácia média sozinha esconde, e que reforça o argumento da Seção 6.4.8 — métricas agregadas ocultam risco de exclusão sistemática de indivíduos.

Duas ressalvas delimitam o alcance. O experimento isola o pré-treinamento e *não* a resolução: para atribuir a parcela devida à grade seria necessário um segundo braço ( $100 \times 80$  com pesos transferidos), não executado. E trata-se de execução única, com o mesmo sobreajuste severo observado nos demais modelos treinados do zero — a acurácia de treino atingiu 1,000 na última época, e o *checkpoint* avaliado é o de menor perda de validação, obtido na época 25.

#### 6.4.7 Quanto Custa a Não Cegueira dos Classificadores Clássicos

A Seção 5.3 declara que o vetor tabular de 183 descritores foi definido depois de observar o comportamento de SVM e *Random Forest* sob ruído, o que retira desses dois modelos — e apenas deles — a garantia de cegueira que o congelamento da partição confere aos nove neurais. Essa perda pode ser *medida*, e não apenas declarada.

Construiu-se, para isso, uma partição de avaliação **inteiramente nova** a partir do *corpus* pareado completo, sujeita a três condições simultâneas: (i) as amostras não pertencem a nenhuma partição do conjunto de 15.000; (ii) os locutores não aparecem no treino; e (iii) os textos tampouco. Restaram 4916 amostras — 2458 pares completos, perfeitamente balanceados, de 22 locutores e 397 textos, ou 3,6 vezes o tamanho do conjunto de teste original. Os modelos **não foram retreinados**: são os mesmos artefatos promovidos, avaliados sobre dados que não participaram de decisão alguma. A Tabela 18 confronta o resultado com os valores publicados.

Antes de interpretá-lo, um controle: os dois modelos foram reavaliados neste mesmo ambiente sobre o conjunto de teste *original* e reproduziram exatamente as acurácias publicadas (93,20% e 91,53%). A diferença observada adiante é, portanto, do *conjunto de avaliação*, e não de divergência de versão de biblioteca ou de recomputação das características.

**Tabela 18:** Avaliação cega de SVM e *Random Forest* sobre partição fresca ( $n = 4916$ ), contra os valores publicados no conjunto de teste de 1382 amostras. Nenhum retreinamento: apenas o conjunto de avaliação muda.

| Modelo / métrica     | Teste original | Partição fresca | Diferença   |
|----------------------|----------------|-----------------|-------------|
| <i>SVM</i>           |                |                 |             |
| Acurácia             | 93,20%         | 90,58%          | −2,62 p.p.  |
| EER                  | 6,95%          | 9,80%           | +2,85 p.p.  |
| AUC-ROC              | 0,9731         | 0,9562          | −0,0169     |
| Pior locutor         | 54,03%         | 53,39%          | −0,64 p.p.  |
| <i>Random Forest</i> |                |                 |             |
| Acurácia             | 91,53%         | 88,04%          | −3,49 p.p.  |
| EER                  | 6,95%          | 12,39%          | +5,44 p.p.  |
| AUC-ROC              | 0,9843         | 0,9579          | −0,0264     |
| Pior locutor         | 73,39%         | 56,12%          | −17,27 p.p. |

Três leituras seguem daí. Primeira, o **viés otimista existe e é moderado**: 2,6 e 3,5 pontos percentuais de acurácia. Os classificadores clássicos não deixam de funcionar fora do conjunto que ajudou a desenhá-los, mas os números publicados para eles são um limite superior.

Segunda, a degradação é *maior nas métricas de ordenação* do que na acurácia — o EER do *Random Forest* quase dobra, de 6,95% para 12,39%. O empate em EER entre SVM e *Random Forest* observado no conjunto original (Seção 6.4) **não sobrevive** à avaliação cega: o SVM passa a ser claramente melhor na ordenação.

Terceira, a dispersão entre locutores é ainda pior do que o conjunto de 11 falantes sugeria. O pior locutor do *Random Forest* cai de 73,39% para 56,12%, e o do SVM permanece próximo do acaso. Com 22 locutores em vez de 11, há simplesmente mais oportunidade de encontrar o caso ruim — o que reforça a ressalva da Seção 6.4.8.

Uma qualificação metodológica delimita a leitura: a partição fresca tem locutores e textos *diferentes* dos do teste original, de modo que a diferença observada mistura o efeito da não cegueira com a variação amostral entre dois conjuntos de falantes. O valor medido deve, portanto, ser lido como **limite superior** do otimismo atribuível à escolha do *front-end*, e não como sua estimativa pontual. Ainda assim, ele converte a limitação (viii) da Seção 7.2 de ressalva qualitativa em quantidade conhecida — e mostra que ela não invalida as conclusões sobre os classificadores clássicos, apenas as desloca.

#### 6.4.8 Dispersão entre Locutores Não Vistos

A acurácia agregada esconde uma dispersão que a última coluna da Tabela 14 expõe. Como o conjunto de teste é disjunto em locutor, cada um dos seus 11 falantes é uma amostra independente da pergunta que o protocolo realmente faz: *o detector funciona em uma voz que nunca viu?* O agregado responde “sim” com folga; o pior locutor, nem

sempre.

O caso extremo é o **SVM**: 93,20% de acurácia média no conjunto limpo e 54,03% no pior dos 11 locutores — quatro pontos acima do acaso. Um detector com esse perfil não falha uniformemente; ele falha *por pessoa*, e a pessoa afetada fica sem serviço. RawGAT-ST (65,08%), HuBERT Original (70,97%), *Random Forest* (73,39%) e RawNet2 (74,19%) exibem o mesmo padrão em grau menor: o RawNet2, com 95,88% de média, perde mais de 21 pontos no seu pior falante.

No extremo oposto, AST (98,41%) e CCT (97,58%) mantêm no pior locutor um desempenho próximo da própria média — diferença abaixo de 2 pontos —, e são os únicos do recorte para os quais a acurácia agregada é um resumo fiel. Conformer (90,32%), AASIST (88,10%) e Res2Net (87,30%) ficam em posição intermediária.

Duas consequências. A primeira é metodológica: a variância entre locutores é grande o bastante para que o veredito conservador dos testes pareados agrupados por **locutor** — 17 pares separados, contra 40 por frase — seja o que deve valer para qualquer afirmação sobre generalização a vozes não vistas. A segunda é operacional e qualifica a recomendação de implantação da Seção 6.4: o SVM é atrativo por latência e custo, mas o seu perfil de erro concentra-se em falantes específicos, o que em biometria significa exclusão sistemática de indivíduos e não degradação difusa. Nenhuma métrica agregada deste trabalho capta esse risco; só esta coluna capta.

#### 6.4.9 Ponto de Operação: AUC, EER e $t$ -DCF\*

Duas leituras complementares emergem das métricas independentes de limiar. Primeiro, a comparação entre AASIST e RawGAT-ST é consistente em todas elas: o AASIST tem EER menor (2,60% contra 11,87%), AUC-ROC maior (0,989 contra 0,947) e  $t$ -DCF\* menor (0,0721 contra 0,3149), e os testes pareados confirmam a diferença com significância após correção de Holm. As distribuições de pontuação têm formas bastante distintas (Figura 11): a cabeça AM-Softmax do AASIST, por maximizar margem angular, satura as pontuações nos extremos do intervalo — 86,5% dos *scores* ficam abaixo de 0,05 ou acima de 0,95, contra apenas 24,5% no RawGAT-ST, cujas pontuações se espalham pelo interior do intervalo.

Essa saturação produz separação nítida no ponto de equilíbrio, mas concentra o erro residual do AASIST em uma cauda de 2,3% de amostras *bonafide* com pontuação acima de 0,95 — classificadas como *spoof* com alta confiança. É o mesmo desequilíbrio visto na matriz de confusão (66 falsos positivos contra 7 falsos negativos), e ele não é removível por recalibração de limiar: a curva DET (Figura 12) mostra que, para operar com  $P_{fa} \leq 1\%$ , o AASIST precisa aceitar  $P_{miss} = 42,3\%$ , contra 1,9% quando se admite  $P_{fa} \leq 5\%$ . A transição abrupta entre esses dois regimes é a assinatura da cauda saturada.

Segundo, o  $t$ -DCF\* acompanha o ordenamento do EER, mas *comprime* as distâncias em vez de ampliá-las. Tomando o AST como referência, o RawGAT-ST está 82 vezes acima

em EER e apenas 76 vezes em  $t$ -DCF\*; a compressão se repete em todos os dez pares, variando de 1% a 35%. O efeito decorre da normalização por  $\min(C_1, C_2)$  (Equação 12): com os parâmetros do ASV fixados, o denominador impõe uma escala comum que limita o quanto a razão entre dois detectores pode crescer. A parametrização adotada (ASVspoof 2019 LA, com  $C_{fa} = 10 \gg C_{miss} = 1$ ) pondera fortemente a região de baixa taxa de falso-alarme, mas esse peso atua sobre o *formato* da curva de erro de cada modelo, não sobre a distância entre modelos.

Cabe registrar, porém, um resultado negativo relevante: neste *corpus* o  $t$ -DCF\* **não reordena o recorte**. Comparadas as posições nas duas métricas, a única troca ocorre entre SVM e *Random Forest*, que têm EER numericamente idêntico (6,95%) e são desempatados pelo custo. Os demais nove modelos ocupam a mesma posição em ambas. A assimetria de custo  $C_{fa} = 10 \gg C_{miss} = 1$ , portanto, não altera a escolha *entre* detectores aqui — o que é informativo em si, e contraria a expectativa de que uma métrica ponderada por custo produziria um ordenamento distinto.

O que a ponderação revela não é uma troca de posições, e sim quanto cada modelo é penalizado pela forma da sua curva de erro. A razão  $t$ -DCF\*/EER varia de 1,86 (Conformer) a 2,86 (AST), e o AASIST fica entre os mais penalizados (2,77) — coerente com a sua cauda de falsos positivos e com o salto de  $P_{miss}$  medido na curva DET. A leitura correta é, portanto, que a assimetria de custo importa para escolher o *ponto de operação* de um detector já selecionado, e não para selecioná-lo. É também por isso que o protocolo reporta as três métricas em conjunto: elas concordam no ordenamento, mas descrevem regimes de operação diferentes.

#### 6.4.10 Vazamento de Domínio e Separabilidade

Os valores de AUC-ROC muito próximos de 1,000 (CCT e Conformer, 0,9999) e os picos de acurácia de validação acima de 99% (Conformer, 99,59% na época 69 — Tabela 15) exigem interpretação cuidadosa. Separabilidade quase perfeita é, em princípio, compatível tanto com um detector genuinamente eficaz quanto com um classificador que explora correlações espúrias entre fonte, canal e rótulo.

No caso deste protocolo, as três explicações espúrias mais comuns estão descartadas por construção (Seção 5.4.3): o pareamento mantém texto e locutor constantes entre as classes, a disjunção dupla foi auditada com sobreposição nula em quatro dimensões, e o oráculo majoritário fica no nível do acaso para fonte, locutor e texto. Resta uma explicação legítima — a de que os clones XTTS-v2 carregam artefatos de *vocoder* de fato detectáveis — e uma alternativa que este protocolo *não* consegue afastar: a de que os modelos aprenderam a assinatura *daquele* sintetizador específico, e não a distinguir fala natural de fala sintética em geral.

Essa segunda hipótese é a limitação real, e ela é de **generalização entre geradores**, não de vazamento de partição. A separabilidade alta em um *corpus* de gerador único é

o resultado esperado mesmo para um detector que não generalizaria; distinguir os dois casos exige avaliação contra sistemas de síntese não vistos no treino (Seção 7.3). Até lá, as métricas devem ser lidas como desempenho contra o XTTS-v2, e não como capacidade geral de detecção.

#### 6.4.11 Eficiência Computacional e Trade-offs

A Tabela 13 e as Figuras 6 e 7 mostram que desempenho e custo computacional não são monotonicamente relacionados (QP3). Uma ressalva metodológica antecede a leitura: as latências da Tabela 13 foram medidas em três *runtimes* distintos (Seção 6.1), de modo que os seus valores absolutos **não formam um *ranking* único**. Os 15,77 ms do WavLM Original refletem PyTorch em GPU e não são comparáveis aos 50,46 ms do Conformer em Keras, nem aos 0,90 ms do SVM em scikit-learn sobre CPU.

**Medição em dispositivo único.** Para remover essa confusão, os 11 modelos foram remediados em um **mesmo dispositivo** — CPU, o único denominador comum, já que SVM e *Random Forest* não têm caminho de GPU —, sob protocolo idêntico ao original: lote unitário, duas execuções de aquecimento, 30 medidas, mediana, e apenas a passagem direta do modelo, sem *front-end*. A Tabela 19 traz o resultado, e ele **reordena o recorte**.

**Tabela 19:** Latência de inferência dos 11 modelos no *mesmo* dispositivo (CPU), contra os valores publicados em três *runtimes* distintos. A coluna *Fator* é a razão entre as duas medidas.

| Modelo               | Publicado | Ambiente    | CPU única | Fator |
|----------------------|-----------|-------------|-----------|-------|
| SVM                  | 0,90      | CPU/sklearn | 1,03      | 1,1×  |
| Res2Net              | 46,41     | GPU/Keras   | 22,05     | 0,5×  |
| CCT                  | 31,83     | GPU/Keras   | 24,05     | 0,8×  |
| Conformer            | 50,46     | GPU/Keras   | 51,96     | 1,0×  |
| <i>Random Forest</i> | 56,43     | CPU/sklearn | 64,70     | 1,1×  |
| AASIST               | 53,38     | GPU/Keras   | 94,82     | 1,8×  |
| RawNet2              | 90,08     | GPU/Keras   | 119,26    | 1,3×  |
| RawGAT-ST            | 54,45     | GPU/Keras   | 176,81    | 3,2×  |
| AST                  | 44,46     | GPU/Keras   | 223,76    | 5,0×  |
| WavLM Original       | 15,77     | GPU/PyTorch | 379,14    | 24,0× |
| HuBERT Original      | 15,98     | GPU/PyTorch | 395,35    | 24,7× |

Valores em milissegundos, mediana de 30 inferências de amostra única.

Três leituras seguem, e a primeira **corrige uma impressão que a tabela original induzia**. Os dois *backbones* SSL apareciam como os *mais rápidos* do recorte (15,77 ms e 15,98 ms); no mesmo dispositivo, são dos *mais lentos*, com quase 400 ms — fator de 25×. A leitura anterior media a GPU, não o modelo: 94,8 milhões de parâmetros continuam sendo 94,8 milhões quando não há acelerador.

Segundo, dois modelos ficam **mais rápidos** em CPU do que os seus próprios números de GPU — Res2Net (0,5×) e CCT (0,9×). Não é anomalia: em lote unitário, o custo de

lançamento de núcleos na GPU domina o tempo de redes pequenas, e a aceleração não se paga.

Terceiro, os modelos de áudio bruto ocupam a faixa intermediária-alta (AASIST 94,82 ms, RawNet2 119,26 ms, RawGAT-ST 176,81 ms), o que é coerente com o comprimento da sequência que processam: 48.000 amostras contra as 100 ou 300 linhas dos mapas espectrais. O ordenamento por custo de inferência em dispositivo comum é, portanto: SVM, Res2Net, CCT, Conformer, *Random Forest*, AASIST, RawNet2, RawGAT-ST, AST, WavLM e HuBERT.

#### Condição de medição: por que os valores são de máquina ociosa

Uma primeira série destas medidas foi tomada com a máquina sob carga (um treinamento concorrente ocupava 8 dos 12 núcleos) e foi **descartada**, porque a contenção distorceu os resultados de forma  *muito*  desigual. A maior parte dos modelos variou entre 0 e 14 % entre as duas séries, mas o **RawNet2 caiu** 78,6 % — de 556 ms sob carga para 119,26 ms em máquina ociosa — deslocando-se da última para a 7.<sup>a</sup> posição.

O episódio é instrutivo. Sob a medida contaminada, o RawNet2 aparecia como o modelo mais lento do recorte, e havia uma explicação mecanicista convincente à mão: a sua GRU é sequencial por construção e não se beneficia de paralelismo. A explicação era plausível, ajustava-se ao número — e estava errada. Redes recorrentes são sensíveis à disponibilidade de núcleos e ao escalonamento, o que as torna as  *mais*  penalizadas por contenção, não as mais lentas em absoluto. Os valores da Tabela 19 são os da máquina ociosa, com o processo restrito aos mesmos 3 núcleos em ambas as séries para que a única variável fosse a carga.

Feita a ressalva, o modelo mais compacto do conjunto é o **AASIST** (396.312 parâmetros, 1,88 MB), que combina esse custo com a 7.<sup>a</sup> melhor acurácia limpa (94,72%; 5.<sup>a</sup> em EER) e a 2.<sup>a</sup> melhor robustez a 10 dB (91,68%) — a relação custo-benefício mais favorável do recorte, com a ressalva do seu *t*-DCF\* discutida acima. O RawGAT-ST vem logo depois em tamanho (650.204 parâmetros, 2,80 MB), mas com o pior desempenho do conjunto em todas as métricas.

O SVM alia latência mínima (0,90 ms) a acurácia de 93,20% em condição limpa e 85,53% a 10 dB, o que o torna atrativo sob restrição severa de recursos — desde que o ponto de operação não precise sobreviver a SNR abaixo do visto em treino (Seção 6.4.5). O CCT ocupa uma posição notável: com apenas 1,82 milhão de parâmetros e 7,13 MB, alcança 99,57% de acurácia limpa, a 2.<sup>a</sup> melhor do recorte — duas ordens de grandeza mais leve que o AST, com desempenho estatisticamente indistinguível dele.

No outro extremo, os *backbones* SSL são os mais pesados ( $\approx$ 94,8 milhões de parâmetros,  $\approx$ 362 MB), seguidos pelo AST (85,5 milhões, 326,73 MB) e pelo Conformer (28,6 milhões, 110,23 MB). Não há, portanto, um único modelo dominante em todas as dimensões: a escolha depende de qual restrição — latência, memória, robustez ou custo de falso-alarme — é prioritária na aplicação-alvo.

#### 6.4.12 Estabilidade de Treinamento

A Tabela 15 e a Figura 8 mostram sete dos nove modelos neurais convergindo de forma estável. Os picos de validação ocorrem em épocas variadas — HuBERT Original na 12, Res2Net e RawGAT-ST na 17, RawNet2 na 30, AST na 33, CCT na 48, AASIST na 52, WavLM Original na 57 e Conformer na 69 —, sem convergência precoce sistemática sob o protocolo com *augmentation* de ruído no treino.

Duas observações sobre a leitura dessa tabela. A coluna *Época* registra o pico da *acurácia* de validação, grandeza descritiva da curva; o *checkpoint* avaliado é o de menor *perda* de validação limpa (Seção 5.7), e os dois critérios podem apontar épocas distintas. E a coluna *Queda* é a diferença pico—final, que a própria legenda adverte não demonstrar estabilidade: os indicadores do diagnóstico automático são outros e vêm do campo `training_stability` de cada `results.json`, citados a seguir.

**Dois modelos violam o critério de estabilidade** adotado (desvio do monitor  $\leq 0,02$  nas últimas 50 épocas e queda máxima entre épocas consecutivas  $\leq 0,20$ ), e ambos são classificados automaticamente como `unstable_oscillation` pelo diagnóstico do *pipeline*:

- **Conformer** — desvio de 0,0963 na cauda (quase cinco vezes o critério) e queda máxima de 0,4890 entre épocas consecutivas, a maior do recorte. É o caso mais grave, e recai justamente sobre o 3.º melhor modelo do trabalho. Note-se que a coluna *Queda* da Tabela 15 registra 0,34% para o Conformer, a *menor* do recorte: é a diferença pico—final, que nada diz sobre a oscilação percorrida entre as duas pontas. As duas grandezas medem coisas diferentes e não se contradizem.
- **RawGAT-ST** — desvio de 0,0274 e queda máxima de 0,2232, com o melhor *checkpoint* ocorrendo já na época 17 de 100 e degradação sustentada depois disso (*val\_loss* de 0,511 no melhor ponto contra 1,18 ao final): a assinatura clássica de sobreajuste.

A consequência metodológica é relevante e precisa ser declarada: sob oscilação dessa magnitude, a época selecionada pelo critério de menor *val\_loss* limpa depende parcialmente do *ruído* da curva de validação, e não apenas do mérito do modelo naquele ponto. Uma reexecução com outra semente poderia selecionar uma época distinta e produzir métricas apreciavelmente diferentes para esses dois modelos — ao contrário dos outros sete, cujas curvas são planas na cauda. Os resultados do Conformer e do RawGAT-ST devem, portanto, ser lidos com uma margem de incerteza maior que a dos demais, e o RawGAT-ST em particular é candidato natural a retreinamento com regularização reforçada.

#### 6.4.13 Ressalva sobre Execução Única

Todos os resultados reportados correspondem a uma única execução por modelo, sem repetição sobre múltiplas sementes aleatórias de inicialização e particionamento. Diferenças pequenas entre modelos próximos, como AST e CCT (99,71% vs. 99,57% de acurácia),

não devem ser interpretadas como estatisticamente significativas. Uma régua útil é o intervalo de confiança de Wilson a 95% para a acurácia, que quantifica apenas a incerteza amostral do conjunto de teste ( $n = 1.382$ ): no topo, 99,71% corresponde a [99,26%; 99,89%]; no modelo de menor acurácia, 87,55% corresponde a [85,71%; 89,19%]. O intervalo de Wilson **não é simétrico** em torno da proporção observada — e quanto mais próxima de 1 ela está, mais assimétrico ele fica —, razão pela qual se reporta o intervalo, e não uma margem  $\pm$ . Sob essa régua — e sob os testes pareados por *cluster* com correção de Holm, que não rejeitam a hipótese nula para nenhum par entre AST, CCT e Conformer —, o topo do recorte é indistinguível, enquanto as diferenças de robustez a 10 dB discutidas acima — de 15,2 pontos percentuais entre os extremos (Res2Net, 92,40%; RawGAT-ST, 77,21%) — excedem em muito a incerteza amostral. Note-se que esse intervalo não captura a variância de treinamento (inicialização, ordem dos lotes), que só múltiplas execuções revelariam. A magnitude dos efeitos discutidos acima (robustez a ruído e diferença entre famílias de modelos) é suficiente para uma interpretação qualitativa, mas comparações finas entre os modelos de melhor colocação exigiriam múltiplas execuções com relato de média e desvio-padrão.

A leitura das métricas deve ser feita, portanto, com parcimônia: os resultados caracterizam desempenho em distribuição fechada e dependem do *corpus*, das fontes de geração e do protocolo de particionamento utilizado. A eficiência computacional, a estabilidade e, sobretudo, a robustez a ruído são parte essencial da interpretação, não apenas complementos gráficos ao desempenho no conjunto limpo.

## 7 Conclusões

Este trabalho desenvolveu e avaliou um *pipeline* modular e de código aberto para detecção de áudio sintético em português, submetendo 11 modelos heterogêneos — de classificadores estatísticos clássicos a redes espectro-temporais e *backbones* auto-supervisionados — a um protocolo comum de treinamento, inferência e *benchmark*. Nesse sentido, os objetivos específicos e as questões de pesquisa delineados na Seção 1 foram atendidos: consolidou-se uma base balanceada com particionamento reprodutível, implementou-se o pré-processamento acústico e a extração de características, e os modelos foram comparados sob métricas de acurácia, F1-score, AUC-ROC, EER, *t*-DCF, latência, tamanho de artefato e robustez a ruído.

### Resposta às Questões de Pesquisa

**QP1 — como as famílias se comparam sob protocolo único.** No conjunto de teste limpo lideram as arquiteturas espectro-temporais densas, com AST, CCT e Conformer acima de 98% de acurácia; os testes pareados, porém, não separam os três, que constituem **empate técnico** (Seção 6.4). A primeira colocação do AST não isola mérito arquitetural: ele é o único modelo do recorte que parte de pesos pré-treinados e o único que recebe a grade espectral mais fina (Seção 5.3). A ablação executada (Seção 6.4.6) quantifica a primeira dessas vantagens e **reverte o ordenamento**: sem os pesos AudioSet, o AST cai a 94,93%, abaixo de CCT, Conformer e Res2Net — todos treinados do zero. A topologia ViT-Base não é, portanto, superior às alternativas neste *corpus*; o que a colocava em primeiro lugar era o pré-treinamento em larga escala, que vale 4,78 pontos percentuais de acurácia e multiplica o EER por 30. Os *backbones* auto-supervisionados congelados ficaram na metade inferior — posição que contrasta com a literatura recente, em que *front-ends* SSL figuram entre os melhores sistemas. A explicação mais provável está no *back-end* raso aqui empregado (um perceptron sobre *pooling* global), e não no congelamento em si; o ajuste fino integral permanece como caminho não explorado, ao custo de duas ordens de grandeza a mais de parâmetros treináveis.

**QP2 — se a acurácia limpa prediz a robustez.** Não prediz, e esse é o achado central. Sob ruído o ordenamento se reorganiza: AST e CCT deixam o bloco de topo, e o RawGAT-ST separa-se de todos os demais como o menos robusto. O que os dados sustentam é o deslocamento *entre blocos*, não a ordem dentro deles — a 10 dB os cinco primeiros formam um conjunto mutuamente indistinguível por McNemar e por *bootstrap* de EER.

Quanto aos *fatores* que explicam a dissociação, a resposta é **negativa em três frentes**. A representação de entrada não explica: AASIST e RawGAT-ST partilham forma de onda e *front-end* SincConv e separam-se por 14,5 pontos percentuais. A capacidade

tampouco: são os dois menores modelos do recorte. E o diagnóstico automático de estabilidade também não ordena — o Conformer recebe o mesmo rótulo de instabilidade do RawGAT-ST e é o 3.<sup>o</sup> mais robusto do conjunto. O sobreajuste do RawGAT-ST permanece a leitura mais econômica para aquele par específico, mas com  $n = 2$  é hipótese para trabalho futuro, não achado estabelecido. O fator discriminante da robustez a ruído **não foi identificado**, e delimitá-lo exigiria as múltiplas sementes que este recorte não tem.

No nível *não visto* de 5 dB (Seção 6.4.5), o SVM revela falha de natureza distinta: colapsa a 58,68% de acurácia mantendo AUC-ROC de 0,884 — perda de calibração, não de capacidade discriminativa. A ressalva importa: essa perda *não* é recuperável pelos meios disponíveis, já que o limiar calibrado fora do teste piora o resultado e o teto de 79,59% exige um oráculo ajustado na própria partição de avaliação.

**QP3 — compromissos de custo e modelo dominante.** Nenhum modelo domina simultaneamente acurácia, robustez, latência e memória. Três perfis se destacam por razões distintas: o **CCT**, com desempenho indistinguível do AST a duas ordens de grandeza menos parâmetros; o **Res2Net**, que encabeça a robustez a 10 dB a custo intermediário; e o **SVM**, de latência mínima em CPU, com duas ressalvas que pesam sobre a recomendação — o ponto de operação não sobrevive a SNR abaixo do visto em treino e a acurácia despenca a 54,03% no pior dos 11 locutores de teste. Os *backbones* SSL, os mais pesados do conjunto, não compensam o custo neste protocolo.

A remediação em dispositivo único (Tabela 19) reforça esse último ponto e desfaz uma impressão da tabela original: WavLM e HuBERT *pareciam* os modelos mais rápidos do recorte ( $\approx 16$  ms), mas aquilo media a GPU, não o modelo. Em dispositivo comum eles gastam quase 400 ms — fator de  $24\times$  — e passam a ser os **dois mais lentos** do recorte. Res2Net e CCT, ao contrário, são *mais rápidos* em CPU do que os seus próprios números de GPU, porque em lote unitário o custo de lançamento na GPU domina o tempo de redes pequenas. Para implantação sem acelerador — o cenário mais comum em produção — o compromisso favorece SVM, Res2Net e CCT, e desaconselha os SSL.

O custo de *treinamento* acrescenta uma quarta dimensão que não acompanha as demais: RawGAT-ST e AASIST consomem 67% das 63,3 h do *benchmark* e terminam nas duas piores posições entre os neurais, enquanto o CCT alcança o segundo melhor resultado em menos de uma hora. O que domina é o custo por passo da topologia, não a contagem de parâmetros — os grafos de atenção dinâmicos do RawGAT-ST, com 650 mil parâmetros, treinam em mais que o dobro do tempo do AST, com 85,5 milhões. É a dimensão de custo que mais pesa para quem itera sobre o detector, e a que menos aparece nas comparações usuais da literatura.

## Duas Ressalvas de Validade

A **dispersão entre locutores não vistos** qualifica todas as médias reportadas (Seção 6.4.8). O SVM cai de 93,20% de média para 54,03% no pior dos 11 falantes do teste — quatro pontos acima do acaso —, e RawGAT-ST, HuBERT Original, *Random Forest* e RawNet2 exibem o mesmo padrão em grau menor. Só AST e CCT mantêm, no pior locutor, desempenho próximo da própria média. Em biometria de voz essa concentração do erro significa exclusão sistemática de indivíduos, e não degradação difusa: é um risco que nenhuma métrica agregada capta.

A **separabilidade elevada** exige leitura cuidadosa. Os valores de AUC-ROC próximos da unidade não decorrem de vazamento de partição — o protocolo audita sobreposição nula em locutor, frase, texto e conteúdo, e o oráculo majoritário fica no nível do acaso (Seção 5.4.3). São compatíveis, porém, com a hipótese de que os modelos aprenderam a assinatura do *gerador específico* usado (XTTS-v2), e não a distinguir fala natural de fala sintética em geral. As métricas reportadas devem, portanto, ser entendidas como desempenho contra aquele sintetizador, não como estimativa de generalização para geradores não vistos ou condições reais de canal.

### 7.1 Principais Contribuições

- Um *pipeline* reproduzível e de código aberto que padroniza pré-processamento, treinamento, inferência e geração automática de relatórios de *benchmark* para detecção de áudio sintético em português.
- Uma comparação empírica de 11 arquiteturas heterogêneas sob protocolo único, contemplando eficiência computacional e robustez a ruído AWGN, em *corpus* de fala em português brasileiro.
- Um **protocolo de avaliação auditável** para detecção de áudio sintético em português: *corpus* pareado (mesma locução, mesmo locutor, clonada), partição por disjunção dupla locutor  $\times$  frase com sobreposição verificada em quatro dimensões, congelamento criptográfico da partição de teste anterior ao treinamento confirmatório e oráculo majoritário reportado como evidência de ausência de atalho. Cada artefato de resultado carrega essas verificações, de modo que a alegação de integridade do protocolo é conferível, e não declarativa.
- Uma comparação estatística pareada entre todos os pares de modelos (McNemar e *bootstrap* agrupados por frase, com correção de Holm sobre 55 comparações), que distingue diferenças reais de ruído amostral — e mostra que o topo do recorte é um empate técnico.
- Uma **implementação de referência operável** (Apêndice B), em que os 11 modelos

avaliados são servidos por três interfaces sobre o mesmo núcleo de domínio — API REST, interface interativa e CLI —, empacotada em contêineres e restrita aos artefatos promovidos. Ela fecha o percurso da QP3: a escolha do detector deixa de ser uma linha de tabela e passa a ser uma configuração de implantação.

## 7.2 Limitações

As conclusões restringem-se ao *corpus* e ao gerador avaliados. As principais limitações são:

- (i) **Gerador único.** Toda a classe sintética provém do XTTS-v2. Os resultados medem a detecção *daquele* processo de síntese; a generalização para geradores não vistos — regime avaliado por campanhas como o ASVspoof, com dezenas de ataques — permanece não medida, e é a limitação mais relevante para validade externa.
- (ii) **Fonte única e canal homogêneo.** O *corpus* deriva de uma única base (CETUC), com condições de gravação uniformes. A robustez a degradações reais de canal (co-decs, reverberação, ruído não estacionário) não é avaliada: o AWGN cobre apenas degradação aditiva estacionária.
- (iii) **Execução única por modelo**, sem repetição sobre múltiplas sementes, o que impede estimar variância de treinamento. A limitação pesa especialmente sobre Conformer e RawGAT-ST, cujas curvas de validação oscilam o bastante para que a época selecionada dependa parcialmente de ruído (Seção 6.4).
- (iv) **SSL apenas com *backbone* congelado**, sem ajuste fino integral — condição que a literatura recente indica ser determinante para o desempenho dessa família.
- (v) ***t*-DCF como *proxy* interno**, sem acoplamento a um sistema ASV oficial; os valores são comparáveis entre si, mas não com os publicados em campanhas ASVspoof.
- (vi) **Latências da Tabela 13 não comparáveis entre *runtimes***: medidas em TensorFlow/GPU, PyTorch/GPU e scikit-learn/CPU, valem como ordem de grandeza dentro de cada ambiente. A limitação foi **sanada** por uma remedição em dispositivo único e máquina ociosa (Tabela 19), que estabelece o ordenamento comparável entre as onze arquiteturas.
- (vii) **Artefato do RawGAT-ST anterior ao ajuste de regularização.** O modelo publicado foi treinado antes do último ajuste de *dropout*, regularização  $L_2$  e cronograma de decaimento; seus números tendem a melhorar em um retreinamento, e devem ser lidos como limite inferior para essa arquitetura.
- (viii) **Os classificadores clássicos não são cegos ao conjunto de teste.** O vetor tabular passou de 63 para 183 descritores depois de uma rodada exploratória em que SVM e *Random Forest* perderam o ponto de operação a 5 dB (Seção 5.3). O congelamento

criptográfico da partição protege os nove modelos neurais, cuja configuração não foi tocada após o selo, mas não esses dois: o seu *front-end* de entrada foi escolhido com conhecimento de um resultado de teste, e as suas métricas são estimativa otimista. O custo dessa não cegueira foi **medido** sobre uma partição fresca de 4916 amostras (Seção 6.4.7): 2,6 e 3,5 pontos percentuais de acurácia para SVM e *Random Forest*, com degradação maior nas métricas de ordenação. A limitação, portanto, é conhecida em magnitude — e não invalida as conclusões sobre os clássicos, apenas as desloca.

- (ix) **Front-end espectral não uniforme.** O AST opera sobre mapas  $300 \times 128$  e parte de pesos AudioSet, enquanto Conformer, CCT e Res2Net operam sobre  $100 \times 80$  e treinam do zero (Seção 5.3). Das duas variáveis confundidas, o **pré-treinamento foi isolado e medido** (Seção 6.4.6): vale 4,78 pontos de acurácia, e a sua remoção derruba o AST abaixo das três redes treinadas do zero. Permanece não atribuída a parcela devida à *resolução*, que exigiria um segundo braço ( $100 \times 80$  com pesos transferidos).

### 7.3 Trabalhos Futuros

A prioridade, decorrente das limitações (i) e (ii), é a avaliação ***cross-generator***: incluir na partição de teste amostras de sintetizadores ausentes do treino — outros modelos zero-shot, sistemas de conversão de voz e *vocoders* de arquitetura distinta — para separar a detecção de artefatos gerais de síntese do reconhecimento da assinatura do XTTS-v2. É esse ensaio que converte as métricas aqui reportadas em estimativa de capacidade de detecção, e não apenas de desempenho contra um gerador.

Em segundo lugar, a validação ***cross-dataset***, com bases de proveniência e condições de gravação distintas, mede a sensibilidade a canal que o *corpus* homogêneo atual não expõe — complementada por ensaios de robustez contra compressão por codec, reverberação e ruído real, mais representativos do que o AWGN.

No protocolo de avaliação, três extensões diretas: (a) repetir o *benchmark* sobre múltiplas sementes com relato de média e desvio-padrão, quantificando a variância de treinamento que o intervalo de confiança amostral não captura — prioritário para Conformer e RawGAT-ST, os dois modelos com oscilação de validação; (b) retreinar o RawGAT-ST com a regularização ajustada (limitação (vii)) e reavaliar sua posição no recorte; e (c) re-medir a latência de todos os modelos em um único *runtime* e dispositivo de referência, tornando a dimensão de custo comparável entre famílias.

Da limitação (ix) resta *um* braço de ablação. O primeiro — grade  $300 \times 128$  com **pretrained = false** — já foi executado e é relatado na Seção 6.4.6: isola o pré-treinamento e mostra que ele responde por 4,78 pontos percentuais de acurácia, além de reverter a posição do AST no recorte. Falta o braço complementar, grade  $100 \times 80$  com **pretrained = true**, que isolaria a contribuição da *resolução* mantendo os pesos transferidos. Com ele, somado à configuração publicada e ao braço já executado, o fatorial  $2 \times 2$  se fecha e a

parcela devida a cada fator torna-se integralmente atribuível.

A limitação (viii) já foi quantificada na Seção 6.4.7; o passo remanescente é estender esse mesmo desenho de partição fresca aos nove modelos neurais, de modo que todo o recorte — e não apenas os clássicos — disponha de uma estimativa fora da amostra que orientou qualquer decisão.

Por fim, destacam-se a pertinência de adotar o *t*-DCF padronizado acoplado a um sistema ASV de referência, avaliar os *backbones* SSL com ajuste fino integral e com *back-ends* mais expressivos que o perceptron sobre *pooling* global, e incorporar métricas de calibração e explicabilidade local (SHAP, Grad-CAM) ao relatório automático do *pipeline*.

## Referências

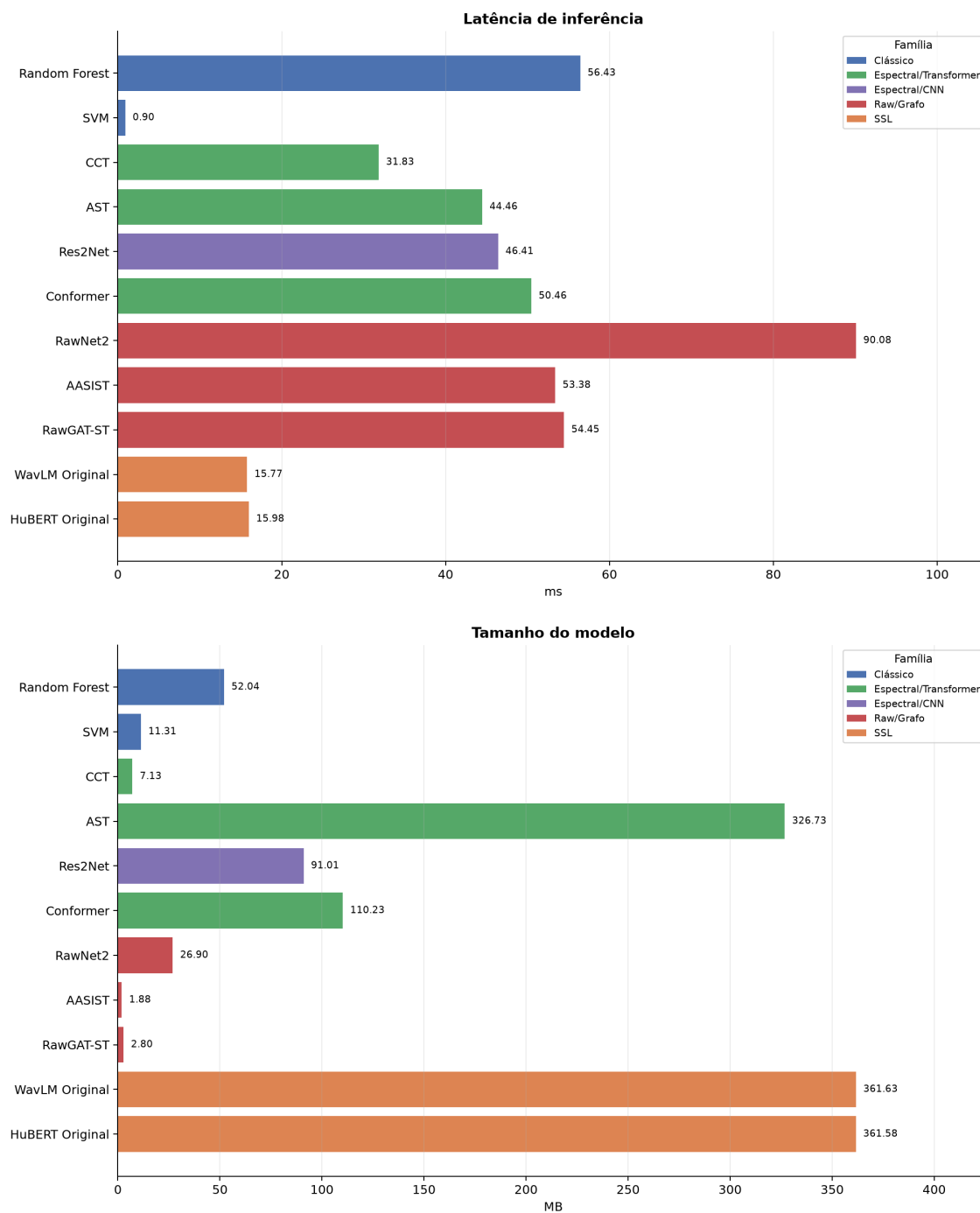
- [1] JIA, Y. et al. Transfer learning from speaker verification to multispeaker text-to-speech synthesis. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 31, 2018.
- [2] DUDLEY, H. The vocoder. **Bell Labs Record**, v. 18, n. 4, p. 122–126, 1939.
- [3] KLATT, D. H. Review of text-to-speech conversion for English. **The Journal of the Acoustical Society of America**, v. 82, n. 3, p. 737–793, 1987.
- [4] HUNT, A. J.; BLACK, A. W. Unit selection in a concatenative speech synthesis system using a large speech database. In: **IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)**, 1996.
- [5] VAN DEN OORD, A. et al. WaveNet: A generative model for raw audio. **arXiv preprint** arXiv:1609.03499, 2016.
- [6] WANG, Y. et al. Tacotron: Towards end-to-end speech synthesis. In: **Proc. Interspeech 2017**, 2017.
- [7] WANG, C. et al. VALL-E: Neural Codec Language Models are Zero-Shot Text-to-Speech Synthesizers. **arXiv preprint** arXiv:2301.02111, 2023. Disponível em: <https://arxiv.org/abs/2301.02111>. Acesso em: 15 jun. 2025.
- [8] MAI, K. T. et al. Warning: humans cannot reliably detect speech deepfakes. **PLOS ONE**, v. 18, n. 8, e0285333, 2023.
- [9] HEIDARI, A. et al. Deepfake detection using deep learning methods: A systematic and comprehensive review. **WIREs Data Mining and Knowledge Discovery**, v. 14, n. 2, e1520, 2024.
- [10] SHEN, J. et al. Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 4779–4783, 2018.
- [11] BIRD, J. J.; LOTFI, A. Real-time Detection of AI-Generated Speech for DeepFake Voice Conversion. **arXiv preprint** arXiv:2308.12734, 2023. Disponível em: <https://arxiv.org/abs/2308.12734>. Acesso em: 15 jun. 2025.
- [12] PINDROP. **Deepfake Trends to Look Out for in 2025**. 2025. Disponível em: <https://www.pindrop.com/article/deepfake-trends/>. Acesso em: 15 jun. 2025.

- [13] FARRUGIA, B. Brazil's electoral deepfake law tested as AI-generated content targeted local elections. **Digital Forensic Research Lab (Atlantic Council)**, 26 nov. 2024. Disponível em: <https://dfrlab.org/2024/11/26/brazil-election-ai-deepfakes/>. Acesso em: 15 jun. 2025.
- [14] FARRUGIA, B. The challenges of identifying deepfakes ahead of the 2024 Brazil election. **Digital Forensic Research Lab (Atlantic Council)**, 2 out. 2024. Disponível em: <https://dfrlab.org/2024/10/02/brazil-election-ai-research/>. Acesso em: 15 jun. 2025.
- [15] TODISCO, M. et al. ASVspoof 2019: Future Horizons in Spoofed and Fake Speech Detection. In: **Proc. Interspeech 2019**, p. 1008–1012, 2019.
- [16] YAMAGISHI, J. et al. ASVspoof 2021: Accelerating Progress in Spoofed and Deepfake Speech Detection. In: **Proc. ASVspoof 2021 Workshop**, p. 47–54, 2021.
- [17] WANG, X. et al. ASVspoof 5: Crowdsourced Speech Data, Deepfakes, and Adversarial Attacks at Scale. **arXiv preprint** arXiv:2408.08739, 2024. Disponível em: <https://arxiv.org/abs/2408.08739>. Acesso em: 15 jun. 2025.
- [18] YI, J. et al. ADD 2022: The First Audio Deep Synthesis Detection Challenge. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 9216–9220, 2022.
- [19] REN, Y. et al. FastSpeech: Fast, Robust and Controllable Text to Speech. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 32, 2019.
- [20] KIM, J.; KONG, J.; SON, J. Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech. In: **International Conference on Machine Learning (ICML)**, p. 5530–5540, 2021.
- [21] CASANOVA, E. et al. YourTTS: Towards Zero-Shot Multi-Speaker TTS and Zero-Shot Voice Conversion for everyone. In: **International Conference on Machine Learning (ICML)**, p. 2709–2720, 2022.
- [22] UNFAKE. Fake Voices dataset card. **Hugging Face Datasets**, 2026. Disponível em: [https://huggingface.co/datasets/unfake/fake\\_voices](https://huggingface.co/datasets/unfake/fake_voices). Acesso em: 27 jun. 2026.
- [23] KAMEOKA, H. et al. StarGAN-VC: Non-parallel many-to-many voice conversion using star generative adversarial networks. In: **IEEE Spoken Language Technology Workshop (SLT)**, p. 266–273, 2018.
- [24] CORTES, C.; VAPNIK, V. Support-vector networks. **Machine Learning**, v. 20, p. 273–297, 1995.

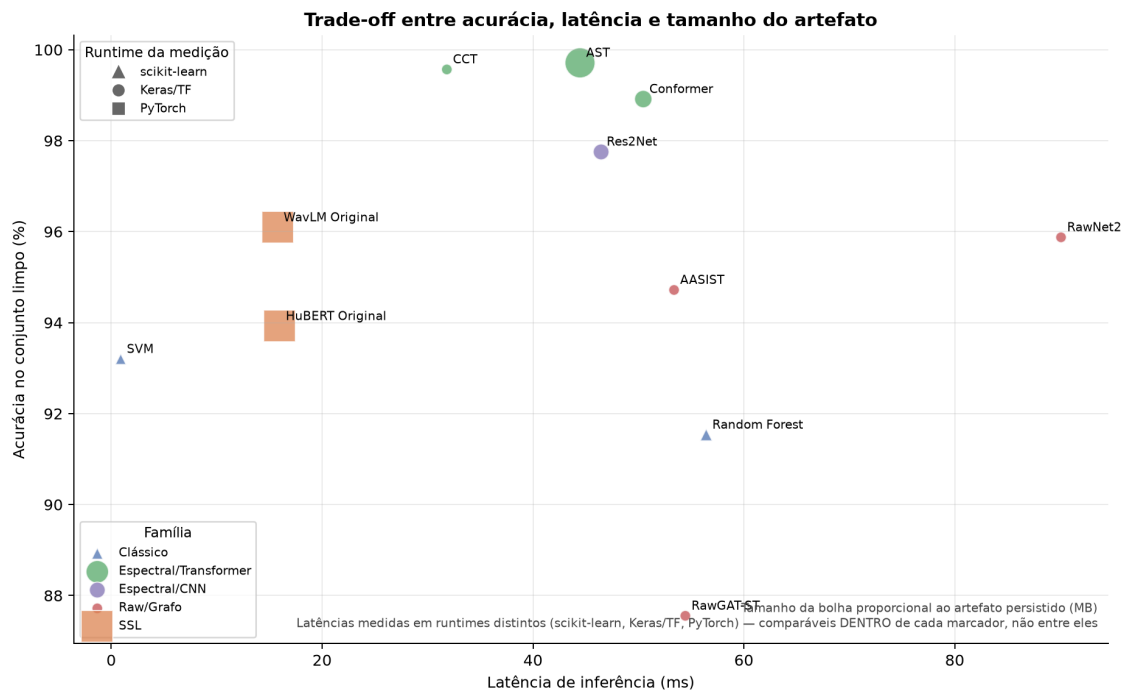
- [25] HOCHREITER, S.; SCHMIDHUBER, J. Long Short-Term Memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997.
- [26] WU, Z. et al. ASVspooF 2015: the First Automatic Speaker Verification Spoofing and Countermeasures Challenge. In: **Proc. Interspeech 2015**, p. 2037–2041, 2015.
- [27] SAHIDULLAH, M. et al. A Comparison of Features for Synthetic Speech Detection. In: **Proc. Interspeech 2015**, p. 2087–2091, 2015.
- [28] TODISCO, M.; DELGADO, H.; EVANS, N. A New Feature for Automatic Speaker Verification Anti-Spoofing: Constant Q Cepstral Coefficients. In: **Proc. Odyssey 2016 — The Speaker and Language Recognition Workshop**, p. 283–290, 2016.
- [29] LAVRENTYEVA, G. et al. Audio Replay Attack Detection with Deep Learning Frameworks. In: **Proc. Interspeech 2017**, p. 82–86, 2017.
- [30] GAO, S. et al. Res2Net: A New Multi-Scale Backbone Architecture. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 43, n. 2, p. 652–662, 2021.
- [31] TAK, H. et al. End-to-End Anti-Spoofing with RawNet2. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 6369–6373, 2021.
- [32] JUNG, J. W. et al. AASIST: Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 6367–6371, 2022.
- [33] CHEN, S. et al. WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing. **IEEE Journal of Selected Topics in Signal Processing**, v. 16, n. 6, p. 1505–1518, 2022.
- [34] HSU, W.-N. et al. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. **IEEE/ACM Transactions on Audio, Speech, and Language Processing**, v. 29, p. 3451–3460, 2021.
- [35] BAEVSKI, A. et al. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. **Advances in Neural Information Processing Systems (NeurIPS)**, v. 33, p. 12449–12460, 2020.
- [36] GONG, Y.; CHUNG, Y.; GLASS, J. AST: Audio Spectrogram Transformer. In: **Proc. Interspeech 2021**, p. 571–575, 2021.

- [37] VASWANI, A. et al. Attention Is All You Need. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 30, 2017.
- [38] DOSOVITSKIY, A. et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In: **International Conference on Learning Representations (ICLR)**, 2021.
- [39] AKCIT-DEEPPFAKE. BRSpeech-DF dataset card. **Hugging Face Datasets**, 2026. Disponível em: <https://huggingface.co/datasets/AKCIT-Deepfake/BRSpeech-DF>. Acesso em: 27 jun. 2026.
- [40] MÜLLER, N. M. et al. Does Audio Deepfake Detection Generalize? In: **Proc. Interspeech 2022**, p. 2783–2787, 2022.
- [41] TAK, H. et al. End-to-End Spectro-Temporal Graph Attention Networks for Speaker Verification Anti-Spoofing and Speech Deepfake Detection. In: **Proc. ASVspoof 2021 Workshop**, p. 1–8, 2021.
- [42] GULATI, A. et al. Conformer: Convolution-augmented Transformer for Speech Recognition. In: **Proc. Interspeech 2020**, p. 5036–5040, 2020.
- [43] HASSANI, A. et al. Escaping the Big Data Paradigm with Compact Transformers. **arXiv preprint** arXiv:2104.05704, 2021. Disponível em: <https://arxiv.org/abs/2104.05704>. Acesso em: 28 jun. 2026.
- [44] BREIMAN, L. Random Forests. **Machine Learning**, v. 45, p. 5–32, 2001.
- [45] HERMANSKY, H.; MORGAN, N. RASTA Processing of Speech. **IEEE Transactions on Speech and Audio Processing**, v. 2, n. 4, p. 578–589, 1994.
- [46] DAVIS, S. B.; MERMELSTEIN, P. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. **IEEE Transactions on Acoustics, Speech, and Signal Processing**, v. 28, n. 4, p. 357–366, 1980.
- [47] PEETERS, G. A large set of audio features for sound description (similarity and classification) in the CUIDADO project. **Relatório técnico**. Paris: IRCAM, 2004.
- [48] BROWN, J. C. Calculation of a constant Q spectral transform. **The Journal of the Acoustical Society of America**, v. 89, n. 1, p. 425–434, 1991.
- [49] BOERSMA, P. Accurate short-term analysis of the fundamental frequency and the harmonics-to-noise ratio of a sampled sound. **Proceedings of the Institute of Phonetic Sciences**, v. 17, p. 97–110, 1993.

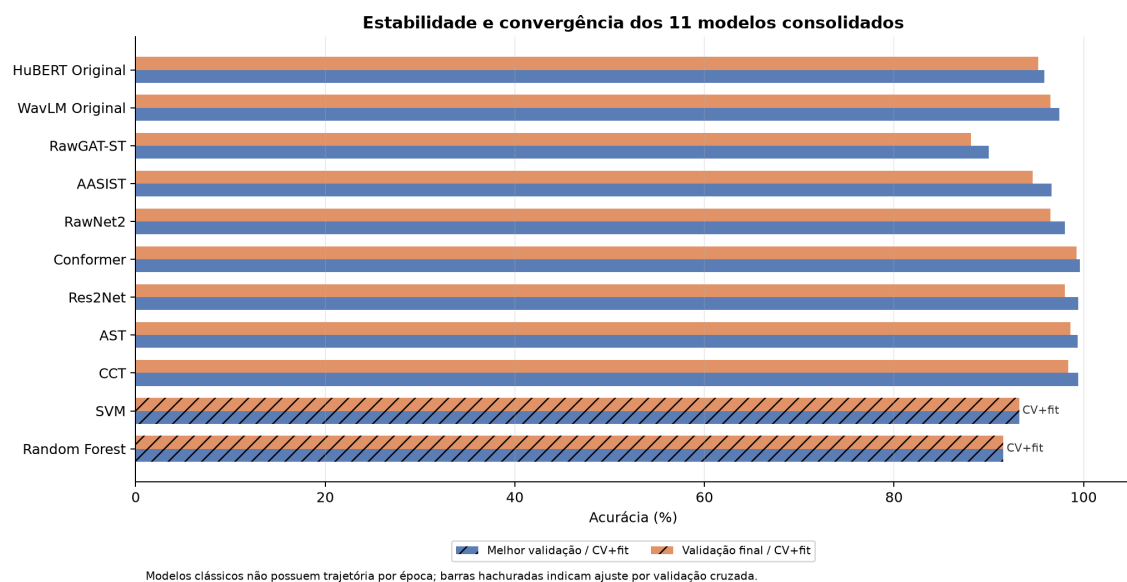
- [50] FURUI, S. Speaker-independent isolated word recognition using dynamic features of speech spectrum. **IEEE Transactions on Acoustics, Speech, and Signal Processing**, v. 34, n. 1, p. 52–59, 1986.
- [51] ALENCAR, V. F. S.; ALCAIM, A. Transformadas LSF e LPC no reconhecimento de voz contínua para o português brasileiro. In: **Anais do XXVI Simpósio Brasileiro de Telecomunicações (SBrT)**, Rio de Janeiro, 2008. Disponível em: <https://www02.smt.ufrj.br/~igor.quintanilha/>. Acesso em: 2 ago. 2026. Referência canônica do *corpus* CETUC.
- [52] CASANOVA, E. et al. XTTS: a Massively Multilingual Zero-Shot Text-to-Speech Model. **Proc. Interspeech 2024**, p. 4978–4982, 2024. Disponível em: <https://arxiv.org/abs/2406.04904>. Acesso em: 2 ago. 2026.
- [53] KINNUNEN, T. et al. t-DCF: a Detection Cost Function for the Tandem Assessment of Spoofing Countermeasures and Automatic Speaker Verification. In: **Proc. Odyssey 2018 — The Speaker and Language Recognition Workshop**, p. 312–319, 2018.
- [54] LUNDBERG, S. M.; LEE, S.-I. A Unified Approach to Interpreting Model Predictions. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 30, p. 4765–4774, 2017.
- [55] KINGMA, D. P.; BA, J. Adam: A Method for Stochastic Optimization. In: **International Conference on Learning Representations (ICLR)**, 2015.



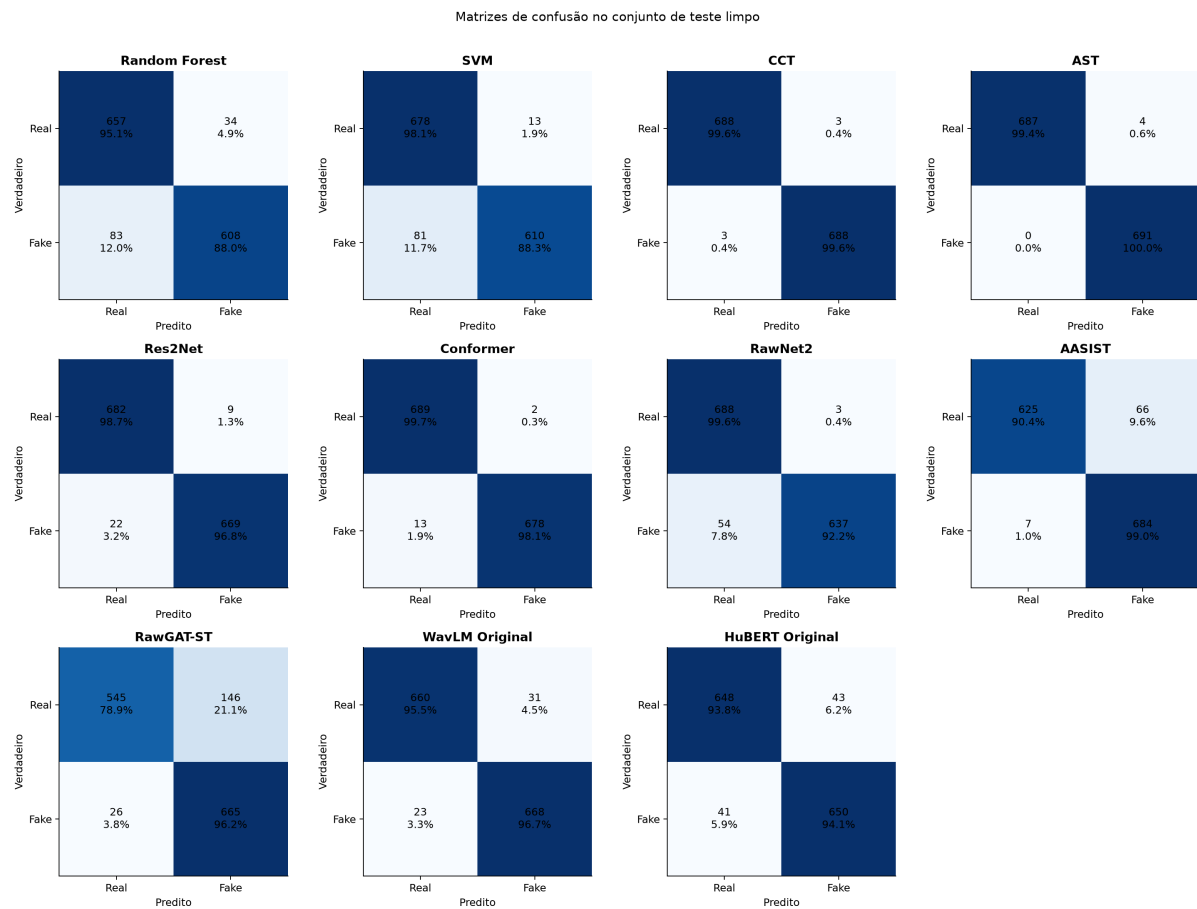
**Figura 6:** Latência e tamanho dos modelos.



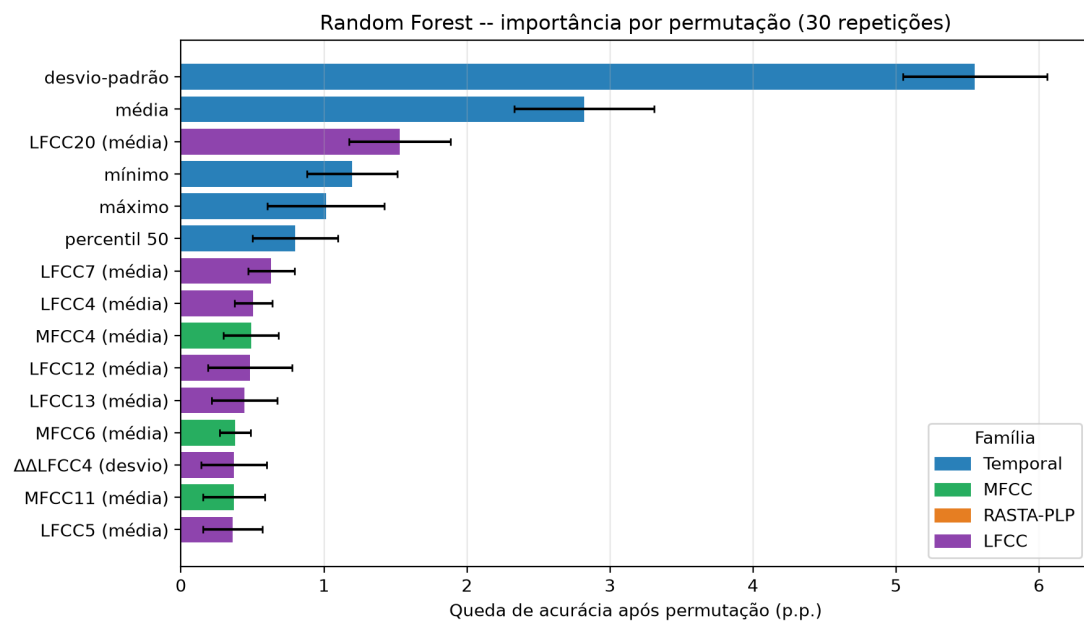
**Figura 7:** Trade-off entre acurácia, latência e tamanho dos artefatos.



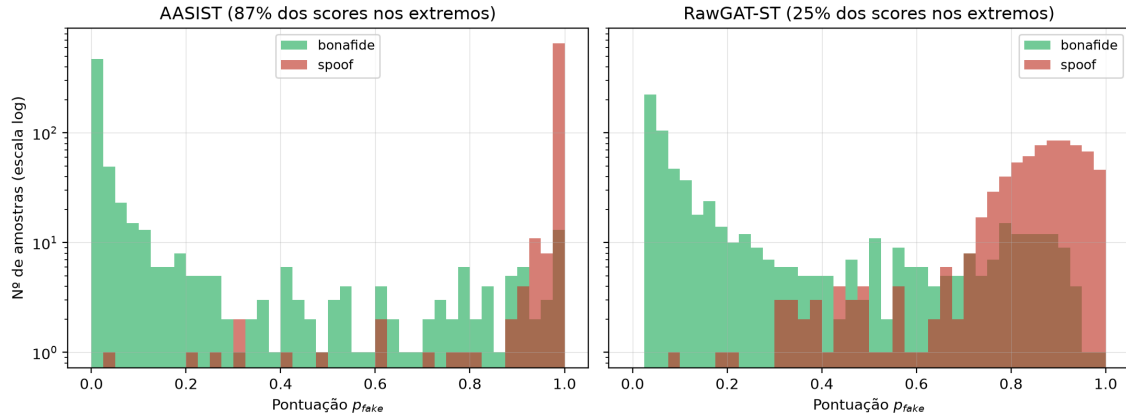
**Figura 8:** Resumo descritivo das curvas de validação em execução única; a diferença pico-final não demonstra, isoladamente, estabilidade de otimização. Modelos clássicos não possuem trajetória por época.



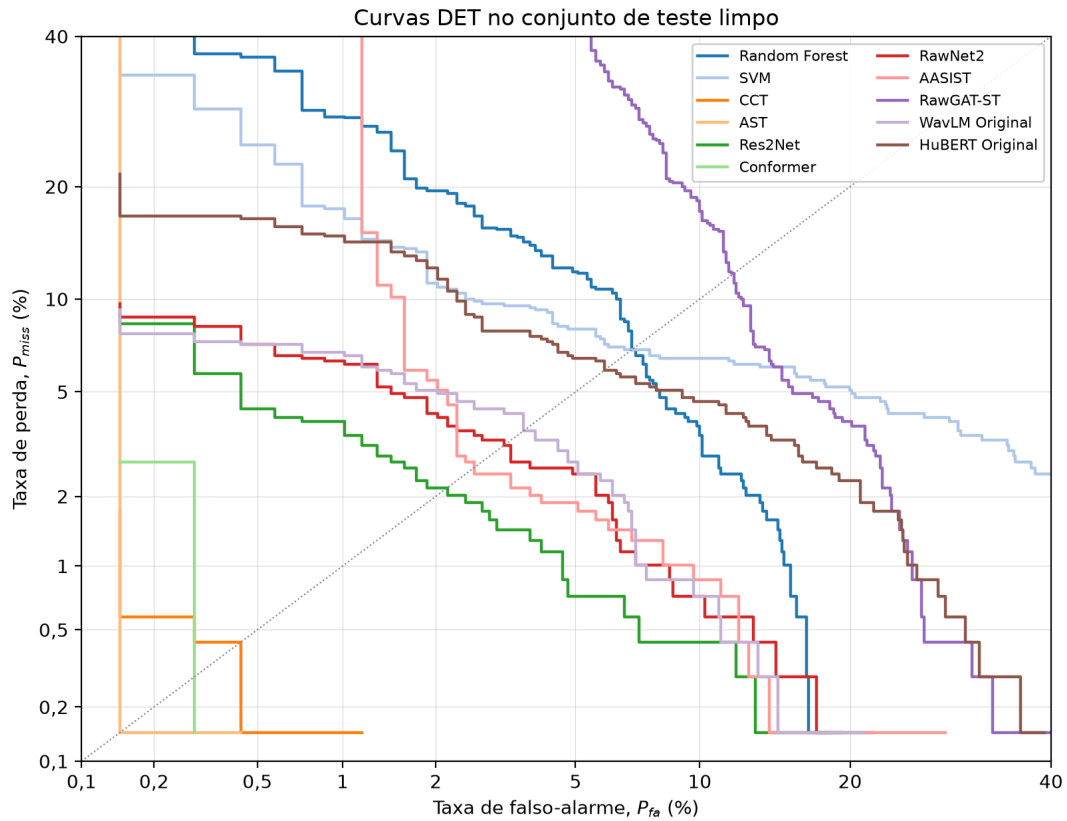
**Figura 9:** Matrizes de confusão dos onze modelos avaliados no artigo.



**Figura 10:** Importância por permutação das 15 características mais relevantes do *Random Forest*, coloridas por família (temporal, LFCC, MFCC, RASTA-PLP), medida na partição de teste com 30 repetições. Agregadas, as famílias respondem por 44,7% (temporal), 41,6% (LFCC), 8,7% (MFCC) e 5,0% (RASTA-PLP) da importância total.



**Figura 11:** Distribuições da pontuação  $p_{\text{fake}}$  por classe verdadeira para AASIST e RawGAT-ST no conjunto de teste limpo, em escala logarítmica (geradas por `scripts/reporting/export_tcc_extra_figures.py`). A cabeça AM-Softmax do AASIST concentra 86,5% dos *scores* nos extremos do intervalo, contra 24,5% no RawGAT-ST.



**Figura 12:** Curvas DET (*Detection Error Tradeoff*) dos 11 modelos no conjunto de teste limpo, em eixos probit (geradas por `scripts/reporting/export_tcc_extra_figures.py`). A curva do AASIST mostra a transição abrupta de  $P_{\text{miss}}$  na região de baixo  $P_{\text{fa}}$ , efeito da cauda de *bonafide* com pontuação saturada no extremo incorreto.

## A Configurações Detalhadas do Pipeline

### A.1 Hiperparâmetros Consolidados

O preset final do *benchmark* utiliza **orçamento fixo de 100 épocas para todas as nove arquiteturas neurais**, *sem parada antecipada* (Seção 5.7), com lote adaptado ao consumo de VRAM, precisão mista quando segura e seleção do *checkpoint* de menor perda de validação limpa. O preset contempla WavLM Original e HuBERT Original como modelos SSL no escopo principal do trabalho. Os modelos clássicos utilizam busca de hiperparâmetros com validação cruzada antes do ajuste final. A Tabela 21 resume esses parâmetros por família de arquitetura, e a Tabela 20 registra o ambiente de software usado nas execuções.

**Tabela 20:** Ambiente de software e parâmetros de reprodução das execuções.

| Item                            | Valor                                                                  |
|---------------------------------|------------------------------------------------------------------------|
| Python                          | 3.11.15                                                                |
| TensorFlow/Keras                | 2.21.0                                                                 |
| PyTorch                         | 2.5.1 (CUDA 12.4)                                                      |
| Execução                        | Docker (WSL2/Linux), imagem<br><code>xfakesong/benchmark:nvidia</code> |
| GPU                             | NVIDIA GeForce RTX 3060 (CC 8.6, precisão mista FP16)                  |
| scikit-learn                    | 1.8.0                                                                  |
| <b>transformers</b>             | 4.45.2                                                                 |
| Semente aleatória               | 42 (particionamento, inicialização e AWGN)                             |
| Dataset                         | <code>data/datasets/benchmark_dataset_15k.npz</code>                   |
| SHA-256 do <i>dataset</i>       | <code>3775b35f...3400f</code>                                          |
| Identidade da partição de teste | <code>4f9b8630...abab6</code>                                          |
| Método da identidade            | SHA-256 sobre (nome, CRC-32, tamanho) de cada membro do arquivo        |

**Tabela 21:** Hiperparâmetros efetivos por modelo, conforme registrados nos artefatos de cada execução. Todas as arquiteturas neurais treinam 100 épocas **sem parada antecipada**;  $\lambda$  = regularização  $L_2$  ou *weight decay*; Aug. = *augmentation* AWGN na forma de onda. Otimizadores Adam/AdamW conforme [55].

| Modelo          | LR                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Lote | Otimizador | $\lambda$          | Aug. |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------|--------------------|------|
| CCT             | $3 \times 10^{-4}$                                                                                                                                                                                                                                                                                                                                                                                                                                 | 32   | AdamW      | $10^{-4}$          | sim  |
| AST             | $10^{-5}$                                                                                                                                                                                                                                                                                                                                                                                                                                          | 8    | AdamW      | $10^{-5}$          | sim  |
| Res2Net         | $10^{-3}$                                                                                                                                                                                                                                                                                                                                                                                                                                          | 32   | AdamW      | —                  | sim  |
| Conformer       | $5 \times 10^{-5}$                                                                                                                                                                                                                                                                                                                                                                                                                                 | 32   | AdamW      | $10^{-4}$          | sim  |
| RawNet2         | $10^{-4}$                                                                                                                                                                                                                                                                                                                                                                                                                                          | 16   | Adam       | —                  | sim  |
| AASIST          | $3 \times 10^{-4}$                                                                                                                                                                                                                                                                                                                                                                                                                                 | 24   | AdamW      | $2 \times 10^{-4}$ | sim  |
| RawGAT-ST       | $5 \times 10^{-5}$                                                                                                                                                                                                                                                                                                                                                                                                                                 | 16   | AdamW      | $10^{-3}$          | sim  |
| WavLM Original  | $10^{-3}$ (cabeça)                                                                                                                                                                                                                                                                                                                                                                                                                                 | 128  | AdamW      | $10^{-4}$          | sim  |
| HuBERT Original | $10^{-3}$ (cabeça)                                                                                                                                                                                                                                                                                                                                                                                                                                 | 128  | AdamW      | $10^{-4}$          | sim  |
| SVM             | GridSearchCV (5 dobras agrupadas por frase), 15 candidatos: RBF com $C \in \{0,1; 1; 10\}$ e $\gamma \in \{\text{scale}, 10^{-3}, 10^{-2}, 10^{-1}\}$ , mais linear com o mesmo $C$ ; reajuste final e calibração isotônica. <b>Selecionado:</b> RBF, $C = 10$ , $\gamma = 0,01$                                                                                                                                                                   |      |            |                    |      |
| Random Forest   | GridSearchCV (5 dobras agrupadas por frase), 108 candidatos: árvores $\in \{200, 300\}$ ; profundidade máx. $\in \{10, 15, 20\}$ ; <i>min. split/leaf</i> $\in \{5, 10, 20\}/\{2, 4, 8\}$ ; <i>max. features</i> $\in \{\text{sqrt}, \log 2\}$ . A grade limita deliberadamente a profundidade e exige folhas maiores, como regularização estrutural. <b>Selecionado:</b> 300 árvores, profundidade 20, <i>split</i> 5, <i>leaf</i> 2, <i>sqrt</i> |      |            |                    |      |

## A.2 Correspondência entre Nomes Acadêmicos e Módulos do Repositório

Três arquiteturas são referidas no texto pelo nome consagrado na literatura, enquanto o código as registra pelo nome descritivo da família. A Tabela 22 estabelece a correspondência, necessária para reproduzir os experimentos a partir do repositório. As implementações seguem os artigos originais nos elementos que definem cada arquitetura — blocos *Bottle2neck* no Res2Net, tokenizador convolucional com *sequence pooling* no CCT, e *patches*  $16 \times 16$  sobre espectrograma com pesos AudioSet no AST.

**Tabela 22:** Correspondência entre o nome usado no texto, a entrada do *benchmark* e o módulo de implementação.

| Nome no texto | Entrada do <i>benchmark</i> | Módulo                                   |
|---------------|-----------------------------|------------------------------------------|
| AST           | SpectrogramTransformer      | architectures/spectrogram_transformer.py |
| CCT           | Hybrid CNN-Transformer      | architectures/hybrid_cnn_transformer.py  |
| Res2Net       | MultiscaleCNN               | architectures/multiscale_cnn.py          |

### A.3 Descritores Acústicos Adicionais Disponíveis no Pipeline

Além dos regimes de entrada efetivamente comparados neste *benchmark*, o repositório implementa extratores adicionais como adaptadores `IFeatureExtractor` (`app/domain/features/extractors/`), utilizáveis em extensões futuras do protocolo (Seção 7.3) mas não consumidos pelos 11 modelos deste trabalho.

**Descritores de forma espectral.** Centroide, *rolloff*, largura de banda, taxa de cruzamento por zero (ZCR), contraste e planura espectrais caracterizam a distribuição de energia no domínio da frequência. As definições adotadas seguem a compilação de Peeters [47], referência canônica para o conjunto de descritores de timbre:

$$SC[m] = \frac{\sum_{k=0}^{K-1} k \cdot |X[m, k]|^2}{\sum_{k=0}^{K-1} |X[m, k]|^2} \quad (\text{centroide}) \quad (14)$$

$$SR[m] = \min \left\{ k_r : \sum_{k=0}^{k_r} |X[m, k]|^2 \geq 0,85 \sum_{k=0}^{K-1} |X[m, k]|^2 \right\} \quad (\text{rolloff a 85\%}) \quad (15)$$

$$SB[m] = \sqrt{\frac{\sum_{k=0}^{K-1} (k - SC[m])^2 \cdot |X[m, k]|^2}{\sum_{k=0}^{K-1} |X[m, k]|^2}} \quad (\text{largura de banda}) \quad (16)$$

$$ZCR[m] = \frac{1}{2N_q} \sum_{n=0}^{N_q-1} |\text{sgn}(x[mH + n]) - \text{sgn}(x[mH + n - 1])| \quad (17)$$

$$SFlatness[m] = \frac{\left( \prod_{k=0}^{K-1} |X[m, k]|^2 \right)^{1/K}}{\frac{1}{K} \sum_{k=0}^{K-1} |X[m, k]|^2} \quad (18)$$

em que  $X[m, k]$  é o espectro de magnitude do quadro  $m$  na *bin* de frequência  $k$ , entre  $K$  *bins* totais, e  $N_q$  é o número de amostras por quadro no cálculo do ZCR. O *rolloff* exige a formulação por  $\min\{\cdot\}$  porque a soma acumulada é discreta: em geral não existe  $k_r$  que satisfaça a igualdade exata, e o descritor é definido como a primeira *bin* em que o limiar de 85% é atingido ou ultrapassado. O centroide e a largura de banda são expressos em índice de *bin*; a conversão para hertz é  $f_k = kf_s/(2(K - 1))$  para um espectro de magnitude obtido por FFT real.

**Constant-Q Transform (CQT).** Alternativa de resolução log-frequencial ao espectrograma linear, formulada por Brown [48] e adotada por Todisco et al. como *front-end*

de referência para *anti-spoofing* [28]:

$$CQT[m, k] = \frac{1}{N_k} \sum_n x[n] w\left(\frac{n - mH}{N_k}\right) e^{-j2\pi Q(n-mH)/N_k} \quad (19)$$

em que  $N_k = \lceil Qf_s/f_k \rceil$  é o comprimento da janela de análise da  $k$ -ésima *bin*,  $f_k = f_{\min} 2^{k/B}$  é a sua frequência central,  $w(\cdot)$  é a janela de análise e

$$Q = \left(2^{1/B} - 1\right)^{-1} \quad (20)$$

é o fator de qualidade, **constante** para todas as *bins* — daí o nome da transformada. É essa constância que faz a razão entre a frequência central e a largura de banda ser a mesma em todas as escalas, produzindo resolução log-frequencial: janelas longas nas baixas frequências, curtas nas altas. Com  $B = 12$  *bins* por oitava e 7 oitavas (84 características) na implementação disponível, a Equação 20 dá  $Q \approx 16,8$  — valor que a Equação 19 aplica igualmente a todas as *bins*.

**Descritores prosódicos e de qualidade vocal.** Complementam os regimes acima com informação de excitação glotal — frequência fundamental (F0), *jitter* e *shimmer* (perturbação ciclo-a-ciclo de período e amplitude) e HNR (*Harmonic-to-Noise Ratio*), nas formulações de Boersma [49] consagradas pela análise clínica de voz:

$$Jitter = \frac{1}{N_c - 1} \sum_{i=1}^{N_c-1} \frac{|T_i - T_{i-1}|}{\bar{T}}, \quad \bar{T} = \frac{1}{N_c} \sum_{i=0}^{N_c-1} T_i \quad (21)$$

$$Shimmer = \frac{1}{N_c - 1} \sum_{i=1}^{N_c-1} \frac{|A_i - A_{i-1}|}{\bar{A}}, \quad \bar{A} = \frac{1}{N_c} \sum_{i=0}^{N_c-1} A_i \quad (22)$$

$$HNR = 10 \log_{10} \left( \frac{P_{\text{harm}}}{P_{\text{ruído}}} \right) \quad (23)$$

em que  $T_i$  e  $A_i$  são o período e a amplitude do  $i$ -ésimo dos  $N_c$  ciclos glotais detectados,  $\bar{T}$  e  $\bar{A}$  as respectivas médias, e  $P_{\text{harm}}/P_{\text{ruído}}$  as potências das componentes harmônica e não-harmônica. As duas primeiras são as formas *locais relativas* de *jitter* e *shimmer*: normalizar pela média torna o descritor adimensional e invariante a mudanças de tom ou de nível, o que é o comportamento desejado para medir *perturbação* ciclo-a-ciclo.

**Características *delta* e *delta-delta*.** Capturam a dinâmica temporal de primeira e segunda ordem de qualquer descritor de quadro  $X[m, c]$ , pela regressão linear proposta por Furui [50]:

$$\Delta X[m, c] = \frac{\sum_{n=1}^N n(X[m+n, c] - X[m-n, c])}{2 \sum_{n=1}^N n^2} \quad (24)$$

A Equação 24 é uma regressão linear sobre janela de  $2N + 1$  quadros; aplicá-la ao próprio

$\Delta$  produz o  $\Delta\Delta$ . É esse par de derivadas que compõe, junto com os coeficientes estáticos, o bloco LFCC de 120 dimensões do vetor tabular (Seção 5.3).

## A.4 Organização dos Artefatos

Cada arquitetura possui diretório próprio em `data/results/`, contendo modelo treinado, configuração, histórico de treinamento, métricas em JSON/CSV, matriz de confusão, curvas ROC/PR, gráficos de acurácia e perda, resultados de robustez e relatório Markdown. Os gráficos consolidados utilizados no TCC ficam em `figures/`.

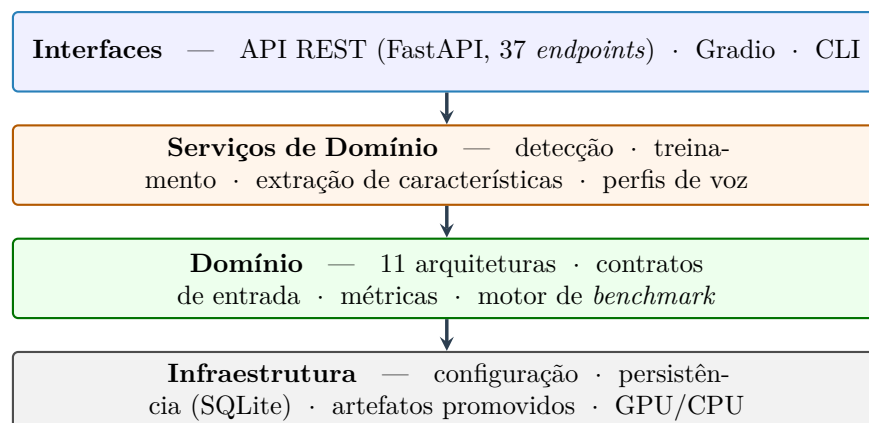
Cada `results.json` carrega ainda os campos de auditoria referidos ao longo do texto: `academic_protocol_guard` (selo criptográfico da partição de teste), `training_stability` (diagnóstico de convergência), `ranking_score_source` (qual *score* alimentou as métricas livres de limiar) e `latency_profile` (com *runtime*, dispositivo e a marcação *cross\_runtime\_comparable*).

## B Sistema Desenvolvido

O *benchmark* das seções anteriores não é um fim em si: ele existe para *selecionar* o detector que um sistema em operação executará. Esta seção descreve esse sistema — a implementação de referência que acompanha o trabalho —, e fecha o percurso que a QP3 abre. A pergunta “qual modelo, sob qual restrição” (Seção 6.4) só se completa quando existe onde aplicá-la.

### B.1 Arquitetura em Camadas

O sistema segue *Clean Architecture*, com a regra de dependência apontando sempre para dentro: as interfaces conhecem o domínio, o domínio não conhece as interfaces. A Figura 13 representa a organização.



**Figura 13:** Arquitetura em camadas do sistema. As setas indicam a direção das dependências: cada camada conhece apenas a inferior. As três interfaces compartilham o mesmo serviço de detecção, de modo que a decisão do *benchmark* vale igualmente para todas.

A consequência prática dessa organização é que o resultado do *benchmark* não precisa ser reimplementado por interface. O contrato de entrada de cada modelo — janela, taxa de amostragem e *front-end* — é declarado no domínio e honrado por todas as vias de acesso, o que elimina a divergência entre o que se mede na avaliação e o que se executa em produção.

## B.2 Interfaces de Acesso

O processo servidor é uma aplicação FastAPI que expõe três vias sobre o mesmo núcleo:

- **API REST** — 37 *endpoints* distribuídos em sete módulos: perfis de voz (9), sistema (8), treinamento (6), detecção (5), *datasets* (4), histórico (3) e características (2). É a via de integração programática.
- **Interface Gradio** — montada em */gradio* sobre a mesma aplicação, com autenticação obrigatória e restrição explícita de caminhos acessíveis (o *mount* bloqueia *.env*, *.git*, a base de dados, os *datasets* e o diretório de modelos). É a via de uso interativo, descrita adiante.
- **CLI** — menus interativos para operação em servidor sem ambiente gráfico, usados nas rotinas de treino e *benchmark*.

## B.3 Organização da Interface Interativa

A interface consolida nove telas funcionais em **cinco seções por perfil de uso**, de modo que cada papel encontre o seu fluxo sem navegar pelo que não lhe diz respeito. A Tabela 23 resume a organização.

**Tabela 23:** Seções da interface interativa e telas que cada uma agrega.

| Seção      | Telas                                        | Função                                                                                                              |
|------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Painel     | <i>dashboard</i>                             | Indicadores, estado dos serviços e atividade recente                                                                |
| Detectar   | detecção, perfis de voz                      | Análise individual e em lote; cadastro de locutores de referência                                                   |
| Investigar | análise forense                              | Inspeção detalhada de uma amostra, com visualizações espectrais                                                     |
| Treinar    | assistente, otimização                       | Treino guiado e busca de hiperparâmetros; desativável por variável de ambiente para implantações somente-inferência |
| Gerenciar  | <i>datasets</i> , características, histórico | Curadoria de dados, inspeção de descritores e registro de execuções                                                 |

A separação entre *Detectar* e *Investigar* reflete a distinção operacional discutida na

Seção 1.1: a primeira responde “é sintético?” com uma pontuação e um limiar; a segunda existe para o caso em que essa resposta precisa ser justificada — perícia, contestação, auditoria —, e por isso expõe as representações intermediárias em vez de apenas o veredito.

## B.4 Implantação

A distribuição é feita por contêineres, com perfis separados para inferência, treino e *benchmark*, cada um em variante CPU e NVIDIA. O perfil de inferência empacota **apenas os 11 modelos promovidos**, monta os diretórios de dados como volumes e desabilita a criação automática de modelos padrão — garantia de que o serviço executa os artefatos avaliados neste trabalho, e não um modelo qualquer criado no primeiro *boot*.

## B.5 Delimitação

Duas ressalvas evitam atribuir ao sistema mais do que ele faz.

A primeira é que os módulos de **explicabilidade** (SHAP e Grad-CAM, Seção 5.6) existem no repositório, mas operam como *ferramental de análise offline*, executado por um *script* dedicado (`scripts/reporting/run_shap_analysis.py`) e **não integrado à interface**. A visualização de importância disponível na tela de análise forense é uma agregação heurística de estatísticas de características, não uma atribuição por SHAP ou Grad-CAM; integrá-los ao produto permanece como trabalho futuro (Seção 7.3).

Cabe uma distinção que o texto não deve deixar ambígua: a Figura 10 **não** é produzida por esses módulos. Ela reporta importância por *permutação* (scikit-learn), e do pacote de explicabilidade reaproveita apenas a nomenclatura das 183 colunas do vetor tabular. SHAP e Grad-CAM não participam de nenhuma figura ou métrica reportada neste trabalho.

A segunda é que o sistema não foi submetido a avaliação de usabilidade nem a teste de carga. As latências reportadas na Seção 6.4 medem o *forward* do modelo, não o tempo de resposta ponta a ponta da aplicação, que inclui transporte, decodificação do áudio e serialização. O sistema é, portanto, uma **implementação de referência** — demonstra que o protocolo de avaliação se traduz em um serviço operável e reproduzível —, e não um produto validado em campo.

## C Implementação de Referência

A implementação avaliada neste trabalho corresponde ao *pipeline* real do repositório, e não a um exemplo isolado de código. A Tabela 24 registra os módulos efetivamente usados para treinamento, inferência, cálculo de métricas e geração dos relatórios.

**Tabela 24:** Módulos reais do *pipeline* referenciados pelo *benchmark*.

| Etapa                   | Módulo                                                     | Responsabilidade                                                 |
|-------------------------|------------------------------------------------------------|------------------------------------------------------------------|
| Orquestração            | <code>benchmarks/runner.py</code>                          | Executa preparação, treino, inferência, métricas e persistência  |
| Entrada operacional     | <code>scripts/benchmark/run_benchmark.py</code>            | Rodadas individuais ou múltiplas via linha de comando            |
| Treinamento             | <code>app/domain/services/training_service.py</code>       | Cria o modelo, aplica callbacks, treina e salva arquivos gerados |
| Fábrica de arquiteturas | <code>app/domain/models/architectures/factory.py</code>    | Normaliza aliases e instancia as arquiteturas avaliadas          |
| Carregamento de modelos | <code>app/domain/services/detection/model_loader.py</code> | Recarrega modelos Keras/sklearn e contratos de entrada           |
| Predição                | <code>app/domain/services/detection/predictor.py</code>    | Executa inferência padronizada e aplicação de limiar             |
| Serviço de detecção     | <code>app/domain/services/detection_service.py</code>      | Fachada usada pela API/Gradio para inferência                    |
| Métricas                | <code>app/domain/models/training/metrics.py</code>         | Calcula métricas, curvas, EER e matriz de confusão               |
| Relatórios              | <code>benchmarks/report.py</code>                          | Consolida tabelas, figuras e Markdown para o TCC                 |

Neste contexto, o *pipeline* de detecção de áudio sintético descrito neste trabalho corresponde à implementação de referência desenvolvida pelo autor como parte deste Trabalho de Conclusão de Curso.

## D Comandos de Execução e Reprodutibilidade

Os comandos a seguir sintetizam o fluxo operacional usado para repetir o *pipeline*, executar modelos isolados e gerar os arquivos do TCC.

- Preparar pastas locais:

```
python main.py -bootstrap-dirs
```

- Iniciar a interface Gradio:

```
python main.py -gradio
```

- Preservar o dataset original consolidado:

```
dir data\datasets\benchmark_dataset_15k.npz
```

- Executar o *pipeline* completo do TCC com o NPZ original:

```
python scripts/benchmark/run_tcc_pipeline.py
-full-benchmark -epochs 100 -device-profile gpu
-npz data/datasets/benchmark_dataset_15k.npz
```

- Executar benchmark completo com o NPZ consolidado:

```
python scripts/benchmark/run_benchmark.py -full -epochs 100
-device-profile gpu
-dataset data/datasets/benchmark_dataset_15k.npz
```

*WavLM Original e HuBERT Original usam um runner PyTorch/**transformers** dedicado, com backbone SSL real:*

```
python scripts/benchmark/run_wavlm_original_benchmark.py
-architecture wavlm
-dataset data/datasets/benchmark_dataset_15k.npz
python scripts/benchmark/run_wavlm_original_benchmark.py
-architecture hubert
-dataset data/datasets/benchmark_dataset_15k.npz
```

- **Executar benchmark de um modelo individual:**

```
python scripts/benchmark/run_benchmark.py -model Conformer -epochs 100
-device-profile gpu
-dataset data/datasets/benchmark_dataset_15k.npz
```

- **Executar testes automatizados:**

```
pytest tests/
```

- **Compilar o TCC:**

```
pdflatex main.tex
```